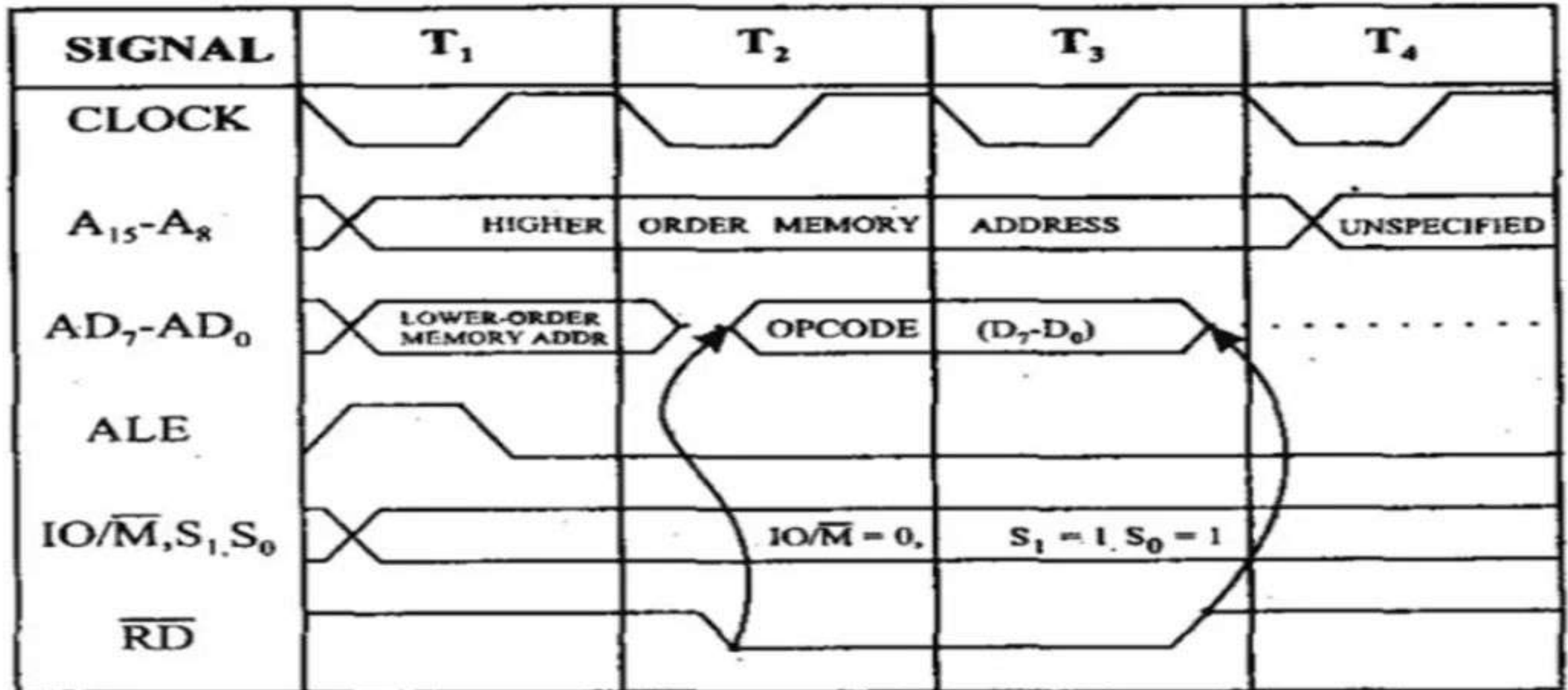


Timing Diagram Of Instruction 8085 Microprocessor

Doaa Yousif

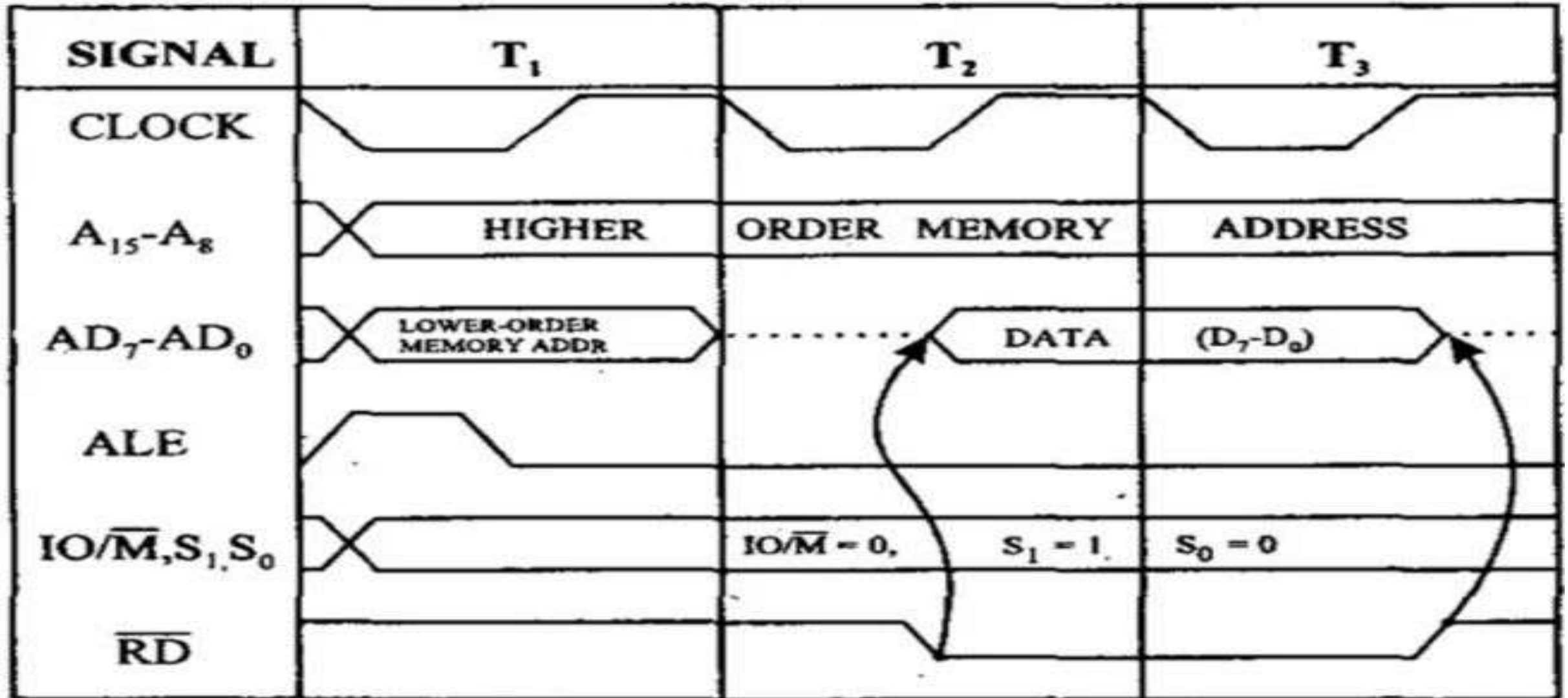
Op-Code Fetch cycle

4 T-State



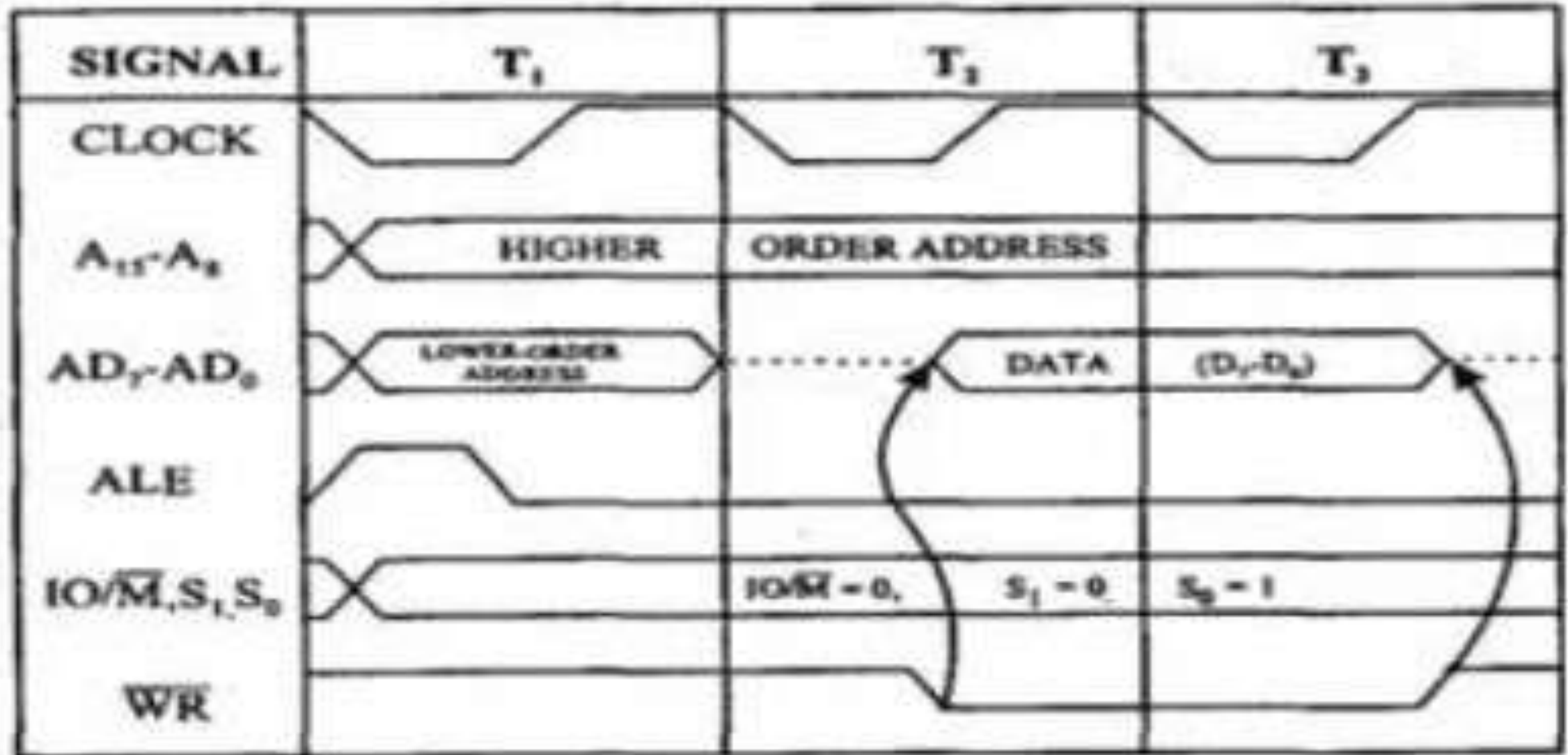
Memory Read cycle

3 T-State



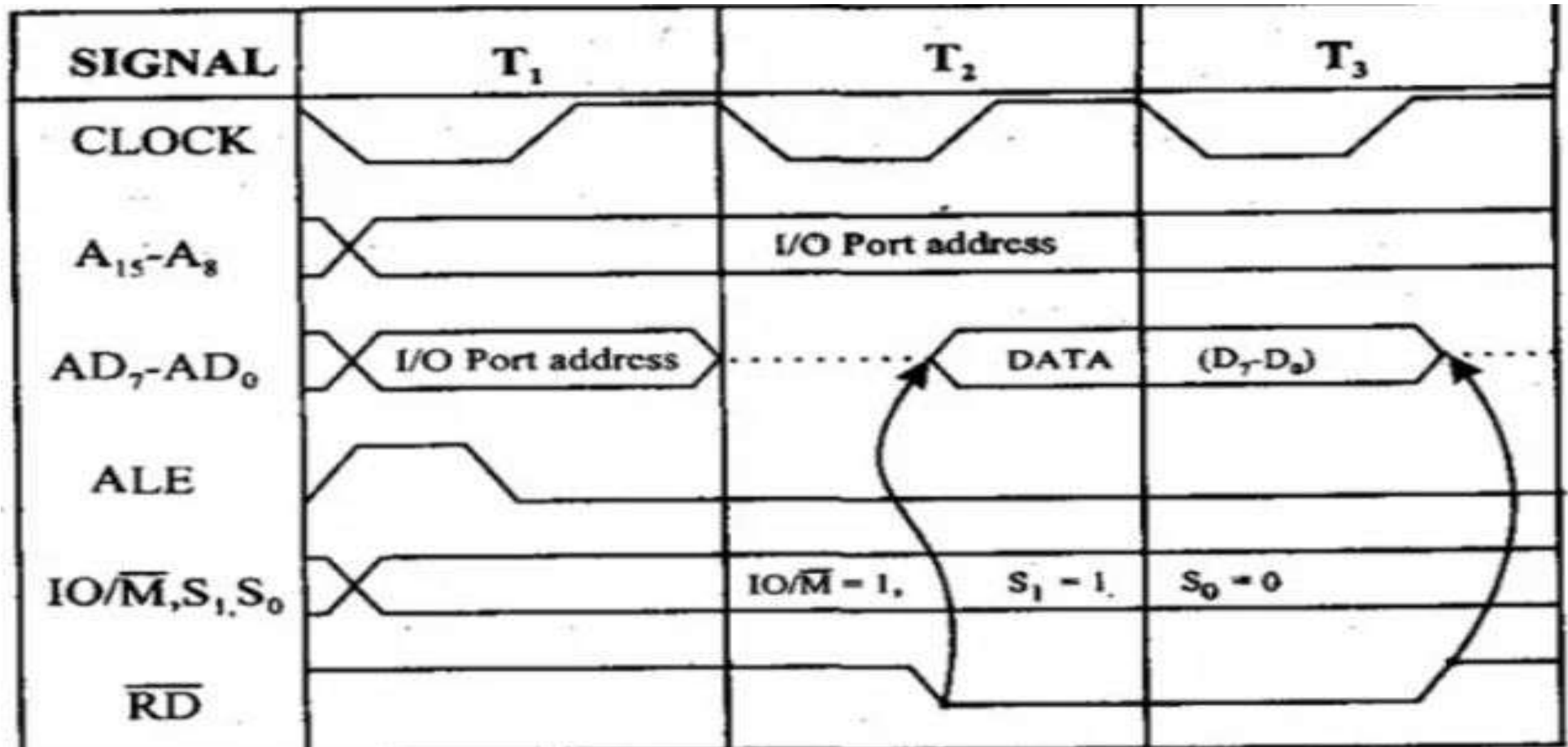
Memory Write cycle

3 T-State



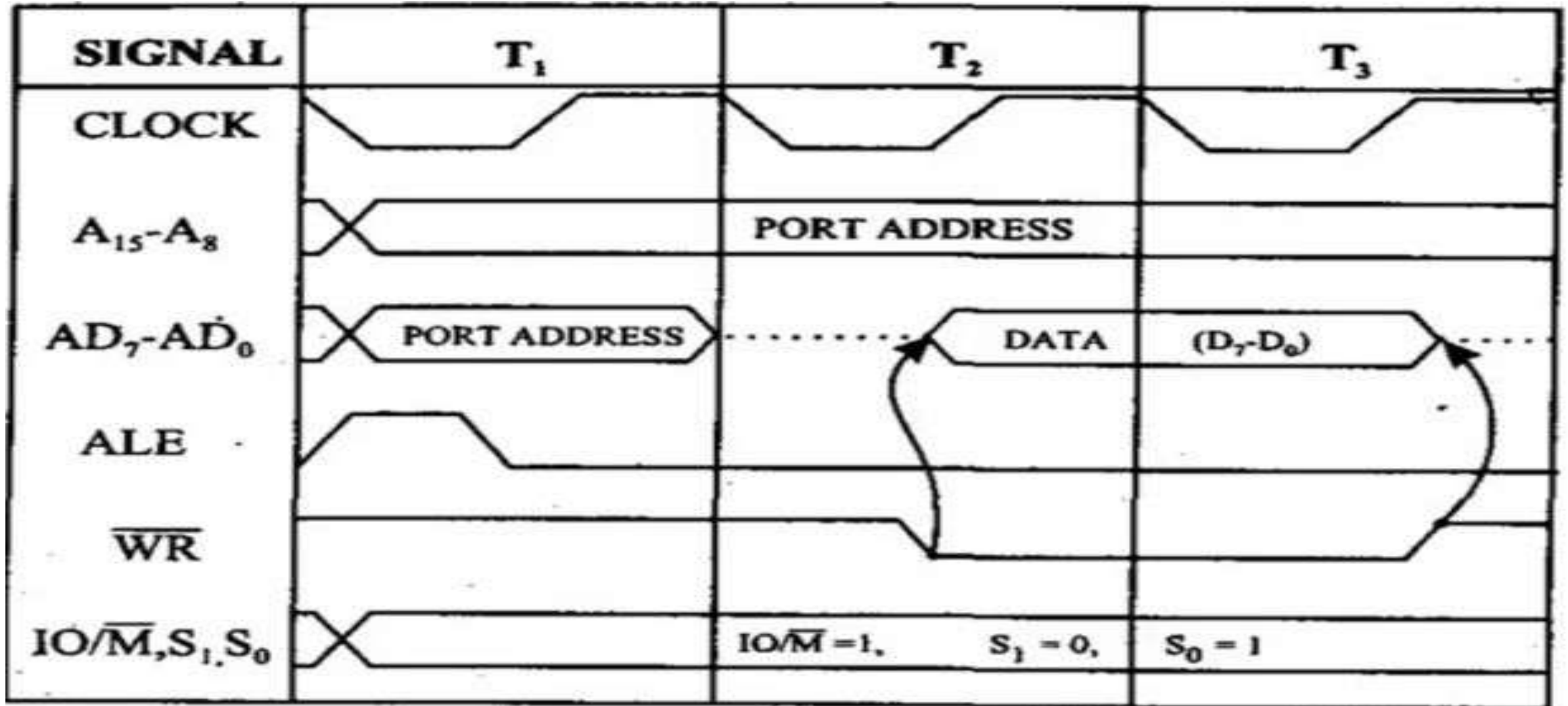
I/O Read cycle

3 T-State



I/O Write cycle

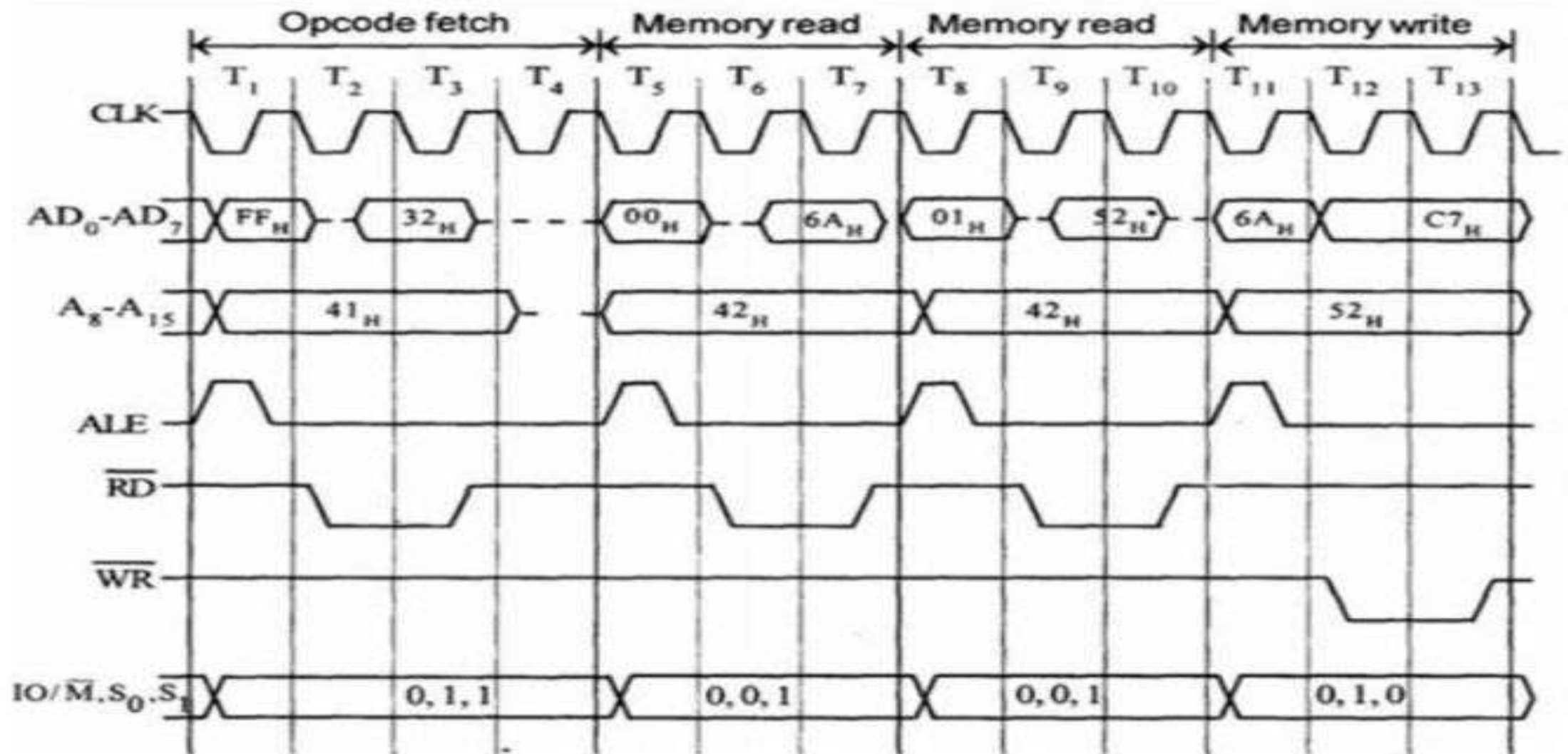
3 T-State



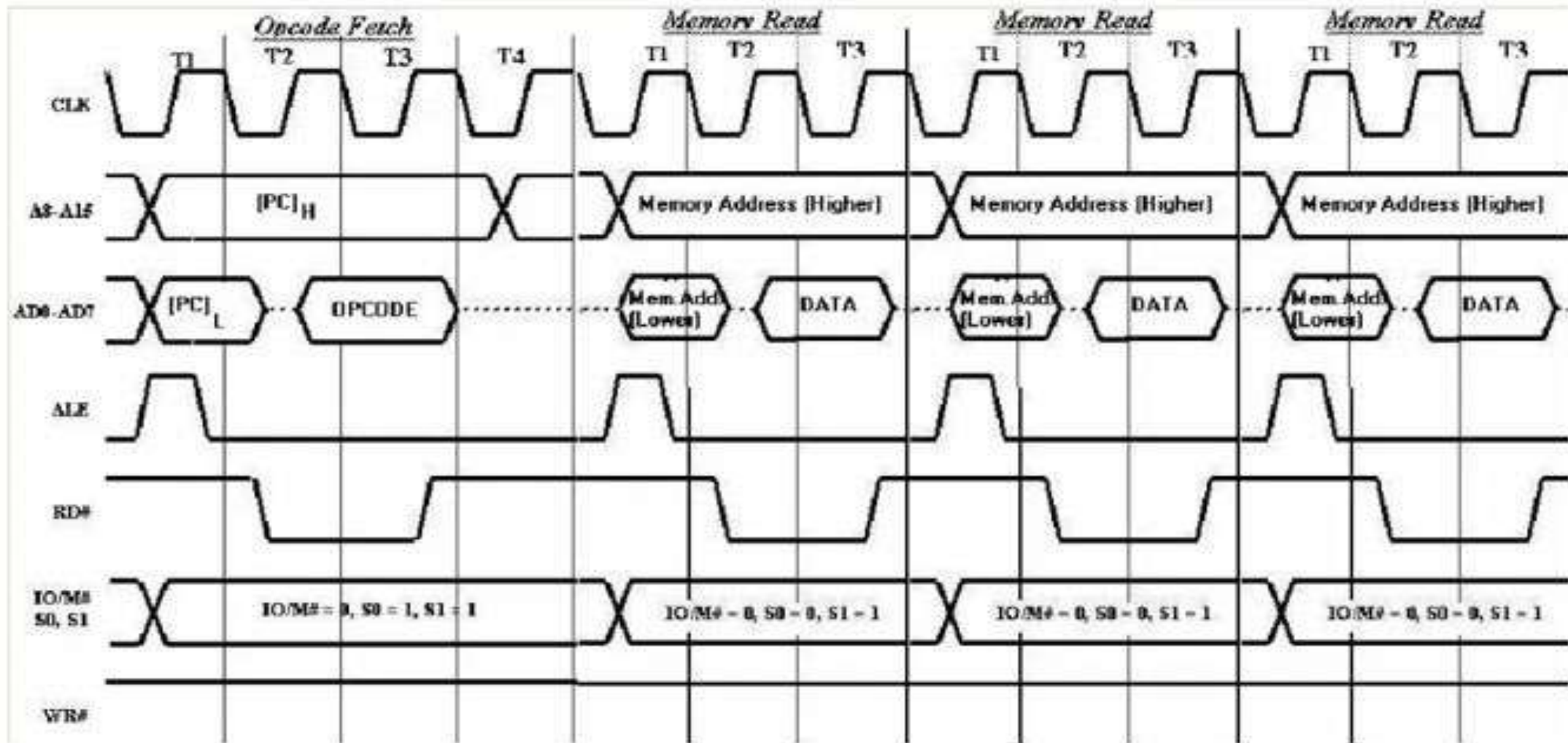
Signal State

IO/M(Active Low)	S1	S2	Data Bus Status(Output)
0	0	0	Halt
0	0	1	Memory WRITE
0	1	0	Memory READ
1	0	1	IO WRITE
1	1	0	IO READ
0	1	1	Opcode fetch
1	1	1	Interrupt acknowledge

STA 526Ah



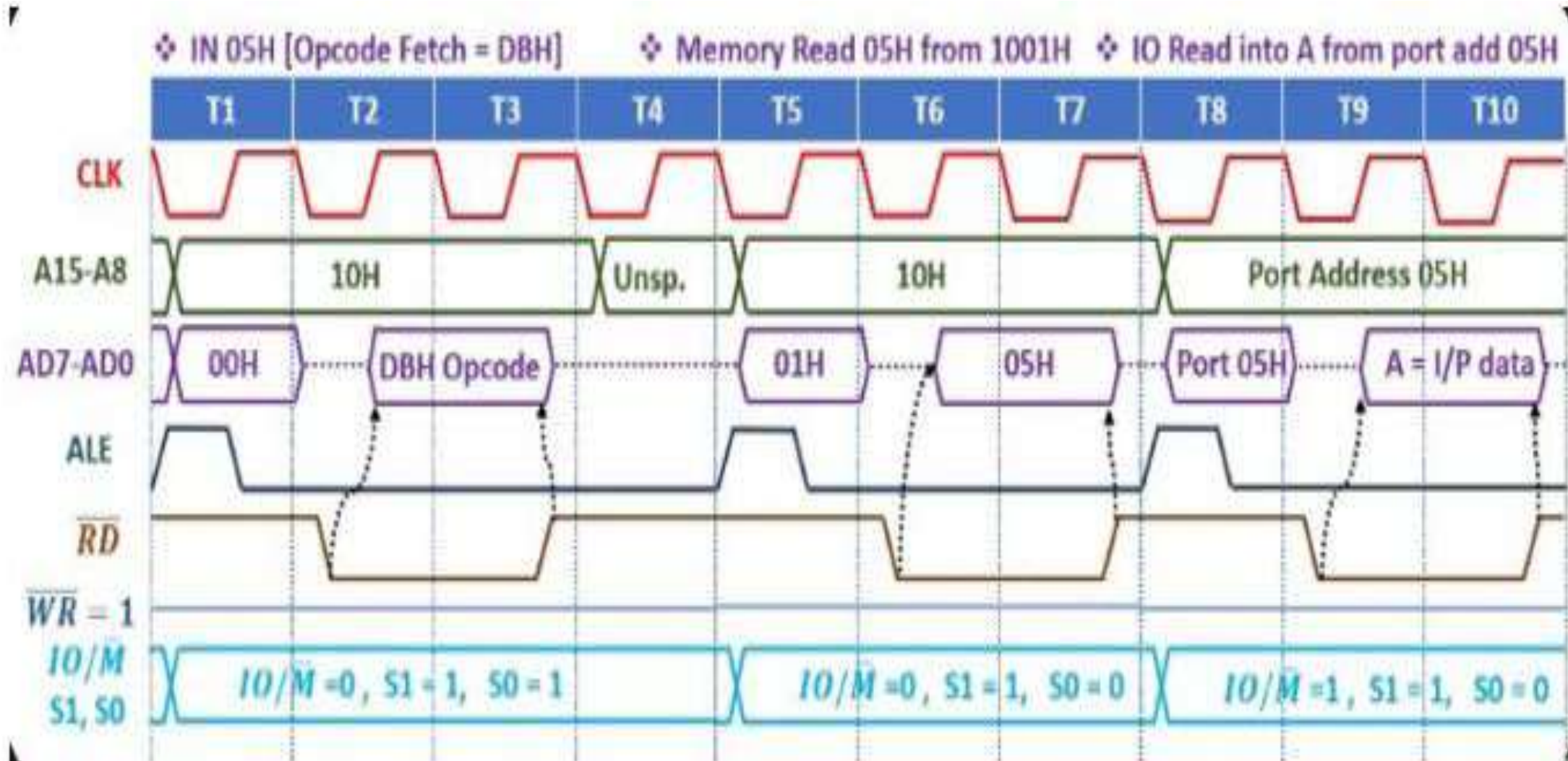
LDA 16bit



1000

IN 05h

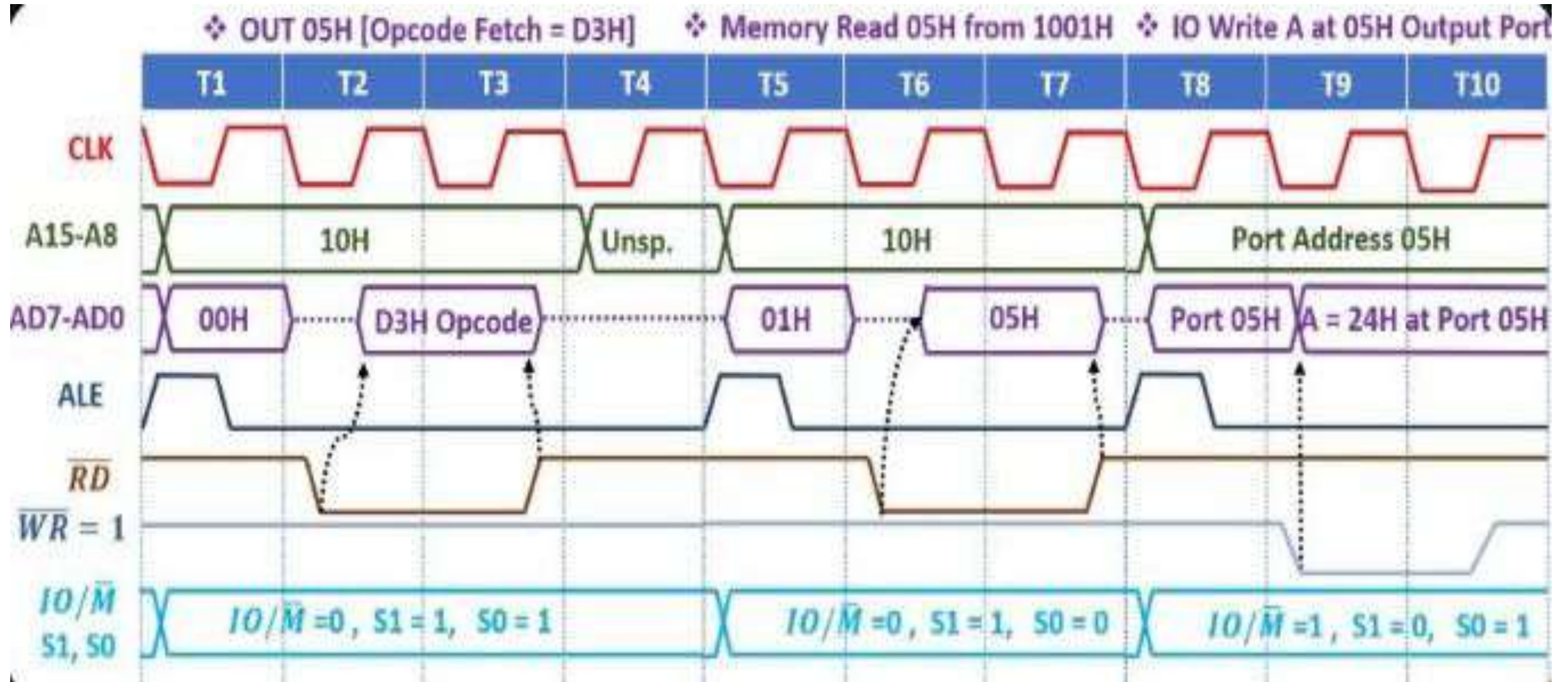
DB



1000

OUT 05h

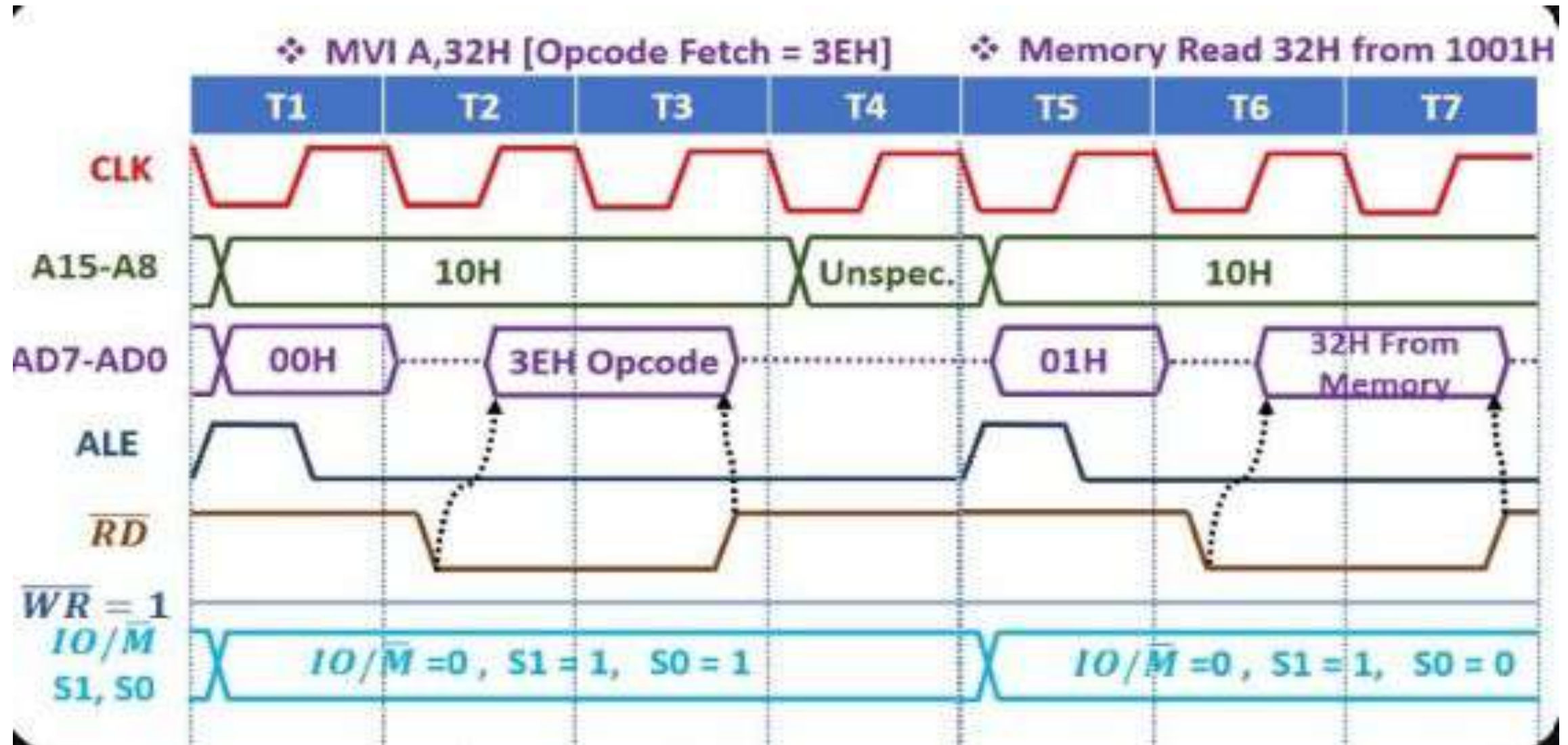
D3



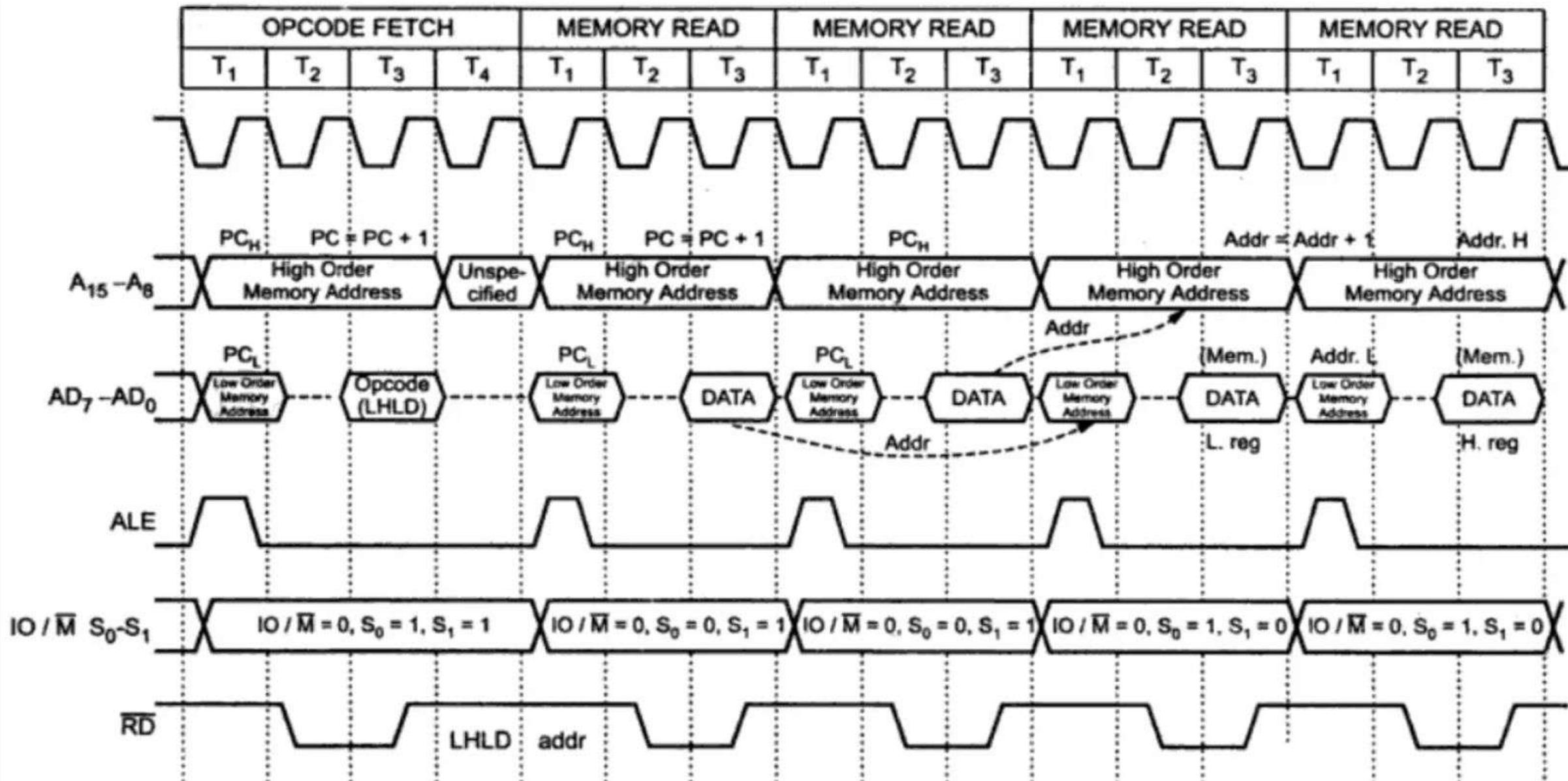
1000

MVI A,32h

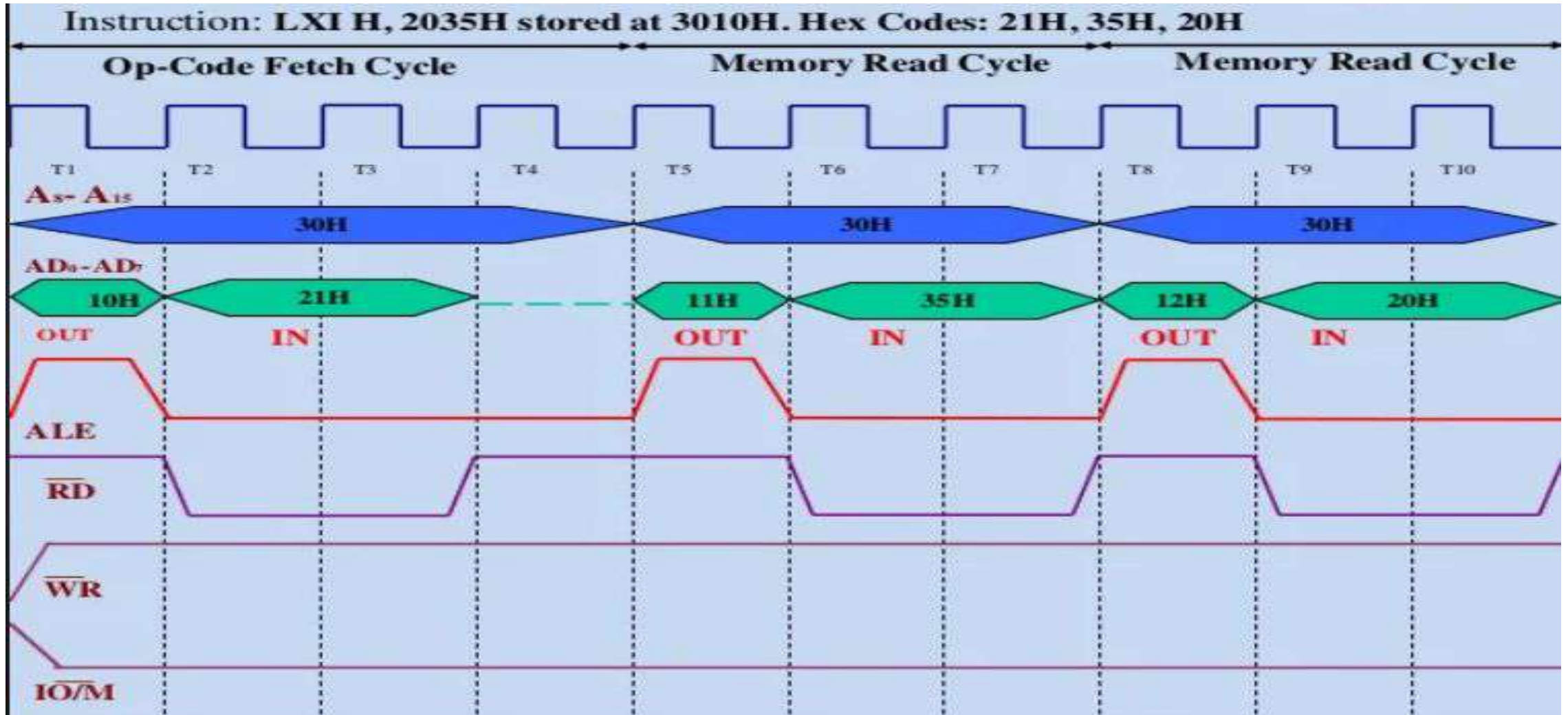
3E



LHLD



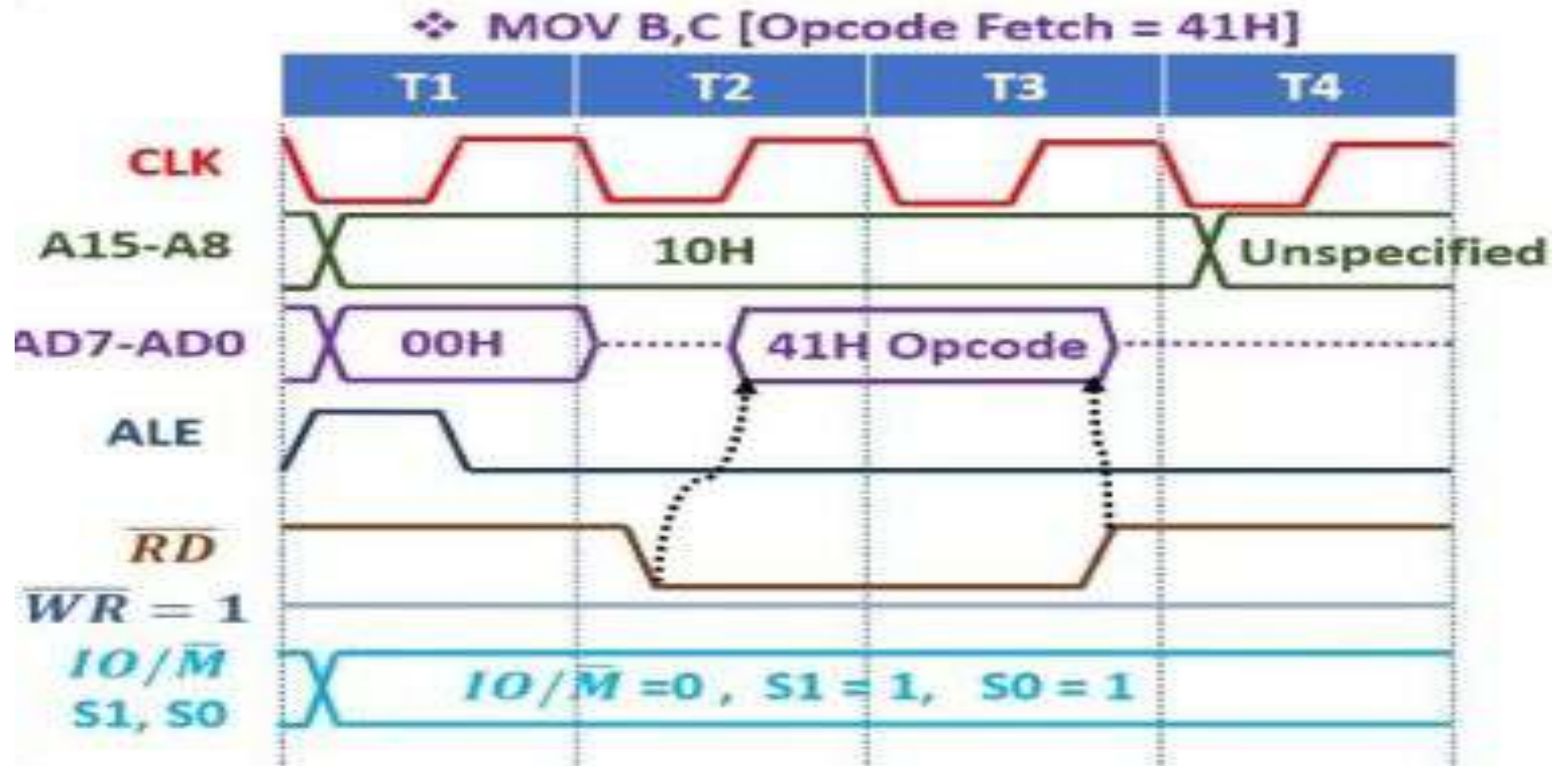
3010 LXI H,2035h 21



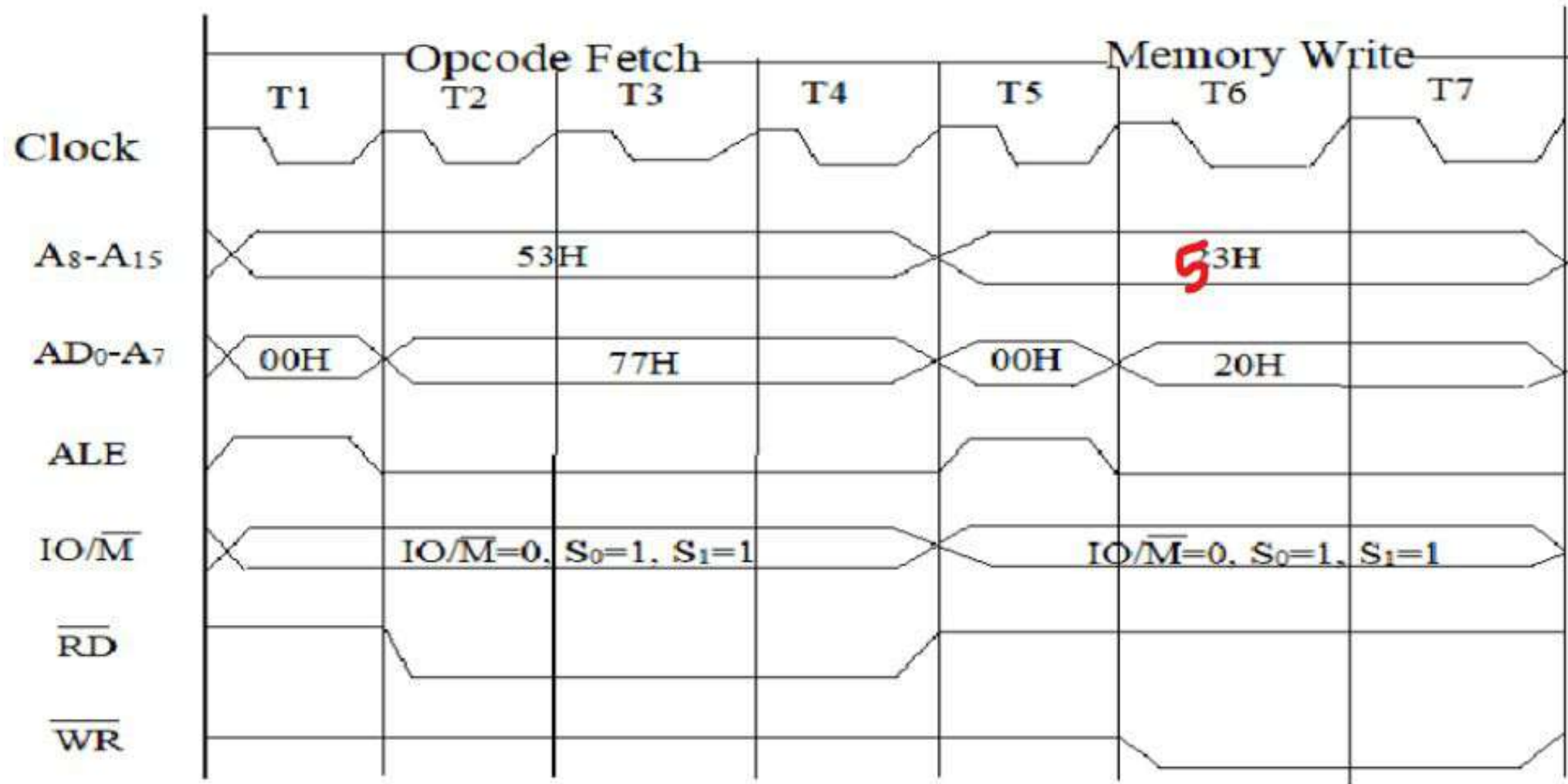
1000

MOV B,C

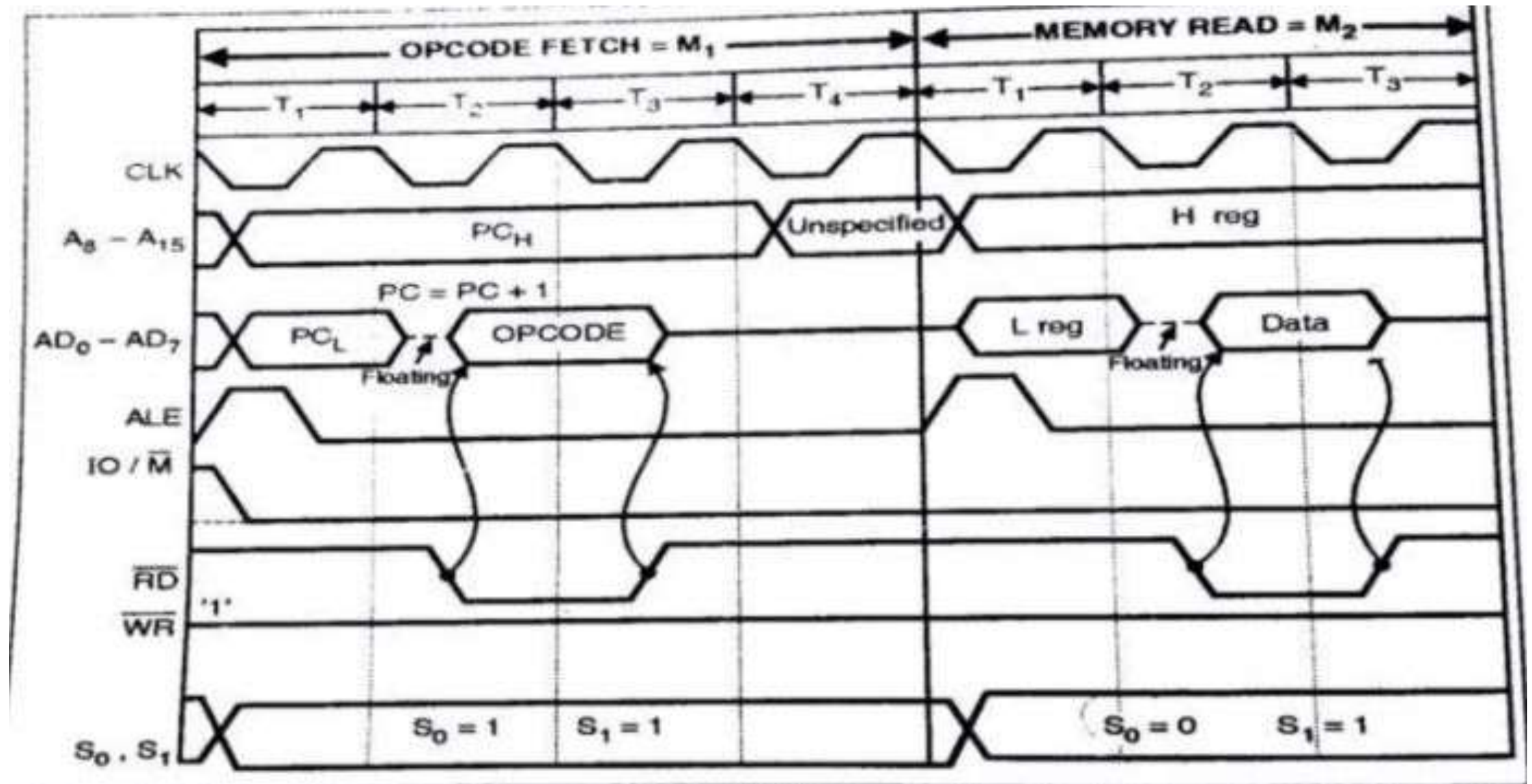
41



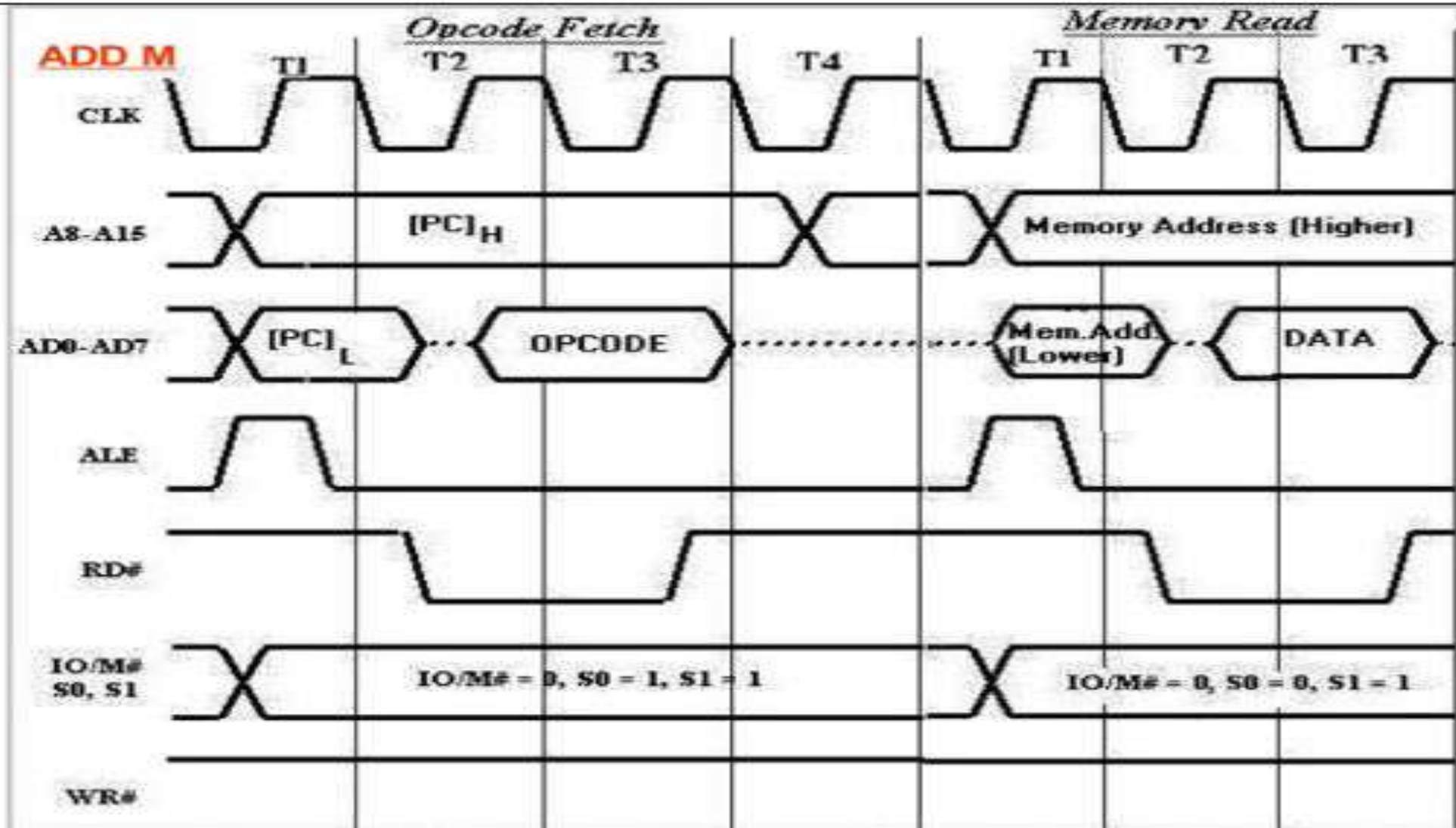
5300 MOV M,A 77 , A=20h



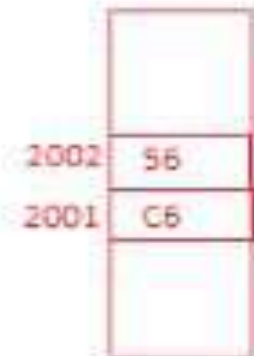
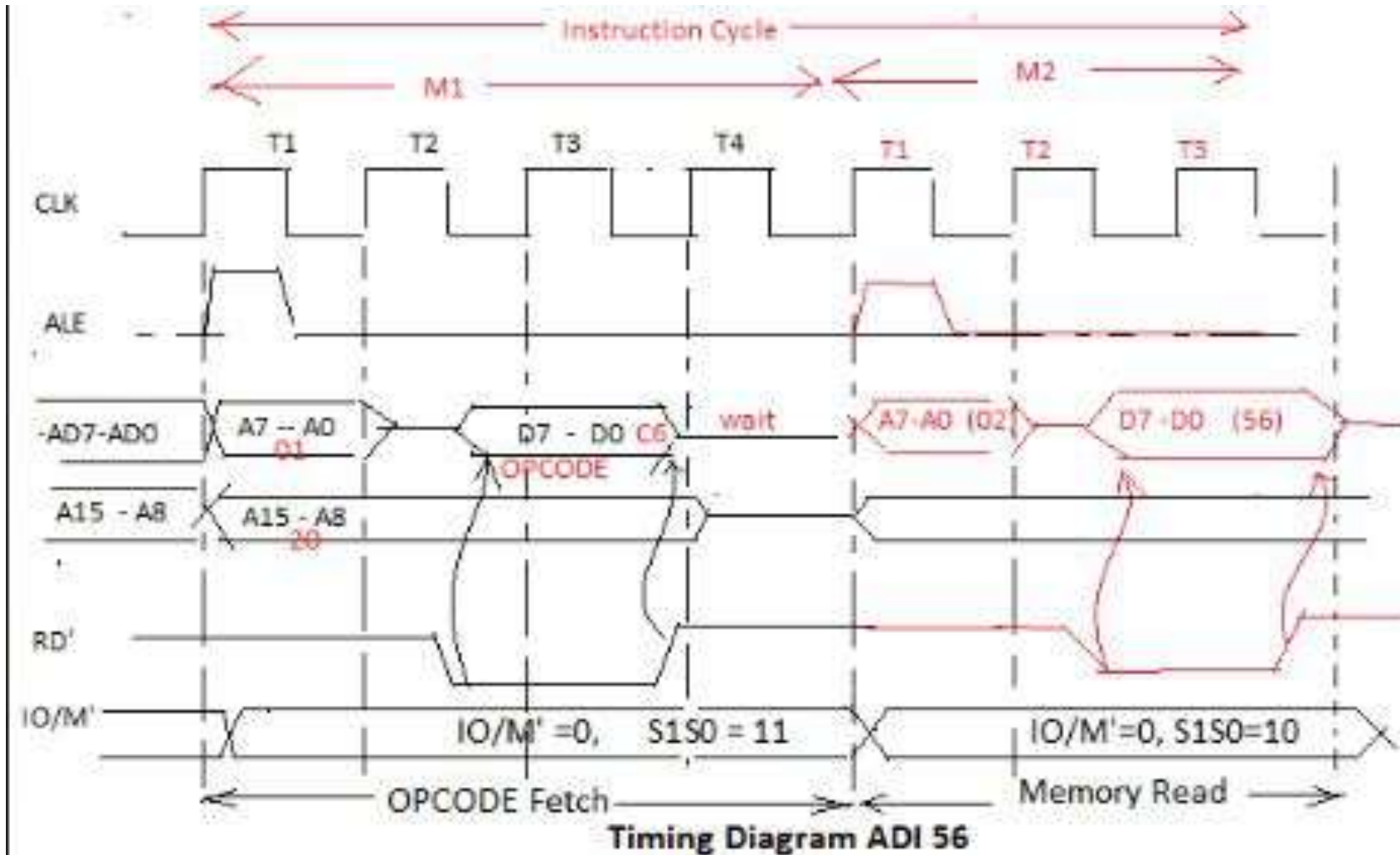
MOV C,M



ADD M



2001 ADI 56 C6



Instruction cycle

- 1 - Instr fetch
- 2 - Instr decode
- 3 - data fetch
- 4 - execute

Two Machine Cycle Instruction

Instruction	Machine Cycle 1	Machine Cycle 2
MVI Rd, DATA	F	R
MOV R, M	F	R
MOV M, R	F	W
ADI DATA	F	R
ACI DATA	F	R
SUI DATA	F	R
SBI DATA	F	R
ANI DATA	F	R

Code for 8085 Instruction

00 NOP	2C INR L †	58 MOV E, B	83 ADD E *	AF XRA A *	DB IN b
01 LXI B, d	2D DCR L †	59 MOV E, C	84 ADD H *	B0 ORA B *	DC CC a
02 STAX B	2E MVI L, b	5A MOV E, D	85 ADD L *	B1 ORA C *	DD [JNU]
03 INX B	2F CMA	5B MOV E, E	86 ADD M *	B2 ORA D *	DE SBI b *
04 INR B †	30 SIM	5C MOV E, H	87 ADD A *	B3 ORA E *	DF RST 3
05 DCR B †	31 LXI SP, d	5D MOV E, L	88 ADC B *	B4 ORA H *	E0 RPO
06 MVI B, b	32 STA a	5E MOV E, M	89 ADC C *	B5 ORA L *	E1 POP H
07 RLC †	33 INX SP	5F MOV E, A	8A ADC D *	B6 ORA M *	E2 JPO a
08 [DSUB] *	34 INR M †	60 MOV H, B	8B ADC E *	B7 ORA A *	E3 XTHL
09 DAD B †	35 DCR M †	61 MOV H, C	8C ADC H *	B8 CMP B *	E4 CPO a
0A LDAX B	36 MVI M, b	62 MOV H, D	8D ADC L *	B9 CMP C *	E5 PUSH H
0B DCX B	37 STC †	63 MOV H, E	8E ADC M *	BA CMP D *	E6 ANI b
0C INR C †	38 [LDSI]	64 MOV H, H	8F ADC A *	BB CMP E *	E7 RST 4
0D DCR C †	39 DAD SP	65 MOV H, L	90 SUB B *	BC CMP H *	E8 RPE
0E MVI C, b	3A LDA a	66 MOV H, M	91 SUB C *	BD CMP L *	E9 PCHL
0F RRC †	3B DCX SP	67 MOV H, A	92 SUB D *	BE CMP M *	EA JPE a
10 [AHRL] †	3C INR A †	68 MOV L, B	93 SUB E *	BF CMP A *	EB XCHG
11 LXI D, d	3D DCR A †	69 MOV L, C	94 SUB H *	C0 RNZ	EC CPE a
12 STAX D	3E MVI A, b	6A MOV L, D	95 SUB L *	C1 POP B	ED [LHLX]
13 INX D	3F CMC †	6B MOV L, E	96 SUB M *	C2 JNZ a	EE XRI b *
14 INR D †	40 MOV B, B	6C MOV L, H	97 SUB A *	C3 JMP a	EF RST 5
15 DCR D †	41 MOV B, C	6D MOV L, L	98 SBB B *	C4 CNZ a	F0 RP
16 MVI D, b	42 MOV B, D	6E MOV L, M	99 SBB C *	C5 PUSH B	F1 POP PSW *
17 RAL †	43 MOV B, E	6F MOV L, A	9A SBB D *	C6 ADI b *	F2 JP a
18 [RDEL] ^{cy, v}	44 MOV B, H	70 MOV M, B	9B SBB E *	C7 RST 0	F3 DI
19 DAD D †	45 MOV B, L	71 MOV M, C	9C SBB H *	C8 RZ	F4 CP a
1A LDAX D	46 MOV B, M	72 MOV M, D	9D SBB L *	C9 RET	F5 PUSH PSW
1B DCX D	47 MOV B, A	73 MOV M, E	9E SBB M *	CA JZ a	F6 ORI b *
1C INR E †	48 MOV C, B	74 MOV M, H	9F SBB A *	CB [RSTV]	F7 RST 6
1D DCR E †	49 MOV C, C	75 MOV M, L	A0 ANA B *	CC CZ a	F8 RM
1E MVI E, b	4A MOV C, D	76 HLT	A1 ANA C *	CD CALL a	F9 SPHL
1F RAR †	4B MOV C, E	77 MOV M, A	A2 ANA D *	CE ACI b *	FA JM a
20 RIM	4C MOV C, H	78 MOV A, B	A3 ANA E *	CF RST 1	FB EI
21 LXI H, d	4D MOV C, L	79 MOV A, C	A4 ANA H *	D0 RNC	FC CM a
22 SHLD a	4E MOV C, M	7A MOV A, D	A5 ANA L *	D1 POP D	FD [JUI]
23 INX H	4F MOV C, A	7B MOV A, E	A6 ANA M *	D2 JNC a	FE CPI b *
24 INR H †	50 MOV D, B	7C MOV A, H	A7 ANA A *	D3 OUT b	FF RST 7
25 DCR H †	51 MOV D, C	7D MOV A, L	A8 XRA B *	D4 CNC a	b = byte data
26 MVI H, b	52 MOV D, D	7E MOV A, M	A9 XRA C *	D5 PUSH D	d = 16b data
27 DAA *	53 MOV D, E	7F MOV A, A	AA XRA D *	D6 SUI b *	a = 16b address
28 [LDHI]	54 MOV D, H	80 ADD B *	AB XRA E *	D7 RST 2	flags:
29 DAD H †	55 MOV D, L	81 ADD C *	AC XRA H *	D8 RC	* = affects all
2A LHLD a	56 MOV D, M	82 ADD D *	AD XRA L *	D9 [SHLX]	† = carry only
2B DCX H	57 MOV D, A	83 ADD E *	AE XRA M *	DA JC a	‡ = all but carry

Instruction Timing (T States)

Data Transfer	CMP r	4	Control
MOV rd, rs	CMP M	7	DI 4
MOV M, r		7	EI 4
MOV r, M	16 bit	7	NOP 4
XCHG	DAD x	10	HLT 5
MVI r	INX x	6	
MVI M	DCX x	6	Stack
LXI x		10	PUSH 12
	Rotate		POP 10
	RLC	4	
Load/Store	RRC	4	Branch
LDAX x	RAL	4	RST x 12
LDA	RAR	4	JMP 10
STAX x		7	Jx 7/10
SHLD	Immediate	16	CALL 18
STA	ADI	7	Cx 9/18
	ACI	7	RET 10
Arith & Logic	SUI	7	Rx 6/12
ADD r	SBI	7	
ADD M	ANI	7	T-State Time
ADC r	XRI	7	CPU f tcyc
ADC M	ORI	7	8.0MHz 125n5
SUB r	CPI	7	6.0 166.7
SUB M		7	5.0 200
SBB r	Specials	4	4.0 250
SBB M	DAA	4	3.58 279
INR R	CMA	4	3.125 320
INR M	STC	4	3.072 325.5
DCR r	CMC	4	3.0 333.3
DCR M		10	2.5 400
ANA r	I/O	4	2.0 500
ANA M	OUT	10	1.25 800
XRA r	IN	10	1.2 833.3
XRA M	RIM	4	1.0 1000
ORA r	SIM	4	0.75 1333
ORA M		7	

Accumulator Operations

Instruction	Code	Function
XRA A	AF	Clear A and Clear Carry
CMA	2F	Complement Accumulator
ORA A	B7	Clear Carry
CMC	3F	Complement Carry
STC	37	Set Carry
RLC	7	Rotate Left, MSB=CY
RRC	0F	Rotate Right, LSB=CY
RAL	17	Rotate Left Thru Carry
RAR	1F	Rotate Right Thru Carry

After RIM:

SID	I7.5	I6.5	I5.5	IE	M7.5	M6.5	M5.5
-----	------	------	------	----	------	------	------

SID = Serial In Data

Ix.5 = Interrupt Pending

IE = Interrupt Enable Flag

Mx.5 = Interrupt Masks for external lines

Before SIM:

SOD	SOE	---	R7.5	MSE	M7.5	M6.5	M5.5
-----	-----	-----	------	-----	------	------	------

SOD = Serial Out Data

SOE = Serial Out Enable (enable SOD output)

R7.5= Reset (clear) RST 7.5 Interrupt

MSE = Mask Set Enable (1 to set new masks)

Mx.5= New RSTx.5 Mask Setting (1 to enable)

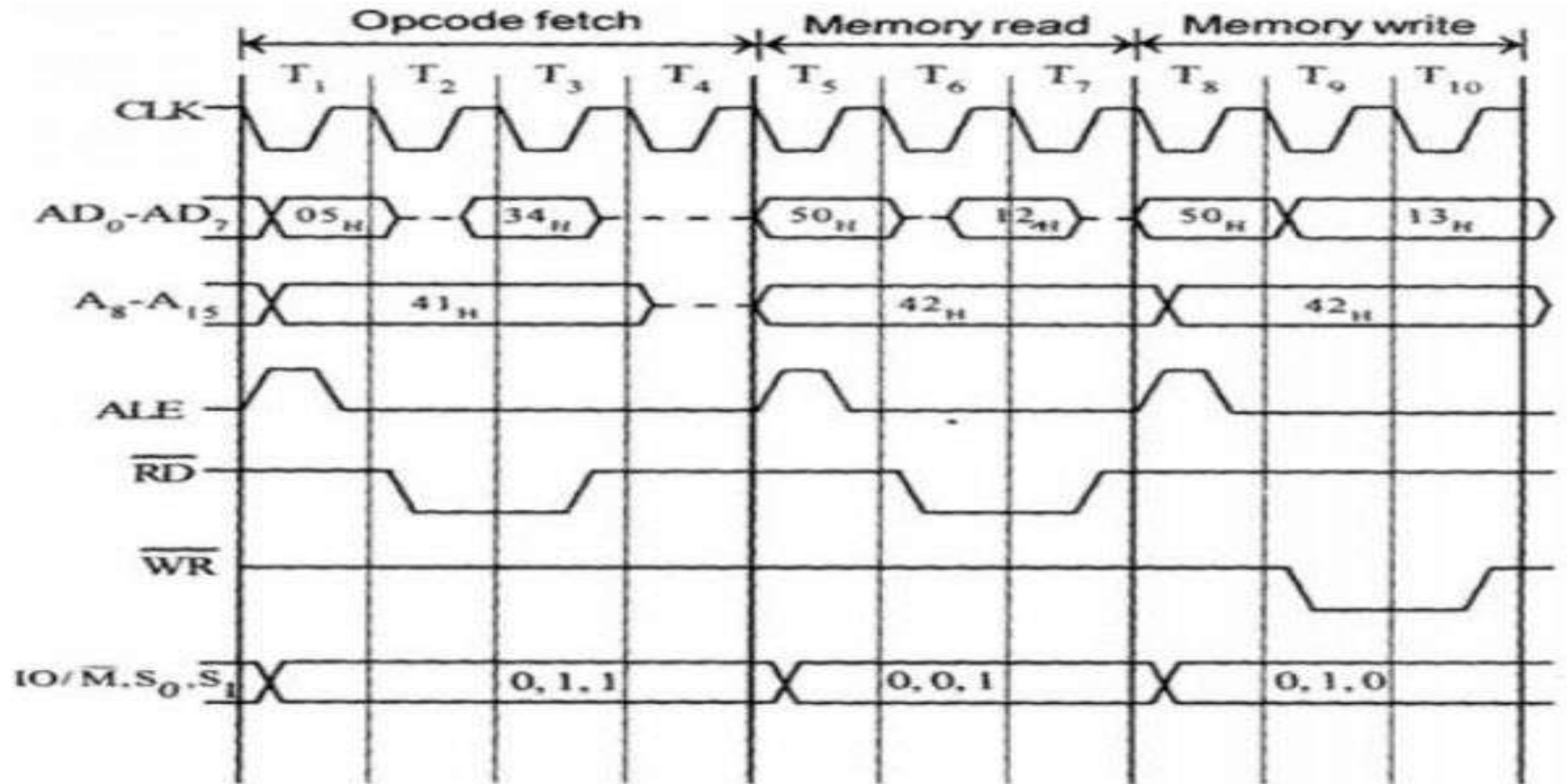
Note: RST5.5, RST6.5, RST7.5 are masked by default. Masks must be cleared before use.

Visit <http://saundby.com/> for more 8085 info.

Instruction	Data	Machine Cycles	T-States	Operation (Opcode, R,W)
MOV A, B	Register	1	4	Opcode
MOV A, M	Memory (M)	2	7	Opcode, R
MOV M, REG	Memory (M)	2	7	Opcode, W
MVI A, 8-bit	Immediate	2	7	Opcode, R
LDA 16-bit	Memory	4	13	Opcode, R,R,R
STA 16-bit	Memory	4	13	Opcode, R,R,W,
IN 8-bit	I/O Port	3	10	Opcode, R,R
OUT 8-bit	I/O Port	3	10	Opcode, R,W
ADD B	Register	1	4	Opcode
ADI 8-bit	Immediate	2	7	Opcode, R
JMP 16-bit	Address	3	10	Opcode, R,R
CALL 16-bit	Address	5	18	Opcode, W,W,R,R
RET	None	3	10	Opcode, R,R
PUSH B	Register	3	10	Opcode, W,W
POP B	Register	3	10	Opcode, R,R
NOP	None	1	4	Opcode

Instruction	Data	Machine Cycles	T-States	Operation (Opcode, R,W)
ORA B	Register	1	4	Opcode
ORI 8-bit	Immediate	2	7	Opcode, R
XCHG	Register Pair	1	4	Opcode
CMP B	Register	1	4	Opcode
INR M	Memory	3	10	Opcode, R,W
INR REG	Register	1	4	Opcode
DCR REG	Register	1	4	Opcode
DCR M	Memory	3	10	Opcode, R,W
DCX ,INX	Pair register	1	5	Opcode, pair reg
CPI 8-bit	Immediate Data	2	7	Opcode, R
CMP REG	Register	1	4	Opcode
CMC	Complement Carry	1	4	Opcode
STC	Set Carry	1	4	Opcode
RAL,RLC	Accumulator	1	4	Opcode
RAR,RRC	Accumulator	1	4	Opcode

4105 INR M 34 , M=12



* Logic Group *

ANA And (R) or (M) with Accumulator

يسوي عملية AND على A وية رجستر اوخلية ذاكرة عنوانها مخزون ب (HL)

عملية AND يعطى نتيجة برائ الطريقة

$$1 \text{ AND } 1 = 1$$

$$1 \text{ AND } 0 = 0$$

$$0 \text{ AND } 1 = 0$$

$$0 \text{ AND } 0 = 0$$

Ex:

$$A = 48$$

$$L = 52$$

خزن القيمة من داخل الaccumulator

ANA L

التي اع يصير برنا الاعاز منسقل

$$A = A \times L$$

$$[48] \quad 0100 \quad 1000$$

$$[52] \quad 0101 \quad 0010 \quad \text{AND}$$

$$[40] \quad 0100 \quad 0000$$

$$4 \quad 0$$

هنا لفلاغات اذا كالتا بالسؤال نملأهم

Carry Flag دائماً 0 { ثوابت * حفظ *

Auxiliary Carry دائماً 1 *

بس لبقية نتوسخ واحدة نياوع على الناتج مثل قبل
فجزء السؤال

$$Z = 0 \quad \text{لان الناتج هو صفر}$$

$$P = 0 \quad \text{لان عدد لواحدات فردية}$$

$$S = 0 \quad \text{لان افرعهم جانب صفر لهم}$$

$$A = 40 \quad \text{بعد التنفيذ}$$

Ex2: AND A with memory [9001]

حتى نسوي AND لـ acc وية محتويات ذاكرة
راغ نسوي مثل قبل ونخلي العنوان بالا HL

LXI H, 9001
ANA M

واذا اعتبرنا انو [9001] = 52 و A = 48
فراغ يطلع نفس ناتج في قبله

ANI يسوي AND لـمين بس هاي الـمرة، الفرق
انو مداع سيقامل وية رجستر او ادي ستي راغ نخلي
وزاه قيمة مباشرة

يسوي عملية AND للقيمة المباشرة وية محتويات A ونخلي ناتج بالا A

Ex:

ANI 90

IF A = 84

A = A x 80

A قيمة [84]

1000 0100

[90] القيمة مباشرة

1001 0000

1000 0000
8 0

AND

بعد التنفيذ

A = 80

ونفس لتي

Carry Flag دائماً 0
AC Flag دائماً 1
AND لـتابة لكل عملية

CMA Complement Accumulator
إيعاز شغلة يُلجى 1's Complement للرجستر A بس

ال 1's Complement

شغلة المنطقية هو انقلب كل 1 سويه 0 والعكس

0 → 1
1 → 0
فراغ تكتب ناتج ال A ونحلي الناتج بال A

EX: IF A = 47

0100 0111 [47] قبل لتنفيذ

CMA

بعد لتنفيذ

كل رقم اخذته عكسه
1011 1000
B 8

A = B8

↑ قيمة A جديدة بعد لتنفيذ

CMC

Complement Carry Flag

هذا الايعاز يُلجى ال 1's Comp. لل Carry Flag
وسيتز على اي فلاح ناتج

يقلب قيمة ال Carry اذا كان عدده مكتوب

1 من شغله يصير 0 واذا كان 0 يصير 1

CMC

نكتبه وده لان شغله واحد

بس يقلب ال Carry Flag

وباقي الفلاغات باق تبقى مثل ما هي

CMP Compare (R) or (M) with A

يسوي مقارنة بين محتويات (ذاكرة أو رجسٹر)

وبعد، لفتحة ما، رجسٹر A

راج تأثر هاتج العملية على الفلاغات Cy و Z

Compare

Cy

Z

$A > B$ لأن ناتج طرح موجب بدون Cy

0

0

$A = B$ لأن ناتج طرح = صفر

0

1

$A < B$ لأن ناتج طرح سالب بدون Cy

1

0

بorrow
①
5 - 6 = -1
-1

توضيح مثير الغضب، لا قام

إذا جان $A = 5, B = 6$ if $1 \times$

Carry Flag = 1 لأن كولو borrow

Zero Flag = 0 لأن ناتج طرح

2* if $A = 5, B = 5$

5 - 5 = 0
0

Cy Flag = 0

Zero Flag = 1

لأن ناتج طرح borrow
لأن ناتج طرح

3* if $A = 6, B = 5$

6 - 5 = 1
1

Cy = 0 لأن كولو borrow

Zero Flag = 0 لأن ناتج طرح

CPI Compare A with Immediate data
نفس سجل قبل شوية فرقته لو هي بيقارن وية قنينة
مباشرة تنكتب وراه كبل بنفس طريقة قبل شوية

Compare	CY	Z
$A > data$	0	0
$A = data$	0	1
$A < data$	1	0

نفس السبب في شوية قبل شوية

Ex: if A 58

CPI 73 $\Rightarrow$

Z	CY
0	1

EX2: if A=50 & C = 43

CMP C $\Rightarrow$

Z	CY
0	0

ORA

يسوي عملية OR بين محتويات رجستر او ذاكرة محددة
والرجستر A

عملية OR منطقيا شغلها

$$1 \text{ OR } 1 = 1$$

$$0 \text{ OR } 1 = 1$$

$$1 \text{ OR } 0 = 1$$

$$0 \text{ OR } 0 = 0$$

ORA B

ينكتب ورة الايعاز في نوية نسوي OR واهي

Ex: if $A = 05$ & $B = 02$

ORA B

سوى عملية OR بين A و B

وختار الناتج بالرجستر A

[05] 0000 10101

[02] 0000 0010 OR

0000 0111
0 7

A = 07

بعد التنفيذ

AcFlag دائماً صفر
CyFlag دائماً صفر

ORI

يسوى عملية OR بنفس الطريقة قبل شوية بس الفرق

مراعيا عامل وية رجسترات اوزاكرة يسوى OR

وية الرقم الذي مكتوب وراء وية رجستر A

ORI 05

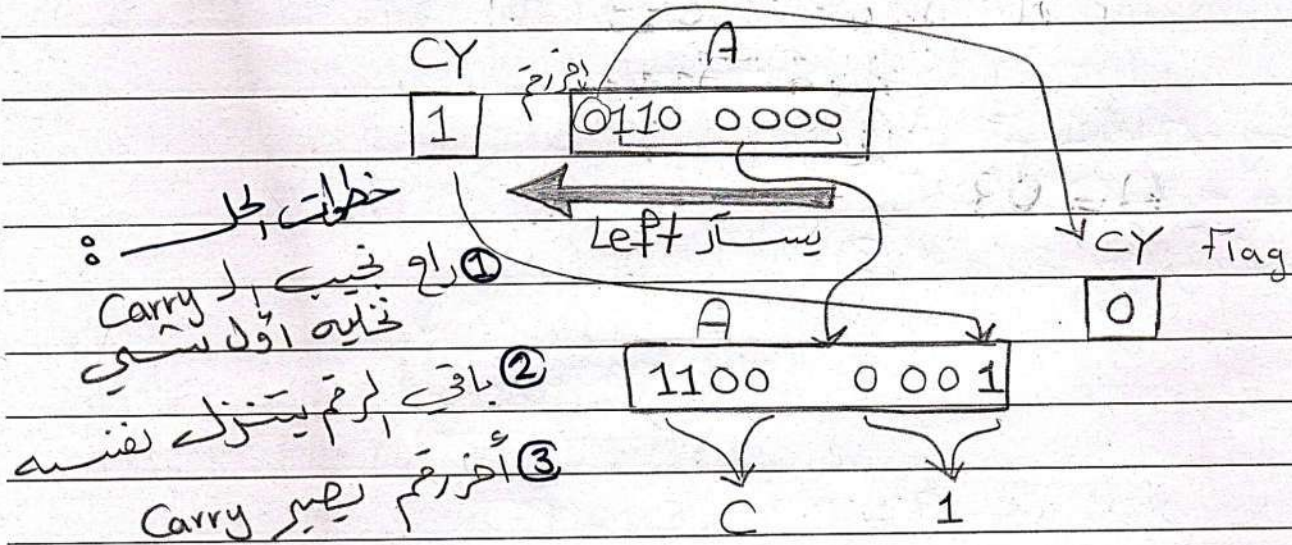
AC Flag و CyFlag دائماً صفر
باقى الفلات تتأثر

RAL Rotate Accumulator left through Carry

ليور الرقم الـ A للمرة وبأستعمال Carry Flag

Ex:

مثلاً عند $A = 60$ و $CarryFlag = 1$



RAR Rotate Accumulator Right through Carry

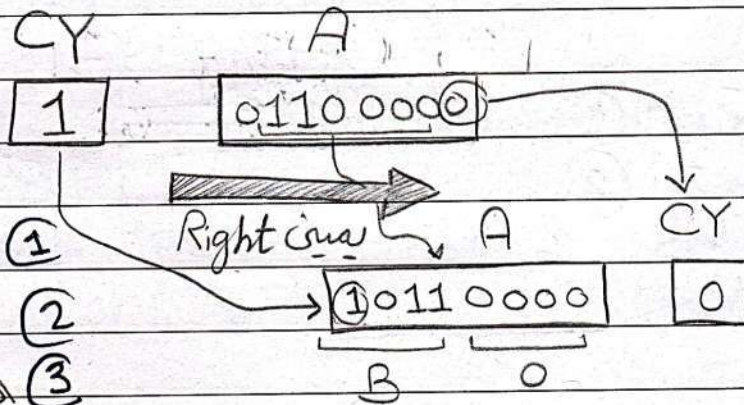
راح اندور الرقم للمرة واحدة وبأستعمال الـ Carry

Ex:

$A = 60$
 $Cy = 1$

الخطوات :

- ① أول رقم يصير Carry
- ② باقي الرقم ينزل لنفسه
- ③ الـ Carry القديم يصير آخر رقم



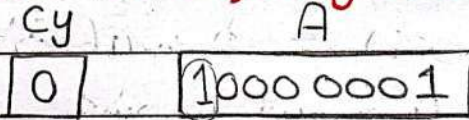
بعد تنفيذ $A = B0$, $Cy = 0$

(RLC) Rotate Accumulator Left

يدور A لليسرة بدون CarryFlag

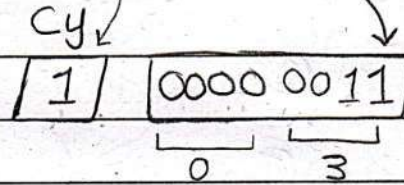
Ex:

if $A = 81$, $CyFlag = 0$



Left

- (1) اخذ رقم خلية CarryFlag وتأخذ نفسه خلية اول واحد
- (2) باقي الرقم راو ينزل نفسه



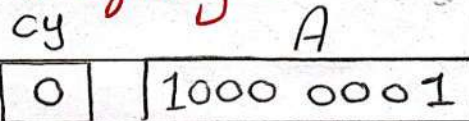
$A = 03$, $Cy = 1$ بعد التنفيذ

(RRC) Rotate Accumulator Right

يدور A الليسرة بدون CarryFlag

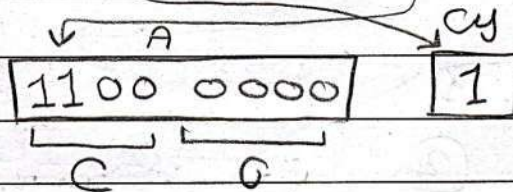
Ex:

if $A = 81$, $CyFlag = 0$



Right

- (1) اخذ رقم خلية Carry
 - (2) الرقم وراها راو ينزل
- طبيعة راو خلية اول واحد اخذ نفسه والباقي ينزل



$A = C0$, $Cy = 1$ بعد التنفيذ

STC

ايجاز شغل - ذات - بال Carry Flag واحد

Ex: if Cy Flag = 0

STC $\Rightarrow$ Cy Flag = 1

XRA

Exclusive OR (R) or (M) with A

يسوي XOR لرجستر او ذاكرة وية محتوي A

ذات الناتج بال A

شغل XOR منطقياً :

$$1 \text{ XOR } 1 = 0$$

$$1 \text{ XOR } 0 = 1$$

$$0 \text{ XOR } 1 = 1$$

$$0 \text{ XOR } 0 = 0$$

Ex: if A = A5 , B = B7

XRA B

↑ ينقلب وراها حرف لرجستر

1010 0101

1011 0111 XOR

0001 0010

1

2

A = 12 بعد التنفيذ

Cy Flag دائماً صفر

* * نأخذ ناكس Complement 1's للارقام بأستعمال XOR

بمنشوي XOR للرقم وية [FF]

XRI

ينقلص المربعة يسوي XOR لـ A وية قمية

مباشرة مكتوبة بالايجاز

والناتج يتخاف بالـ A

XRI 07

* Branch Group *

JMP

Unconditional Jump بدون شرط

يُنكَب الأيعاز بهاء الطريقة

JMP name

تتغلته (يكنز) لما كان له اسم اجنة كاتبه
فندل مينفذ البرنامج بالسرة من يلكة JMP او يطر
وين ميو جينة ، مثلاً

ADD B

RAR

JMP Ali

او تنظر

RAL

Ali :

MVI B, 08

STA 2005

جني ههنا الى هنا

لقس ، الاسم

ويكل النهاية

* Conditional Jump *

ايغاز قبل شوية جان بدون شروط يكنز دائماً يكل كالات ،
بين صرات ، ختاج شروط اذا متحقق يبق يكل البرنامج
غادي ، فاستعملنا Jump وية ، لقلقات .

Carry Flag

JC

Jump if Carry = 1

اذا كان ال CyFlag = 1 يكنز اذا 0 يكل البرنامج ويتجاهل

يُنكَب بنفس الطريقة JMP

JNC

Jump if Carry = 0

اذا كان ماكو Carry ، كنز ، اذا 1 cy يتجاهل ايغاز

ويُنكَب بنفس الطريقة في قبله

التاريخ Zero Flag

الموضوع :

JZ Jump if Zero

بعد تنفيذ الأوامر الحسابية إذا كان الناتج صفر
أو Zero Flag يصبح 1 فإن توقعه فإن Jump عليه
إذا كان Zero Flag = 1 يغير إذا لا يتجاهله

JNZ Jump if result not Zero

إذا كان Zero Flag = 0 يغير إذا كان 1 يتجاهله
ويكمل

Parity Flag

JPE Jump if even

يغير إذا كان عدد لأوامر زوجي (الفلاغ = 1)

JPO Jump if odd

يغير إذا كان عدد لأوامر فردي (Parity Flag = 0)

Sign Flag

JM Jump if minus

إذا كان البت Sign Flag = 1 يعني هذا الأمر يغير
من ال Sign Flag سادس 1

JP Jump if positive

إذا كان موجب Sign Flag = 0 فغير يغير من
الفلاغ صفر

* أمثلة على العبارات Branch group *

Ex1

MVI C, 04 خذ في قية ابتدائية لعدد

→ Name:

MOV A, B خلية قية B إلى A

Add M مضافها قية بالذكرة

DCR C نقصمة العدد 1

JNZ Name

ملاحظة : العبارات Jump تعتمد على -
فلاغات العملية في قبلها

راجع بقية في فقر بعين في حد صيغ العباد 0
ملاحظة : مسمى في Name بالبداية لان هيج
وبكل قية راجع يرجع العدد لا 4 وتسمى لوب لانها في

Ex2

برنامج يجمع 4 فلايا ذاكرة متوالية

MVI A, 0 تضعف A

MVI C, 04 خذ في قية ابتدائية للعدد

LXI H, 2850 خذ في عنوان ذاكرة بال H

→ Name:

ADD M راجع مجموع محتويات، لذاكرة وية A

INX H تزيد محتوي H وامدقت يعبر على لذاكرة في H

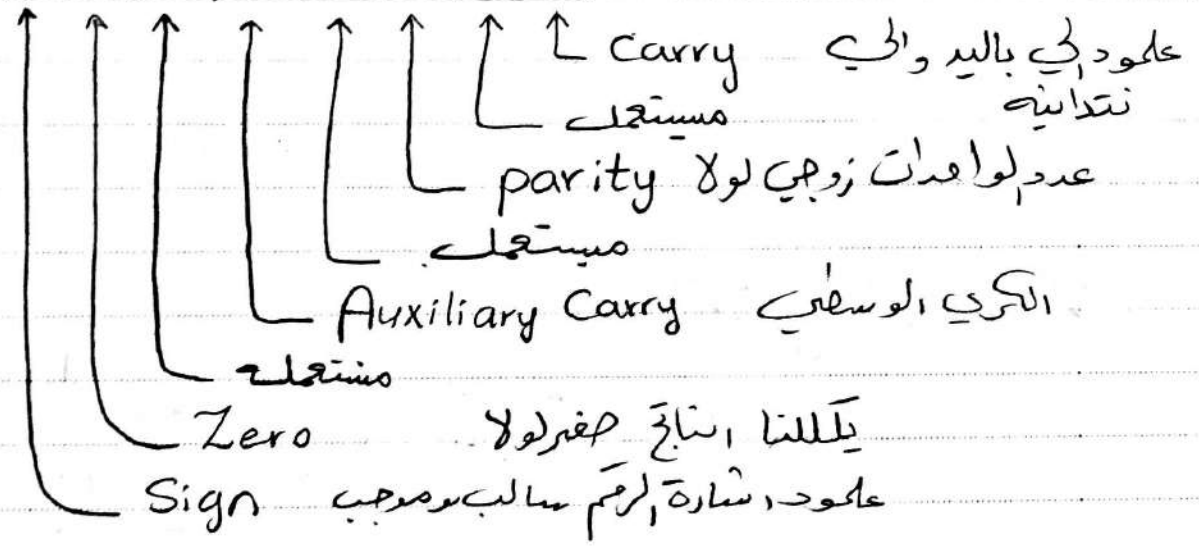
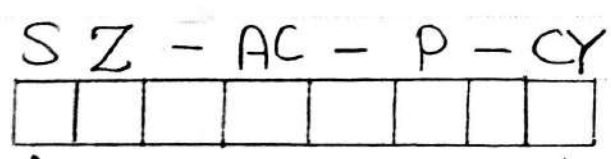
DCR C تنقص ال C وامد

JNZ Name

Flag Register

حرف ال F بالرجسترات
عبارة عن 8 بتات نذكر نشوفها
نصف ال رجسترات بالبرنامج

A			F
B			C
D			E
H			L



الرجسترة اشغلتها بطولنا معلومات عن الناتج ورة متفلة، لعلية
بيه 8 بتات 3 منها مستعمل

والخمس الباقيات

Sign Flag (S) :

منذ يصير 1 يعني الرقم سالب
من يصير 0 يعني الرقم موجب

بالأمري

1101 1011

لأننا نأخذ الرقم لي هنا

S=1 إذا كان 1

S
1

0010 1011

لأننا نأخذ الرقم
↓
S=0

S
0

Zero Flag (Z):

إذا كان ناتج العملية هو 0
ال Zero Flag حينها 1

مثال

1111 1111
10110011
01001101 +
00000000

بقية Carry (1)

الناتج النهائي كله صفر

Z
1

عندما يكون الناتج

101
010 +
111

لأن الناتج هو صفر

Z
0

Carry Flag (C)

اذا جان کو carry (باليد) او اگر borrow (صائبة) ار Carry Flag راج يصر 1 واذا ماكو فهو صفر
مثال

$$\begin{array}{r}
 \overset{1}{1}\overset{1}{0}\overset{1}{1}\overset{1}{0}101 \\
 01101100 + \\
 \hline
 00100001
 \end{array}
 \quad
 \begin{array}{c}
 \text{جمع} \\
 \text{①} \\
 \text{Carry}
 \end{array}
 \quad
 \begin{array}{r}
 10110101 \\
 11001100 - \\
 \hline
 11101001
 \end{array}
 \quad
 \begin{array}{c}
 \text{طرح} \\
 \text{①} \\
 \text{borrow}
 \end{array}$$

C
1

لهذا ال Flag يعتمد بس على افر Carry او borrow
عليه بالاستناد الي دتصير بنفس الرتم
مثال

جان کو carry بدفص الجمع
بس ماكو carry بالآخر

$$\begin{array}{r}
 \overset{1}{1}\overset{0}{0}\overset{1}{1}0010 \\
 00110011 + \\
 \hline
 11000101
 \end{array}$$

C
0

Auxiliary Carry (AC)

يصر 1 من اكو carry من راج
رتم لخاص رتم

فامس رتم راج رتم

Auxiliary carry

$$\begin{array}{r}
 10110101 \\
 00101110 + \\
 \hline
 11100011
 \end{array}$$

AC
1

اذا ماكو يصر صفر

$$\begin{array}{r}
 01100001 \\
 01100010 + \\
 \hline
 11000011
 \end{array}$$

ماكو Carry
AC
0

/ /

P 

یباوع علی بنناجی

10



ACI

جمع قيمة الـ A وقيمة بيانات نكتبه قبل وراه
ويفعل الـ Carry Flag

A	03	01	F
B			C
D			E
H			L

الناتج الحصري
 $A = 03 + 05 + 1$
 قية $\uparrow$ قية $\uparrow$ قية $\uparrow$
 A الى دفتلها Carry Flag
 بالاعاز

- p - cy

		1
--	--	---

ADC

جمع قيمة الـ A وقيمة رجستر او خلية ذاكرة
ووضعها في الـ A
مثال $A = 05$ $CarryFlag = 01$

ADC B

نكرر جمع وقيمة الـ
رجستر عادي

A	05	01	F
B	03		C
D			E
H			L

CY

		01
--	--	----

$$A = 05 + 03 + 01$$

قيمة الـ A
قيمة الـ رجستر
Carry Flag

او جمع وقيمة خلية ذاكرة مثلاً 9000

$$[9000] = 01 \quad H = 90 \quad L = 00 \quad A = 12$$

ADC M

يرجع الـ HL

ياخذ من عنوان الـ رجستر
ياخذ، القيمة مائة مائة

A	12		F
B			C
D			E
H	90	00	L

$$A = A + 01$$

قيمة الـ رجستر A
قيمة 9000
الي عنوان الـ موجود بار HL

9000	01

ADD

تجمع الرجستر او خلية الذاكرة وبيته محتويات A ويرسل carry مسجبه

مثال

A = 05
B = 01
CY = 1

ADD B

A	05	01	F
B	01		C
D			E
H			L

CY

		01
--	--	----

$$A = A + B$$

قيمتة A ← قِيَمَة A
قيمتة B ← قِيَمَة B

مجمع carry والرجح واذا جان
أو Carry قديم مثل هسة راع ينفسح

بعد استغيل

A	06		F
B	01		C
D			E
H			L

CY

		0
--	--	---

يصير ال Carry
بيته قِيَمَة ال Carry جديد

ونكده رجمع بيته وبيته خلية ذاكرة بنفس الطريقة

ADD M

ADI

جمع A وية قية مباشرة تكتب وراه
ويرسل وجهه ال Carry

ADI 05

A	01	01	F
B			C
D			E

CY
01

$A = A + 05$
 ↑
 قية A
 الرقم الي تكتب
 بالايجاز

A	06	

CY
0

DAD

يضيف الرجسترات على شكل زوج
 DE و BC
 جمعها وية الرجسترات HL
 يعني عبالك ديجمع، فتيين كل واحد 16 bit

DAD D

↑
 تكتب لو D
 لو B

DE + HL

A		
B		C
D	05	06
H	01	01
		L

0506
 0101+

 0607

D	05	06	E
H	06	07	L

DCR

ينقص الرجستر الي وراه 1

DCR L

قبل التشغيل $L = 06$

وردة، التشغيل صارت $L = 05$

DCR D

قبل التشغيل $D = 04$

وردة، التشغيل صارت $D = 03$

DCX

ينقص واحد من زوج رجسترات

BC او DE او HL

يعني ينقص 1 من رقم ابو 16 بت

DCX H

اذا كانت $L = 05$ $H = 06$

06 05
1-

06 04

بعد التشغيل

H 06 04 L

INR

يزيد الرجستر الي وراه بمقدار 1

INR D

قبل التشغيل $D = 05$

بعد التشغيل $D = 06$

INR L

قبل التشغيل $L = 04$

بعد التشغيل $L = 05$

INX

يزيد مقدار زوج رجسرات بواحد
BC او DE او HL
بمعدل كرقم ابو 16 بيت

INX D

قبل التشغيل

D 06 05 E

بعد التشغيل

D 06 06 E

$$\begin{array}{r} 06 \ 05 \\ + \ 1 \\ \hline 06 \ 06 \end{array}$$

SBB

يخرج قيمة رجستر او خلية ذاكرة من A
ويطرح منها borrow الـ هم الـ Carry Flag

SBB D

$$A = A - D - CY$$

القيمة ←
الرجستر
الى اختياره
نقص الـ Carry Flag
الى هو اخذ مكانه باليه

$$A = 08 - 05 - 01 = 02$$

A	08	01	F
B			C
D	05		E
H			L

CY

	01
--	----

A	02		F
B			C
D	05		E

SBB M

تلك تخرج عنوان ذاكرة

$A = 02 - 01 - 00$
 قيمة A
 قيمة 0000
 التي مكتوبة ب HL
 Carry Flag

A	02	
H	90	00
L		

9000	01

بعد التنفيذ

A	01	F
---	----	---

SBI

تخرج من A قيمة مباشرة تنكتب ورة الابعاز
 ويخرج متغير Carry Flag

SBI 02

$A = 05 - 02 - 1 = 02$
 قيمة A
 القيمة المباشرة
 Carry Flag

A	05	01	F

CY	01
----	----

بعد التنفيذ

A	02	
---	----	--

SUB

يُطرح قِيَمَةُ رَجِسْتَرٍ أَوْ ذَاكِرَةِ مِمَّا فِي A
بِشْرَافِ رَجِسْتَرِ A
وَالْقِيَمَةُ تُخْزَنُ فِي A

SUB D

$$A = 07 - 02 = 05$$

↑ قِيَمَةُ A ↑ قِيَمَةُ D

تَمَّ الْفِعْلُ وَجُودُ CARRY FLAG

A	07	01	F
B			C
D	02		E
H			L

CY
01

بَعْدَ الْفِعْلِ
A 05

SUI

يُطرح الْقِيَمَةُ الْمُبَاشِرَةُ فِي رَافِعِ تَتَلَبَّسُ وَرَأْسُ
مِمَّا فِي A وَبِشْرَافِ CARRY FLAG

SUI 05

$$A = 08 - 05 = 03$$

↑ قِيَمَةُ A ↑ الْقِيَمَةُ الْمُبَاشِرَةُ
↑ رَجِسْتَرِ A ↑ وَجُودُ CARRY

A	08	01	F

CY
01

بَعْدَ الْفِعْلِ
A 03

8085 Microprocessor Architecture

Main Objectives:

- 1- To introduce the main units of a microcomputer.**
- 2- To introduce the main units of 8085 microprocessor**
- 3- To learn how to interface the 8085 microprocessor with the memory.**
- 4- To learn how to interface the 8085 microprocessor with the I/O ports.**

Lesson One

General Architecture of Digital Computer & Overview of 8085 Microprocessor

Lesson Objectives:

- To introduce the main units of a microcomputer.
- To introduce the main units of the 8085 microcomputer

Pre Test:

- **Numerate the main units of the digital computer?**
- **What is the function of ALU, CU**
- **How can you determine the computer specifications based on its buses.**

General Architecture of Digital Computer

At present there are many types and sizes of computers available. These computers are designed and constructed based on digital and Integrated Circuit (IC) fabrication technology. A **digital computer** is a machine that can be used to solve problems for people and carrying out the tasks by following the instructions given to it. A sequence of instructions describing how to perform a certain task or job is called a program. There are two basic **components** of computer system architecture:

- a) Computer Hardware
- b) Computer Software

Computer Hardware

General purpose computer as shown in figure blow contains four basic hardware blocks that are:

- arithmetic and logic unit (ALU),
- memory unit (MU),
- input/output unit (IOU), and
- control unit (CU).

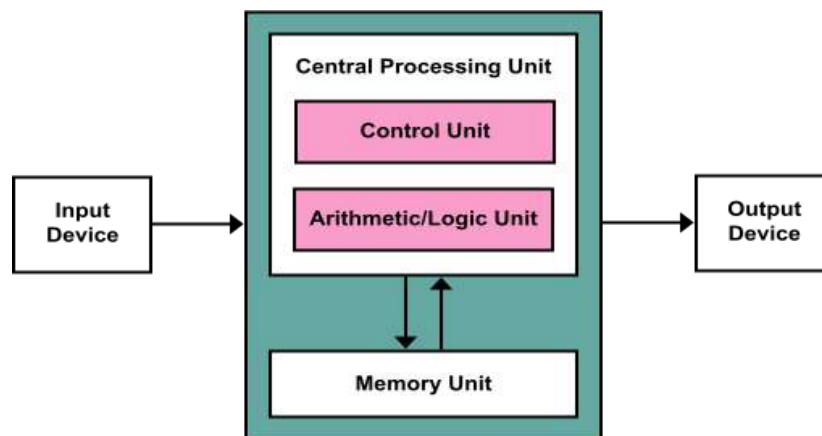


Figure (1): General purpose computer organization

Input/output (I/O) devices input and output data into and out of the memory unit. In some systems, I/O devices send and receive data into and from the ALU rather than the MU. Programs reside in the memory unit. The ALU processes the data taken from the memory unit (or the ALU) and stores the processed data back in the memory unit (or the ALU). The control unit coordinates the activities of the other three units. It retrieves instructions from programs resident in the MU, decodes these instructions, and directs the ALU to perform corresponding processing steps. It also supervises I/O operations.

Component 1: Arithmetic and Logic Unit (ALU)

The ALU unit functions are perform arithmetical operations, i.e. addition, subtraction, multiplication, division as well as logical operations. The control unit gives the instruction to the ALU which operations they have to perform and where it supplied (store purpose), it is directed to perform the operations.

Component 2: Memory Unit (MU)

The memory unit or storage section of the computer consist the devices used to store the data or information during the process. Memory unit is used to hold intermediate and final result of computer program. There are various types of memory which are used in computer system.

Component 3: Input / Output Unit (IOU)

The input unit contains the hardware devices those are used to enter the data in to computer system, Keyboard and mouse are most common devices. The output contain the hardware devices those are used to output the data from the computer system, Monitor and printer are most common output devices. Nowadays so many other I/O devices are used as shown below.

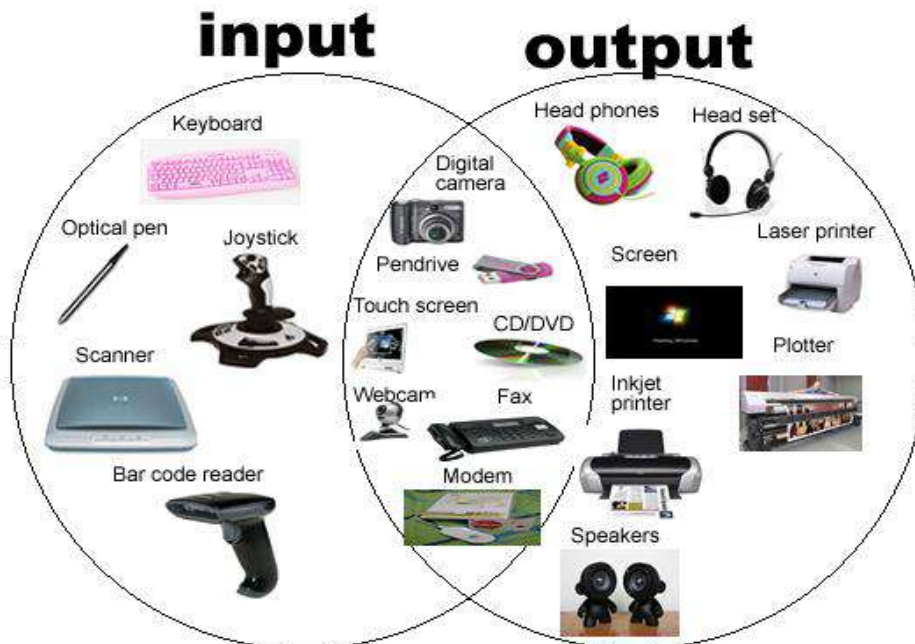


Figure (2): Input / Output hardware devices

Component 4: Control Unit (CU)

The control unit maintains the sequence of the operation, controlling the actions of all other units it can perform the instruction which constitute the program and direct as per the operation perform by machine.

Computer Software

Without software, most hardware would sit there doing nothing or perform specific tasks. To make most hardware run we need to use software, and the task here is to select the correct type of software for each job. The two main classifications of software that all programs fit under are:

- System software
- Application software

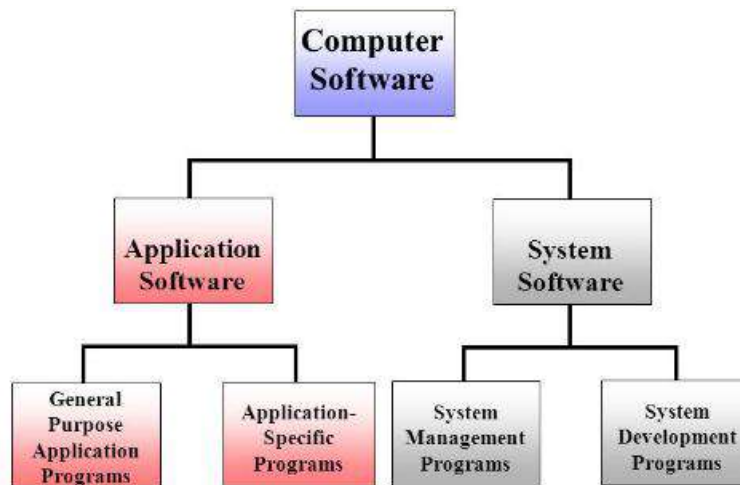


Figure (3): Basic classification of computer software

System software - software designed to operate the computer hardware and to provide a platform for running application software. Modern computers are complex machines involving many different parts. To keep it running well you will need system software. System software will handle the smooth running of all the components of the computer as well as providing general functionality for other programs to use, tools to speed up the computer, tools to develop new software and programs to keep you safe from attacks. There are several different types of system software that we will look at in more detail very shortly:

- ❖ **Operating Systems** are a collection of programs that make the computer hardware conveniently available to the user and also hide the complexities of the computer's operation. The Operating System (such as Windows 7 or Linux) interprets commands issued by application software (e.g. word processor and spreadsheets). The Operating System is also an interface between the application software and

computer. Without the operating system, the application programs would be unable to communicate with the computer.

- ❖ **Utility programs** are small, powerful programs with a limited capability, they are usually operated by the user to maintain a smooth running of the computer system. Various examples include file management, diagnosing problems and finding out information about the computer etc. Notable examples of utility programs include copy, paste, delete, file searching, disk defragmenter, disk cleanup. However, there are also other types that can be separately installable from the Operating System.
- ❖ **Library programs** are a compiled collection of subroutines (e.g. libraries make many functions and procedures available when you write a program)
- ❖ **Translator software** (Assembler, Compiler, Interpreter):
 - Assembler translates assembly language programs into machine code (A binary code that a machine can understand).
 - Compiler translates high level language code into object code (which is the machine language of the target machine).
 - Interpreter analyses and executes a high-level language program a line at a time. Execution will be slower than for the equivalent compiled code as the source code is analyzed line by line.

Application software - software designed to help the user to perform specific tasks

Application software is designed for people like me and you to perform tasks that we consider useful. This might be the ability of a scientist to work out statistical information using a set of results, or someone who wants to play the latest computer game. There are several categories of Application software that we'll look into shortly:

- General purpose application software.
- Special purpose application software.
- Bespoke application software

8085 Microprocessor Architecture

Intel 8085, an 8-bit NMOS microprocessor is available in the form of 40 Pin dual in line IC package. It is fabricated on a single LSI chip. It operates on +5 V d.c. supply. The clock speed used in this microprocessor is about 3 MHz. General Purpose 8-bit microprocessor is capable of addressing up to 64 K bytes (i.e. $2^{16} = 65536$ bytes) of memory. **8085 Microprocessor** can fetch instructions from a memory, decode and execute them i.e. performs certain arithmetic and logical operations, accept data from input device, and send results to output devices.

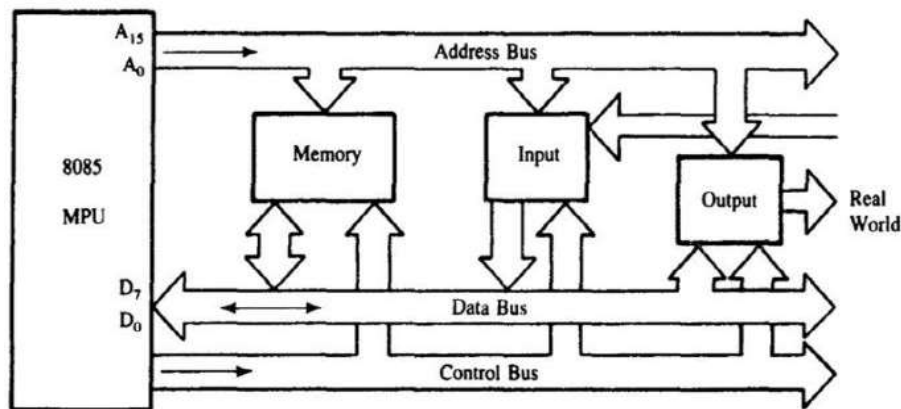


Figure (4): General Architecture of 8085 Microprocessor

In a computer system ALU and Control Unit are combined in one unit called **Central Processing Unit (CPU)**. The **CPU** is analogous to the human **brain** as all the decisions as per the instructions are made by this unit. All other parts are also controlled by this unit. A microprocessor is an integrated circuit designed for use as Central Processing Unit of a computer. The CPU is the primary and central player in communicating with devices such as memory, input and output. However, the timing of communication process is controlled by the group of circuits called control unit.

8085 Microprocessor Architecture

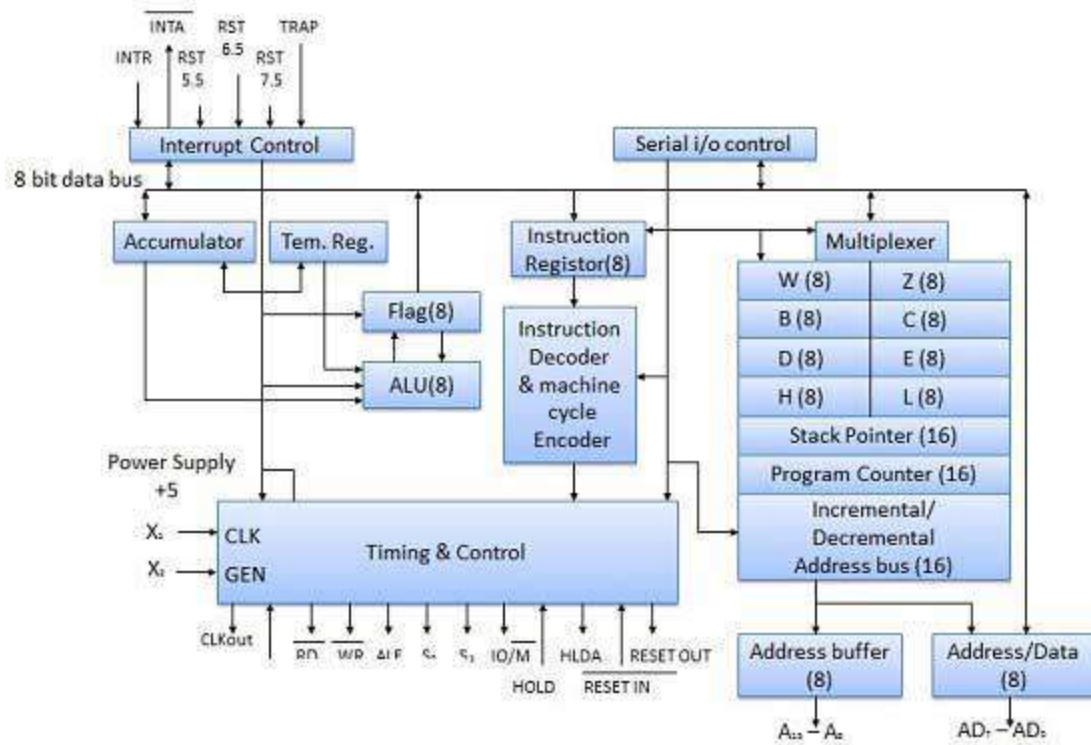
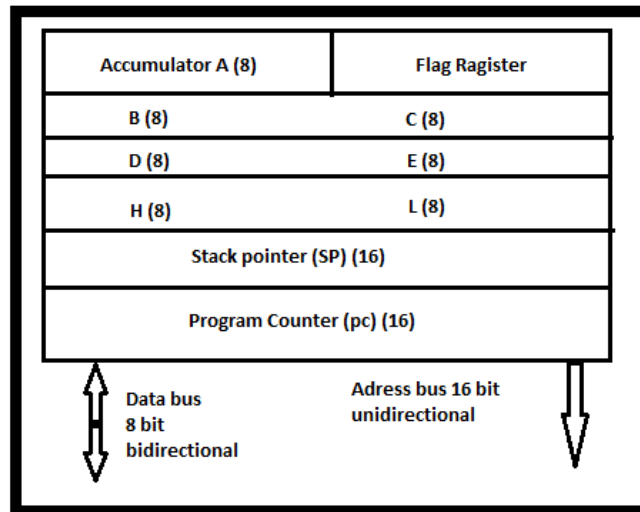


Figure (5): 8085 Microprocessor Block Diagram

The main functional components of 8085 microprocessor are as given below:

- ❖ Register Section
- ❖ Arithmetic and Logic Unit
- ❖ Timing and Control Section
- ❖ Interrupt Control
- ❖ Serial Input / Output Control

Register Section



The 8085 microprocessor contains eight addressable 8-bit registers namely:

- | | |
|-------------------------------------|--------------|
| ➤ A (Accumulator) register | ➤ D register |
| ➤ F Flag register (Flag flip-flops) | ➤ E register |
| ➤ B register | ➤ H register |
| ➤ C register | ➤ L register |

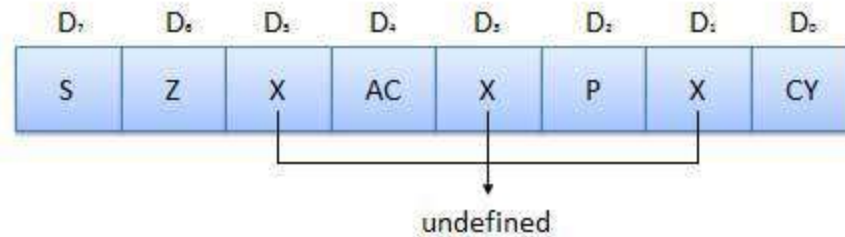
Out of these registers B, C, D, E, H and L registers are 8-bit general purpose registers. These registers can either be used as single register store 8-bit data for processing or a combination of two registers as 16 bit register pair. The valid register pairs are B-C, D-E or H-L register pairs. The higher order byte of 16 bit data is stored in first register (B in B-C register pair), and low order byte in the second register (C in B-C register pair).

The H-L register pair can also be used for register indirect addressing; since this register pair can also function as data pointer. The large number of general purpose registers gives more flexibility and ease in the programming. However, the general purpose registers are limited as registers occupy more space on the silicon chip.

Beside these general purpose registers, the 8085 has remaining two 8-bit registers Accumulator (A) and Flag (F) as special purpose registers and two 16 bit registers namely Program counter (PC) and stack pointer (SP).

- **Accumulator (A)** is an 8 bit register which stores the results of arithmetical and logical operations. It is also used to receive data from input port to microprocessor and to send data to output port from microprocessor. It is referred as register A in the program.

- **Flag Register (F)** is also an 8 bit register which indicate status of the accumulator after any arithmetical or logical operation. Out of 8 bits, five are defined as flags. These flags are given below.



- ♦ **Sign Flag (S)** – It indicates the sign of the number. If the number is negative then sign flag is set (changed to 1) and if positive then reset (changed to 0).
- ♦ **Zero Flag (Z)** – If after any arithmetical or logical operation, all the bits in accumulator are zero then zero flag is set otherwise reset.
- ♦ **Auxiliary Carry Flag (AC)** – If carry is generated from D₃ to D₄ in the accumulator after any operation then auxiliary carry flag is set otherwise reset. This flag is used internally by microprocessor for BCD operations and cannot be used by the programmer.
- ♦ **Parity Flag (P)** – If after any arithmetical or logical operation number of 1's in accumulator are even, then parity flag is set, otherwise reset.
- ♦ **Carry Flag (CY)** – The size of accumulator is 8 bit. If after any operation 9th bit is generated then carry flag is set, otherwise reset.

- **Program Counter (PC)**

Program counter is a 16 bit register which is also known as memory pointer. It stores the address of the next byte of the program to be executed. The microprocessor takes the address from program counter and executes instruction at that address. Each time the address in program counter is incremented by one.

- **Stack Pointer (SP)**

It is a 16 bit register which contains the memory address of top of the stack. Stack is a part of memory which is used to store data temporarily, especially while the execution of subroutines or functions.

Arithmetic and Logic Unit (ALU)

The arithmetic and logical unit (ALU) basically consists of accumulator (A), flag register (F) and a temporary register (which is inaccessible by the programmer or user). This unit carries out the arithmetic and logic calculations of the data stored in general purpose registers or in memory locations. The arithmetic operations are ADD, SUB, compare, increments, decrements and complements etc.; while logical operations are AND, OR, XOR and Rotate. The result of these operations could be placed in the accumulator or elsewhere through the internal bus. Arithmetic operations are usually carried out in 2's complement and for these operations the ALU receives the control signals from the timing and control unit.

Timing and Control Unit

This unit is responsible to synchronize Microprocessor operation as per the clock pulse and to generate the control signals which are necessary for smooth communication between Microprocessor and peripherals devices. The RD bar and WR bar signals are synchronous pulses which indicates whether data is available on the data bus or not. The control unit is responsible to control the flow of data between microprocessor, memory and peripheral devices.

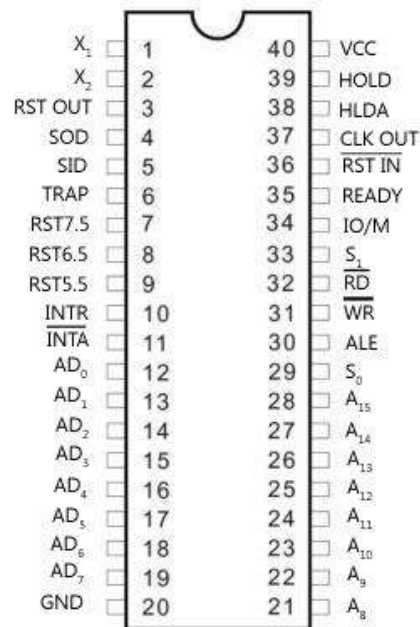


Figure (6): Pin configuration of 8085 μ p.

All the signal can be classified into six groups as explained in the following table:

Table (3): Pins Description of 8085 μ p

S.N.	Group	Description
1	Address bus	8085 requires 8-bit data bus and 16-bit address bus, as the memory address is of 16 bits. More number of IC pins are required if separate address and data bus are introduced. To restrict the number of pins of 8085 to only 40, lower address lines A0-A7 and data lines D0-D7 are used in multiplexed mode. The multiplexed lines are designed as Address/Data Bus (AD0-AD7). So whenever 16-bit address is transmitted by the microprocessor 8 MSBs of the address lines are sent on the Address Bus (A15-A8) and 8 LSBs of the lines are sent on the Address/Data Bus (AD7- AD0). The 8 LSBs of the address are then latched either into memory or external latch so that the complete address remains available for further operation. The 8-bit Address/Data Bus will now be free for the data transmission. The 8085 microprocessor has 8 signal line, A₁₅ - A₈ which are unidirectional and used as a high order address bus.
2	Data bus	The signal line AD7 - AD0 are bi-directional for dual purpose. They are used as low order address bus as well as data bus.
3	Control signal and Status signal	Control Signal $\overline{RD}$ – This is an active low signal to be connected to memory read input (output enable signal to memories) or to input / output read signal to enable input / output buffer. $\overline{WR}$ – Write signal is also active low. This signal is used to write to the memory or input / output devices. ALE (Address Latch Enable) – When ALE is high, 8085 microprocessor use address bus. When ALE is low, 8085 microprocessor is use data bus. The 16 bit address bus is basically divided into two sets. The most significant bits A7-A15 of the address bus are used separately and the least significant bits of the address AD0-AD7 are time multiplexed with the bits of bidirectional data bus (D0-D7). The AD0-AD7 bus serves the dual purpose as they can be used as low-order address bus as well as bidirectional data bus at different times. This is used as address bus, during the first clock cycle of the machine cycle involving memory; and during the remaining clock cycle of the machine cycle, it acts as the data bus. This is accomplished by address latch enable (ALE) signal provided in the processor. During the first clock cycle of the machine cycle ALE is high which enables the lower 8-bit of the address to be latched either into the memory or external latch. $IO/\overline{M}$ – This signal $IO/\overline{M}$ distinguishes that the address and data is meant for either I/O devices or memory. Whenever this signal is high (1), microprocessor will communicate to the I/O devices and whenever it is low (0), microprocessor will communicate to the memory. S₁ and S₀ – The status signals (S₀, S₁) along with $IO/\overline{M}$ signal indicate the type of machine cycle in progress. The type of machine cycle are op code fetch, memory read, memory write, I/O read or I/O write cycles.

		<table><tr><th>Machine Cycle</th><th>IO/$\overline{M}$</th><th>S₁</th><th>S₀</th></tr><tr><td>Op code fetch Cycle</td><td>0</td><td>1</td><td>1</td></tr><tr><td>Memory Read Cycle</td><td>0</td><td>1</td><td>0</td></tr><tr><td>Memory Write Cycle</td><td>0</td><td>0</td><td>1</td></tr><tr><td>I/O Read Cycle</td><td>1</td><td>1</td><td>0</td></tr><tr><td>I/O Write Cycle</td><td>1</td><td>0</td><td>1</td></tr><tr><td>INTR Acknowledge</td><td>1</td><td>1</td><td>1</td></tr><tr><td>Halt</td><td>Hi-Z</td><td>0</td><td>0</td></tr></table>	Machine Cycle	IO/ $\overline{M}$	S ₁	S ₀	Op code fetch Cycle	0	1	1	Memory Read Cycle	0	1	0	Memory Write Cycle	0	0	1	I/O Read Cycle	1	1	0	I/O Write Cycle	1	0	1	INTR Acknowledge	1	1	1	Halt	Hi-Z	0	0
Machine Cycle	IO/ $\overline{M}$	S ₁	S ₀																															
Op code fetch Cycle	0	1	1																															
Memory Read Cycle	0	1	0																															
Memory Write Cycle	0	0	1																															
I/O Read Cycle	1	1	0																															
I/O Write Cycle	1	0	1																															
INTR Acknowledge	1	1	1																															
Halt	Hi-Z	0	0																															
4	Power supply and frequency signal	V_{cc} – +5v power supply. V_{ss} – ground reference. X₁, X₂– A crystal is connected at these two pins. The frequency is internally divided by two operate system at 3-MHz, the crystal should have a frequency of 6-MHz. CLK out – This signal can be used as the system clock for other devices.																																
5	Externally initiated signal	♦ HOLD and HLDA (Hold Acknowledge) signals are used for DMA (Direct Memory Access) operation. In a microprocessor, the data transfer between I/O devices and memory will take place through the microprocessor. The involvement of the processor slows down the data transfer between I/O devices and memory. The transfer of data directly from I/O devices to memory without involvement of microprocessor is called DMA. The DMA will save the time as CPU relinquishes the control of Buses. In this way DMA transfers the large amount of data in a relatively short time. The HOLD and HLDA signals are used in the operation. Whenever HOLD signal is high, CPU temporarily relinquishes its operation by floating the address, data and control buses; and DMA operation is started. A high HLDA (Hold Acknowledge) signal is also sent to DMA controller, indicating that CPU has received the hold request. Whenever the data transfer is complete, then the control to CPU is returned back by sending a HOLD signal. Further the HLDA signal goes low. ♦ Some peripheral devices connected to 8085 microprocessor operate at much slower speed than the processor. To synchronize the speed of CPU and peripheral devices or to slow down the speed of 8085, the READY signal is used. If the READY signal is high the peripheral device is ready and the processor can complete the data transfer. If this signal is low the microprocessor waits (by generating a number of NOP T-states) till it goes high. READY (i/p) – This signal is used to delay the microprocessor read or write cycle until a slow responding peripheral is ready to accept or send data. ♦ The signal $\overline{RESET IN}$ may be low from the operator Reset button or from the processor. When the signal is low, the CPU will reset the program counter, instruction register and other circuits. It also sends a high RESET OUT. The RESET OUT signal goes to peripheral devices to																																

		reset or initialized. When signal $\overline{RESET\ IN}$ goes high and RESET OUT goes low, the data processing may begin. ♦ Interrupt Control: Sometimes it is necessary to interrupt the execution of the main program. For this an interrupt request is obtained from the I/O devices. After receiving the interrupt request (INTR), processor temporarily stops what it was doing and attends to the I/O device. $\overline{INTA}$ is an interrupt acknowledge signal which is sent by the microprocessor after INTR signal is received. After the work of the I/O device is completed it returns to what it was doing earlier. Basically 8085 has five hardware interrupts namely: a. INTR b. RST 5.5 c. RST 6.5 d. RST 7.5 e. and TRAP If two or more of these interrupts are active at the same time, the 8085 takes them in order of priority level as shown below: <table><tr><th>Interrupts</th><th>Priority</th></tr><tr><td>TRAP</td><td>1</td></tr><tr><td>RST 7.5</td><td>2</td></tr><tr><td>RST 6.5</td><td>3</td></tr><tr><td>RST 5.5</td><td>4</td></tr><tr><td>INTR</td><td>5</td></tr></table>	Interrupts	Priority	TRAP	1	RST 7.5	2	RST 6.5	3	RST 5.5	4	INTR	5
Interrupts	Priority													
TRAP	1													
RST 7.5	2													
RST 6.5	3													
RST 5.5	4													
INTR	5													
6	Serial I/O ports	Serial input / output control circuit incorporated in this microprocessor is used for the data transmission. For this purpose two pins SID and SOD are provided in the serial input/output control unit. The SID (Serial Input Data) terminal receives the serial data stream from an input device, the control unit converts serial data stream to parallel data before it is used by the computer. After the conversion 8-bit parallel data is stored in the accumulator. Similarly, SOD (Serial Out Data) terminal outputs the 8-bit parallel available with the accumulator into serial form to the peripheral device connected with the computer.												

Exercises:

EX.1

1. The function of CPU is-----
2. The number of address bus of the 8085 μ p is-----.
3. The function of Acc. Is -----
4. Increasing the lines of data bus will increase -----.
5. Computer software can be classified into -----,-----,-----,-----.

EX.2

Explain with drawing the internal architecture of the 8085 μ p.

Reference:

8085 μ p architecture and programming_Gonkar

Lesson Two

8085 IO / M addressing, Machine Cycle & Bus Timing

Lesson Objectives:

- To determine the machine cycles needed to execute the instructions.
- To Draw the bus timing diagram during the execution of the instruction

Pre Test:

- **Determine the types of machine cycles executed by the 8085 μ p.**
- **What is the status of control signals during memory read cycle.**
- **What is the function of ALE.**

8085 Microprocessor Architecture

8085 memory addressing, I/O addressing

8085 instructions can be classified in following addressing modes

- **Register Addressing mode**

Instructions which have their operands in registers only e.g. MOV, ADD, SUB, ANA, ORA, XRA etc.

- **Immediate Addressing mode**

Instructions in which operand immediately follows the op-code e.g. MVI, LXI, ADI, SUI, ANI, ORI etc.

- **Direct Addressing mode**

Instructions have their operands in memory and the 16-bit memory address is specified in the instruction e.g. LDA, STA, LHLD, SHLD etc.

- **Register Indirect Addressing mode**

Instructions have their operand in memory and the 16-bit memory address is specified in a register pair e.g. LDAX, STAX, PUSH, POP etc.

- **Implicit Addressing mode**

These instruction have their operand implied in the op-code itself e.g. CMA, CMC, STC etc.

8085 machine cycle & bus timing

1. Clock cycle

The speed of a computer processor, or CPU, is determined by the clock cycle, which is the amount of time between two pulses of an oscillator. Generally speaking, the higher number of pulses per second, the faster the computer processor will be able to process information. The clock speed is measured in Hz, typically either megahertz (MHz) or gigahertz (GHz). For example, a 3GHz processor performs 3,000,000,000 clock cycles per second.

Computer processors can execute one or more instructions per clock cycle, depending on the type of processor. Early computer processors and slower processors can only execute one instruction per clock cycle, but faster,

8085 Microprocessor Architecture

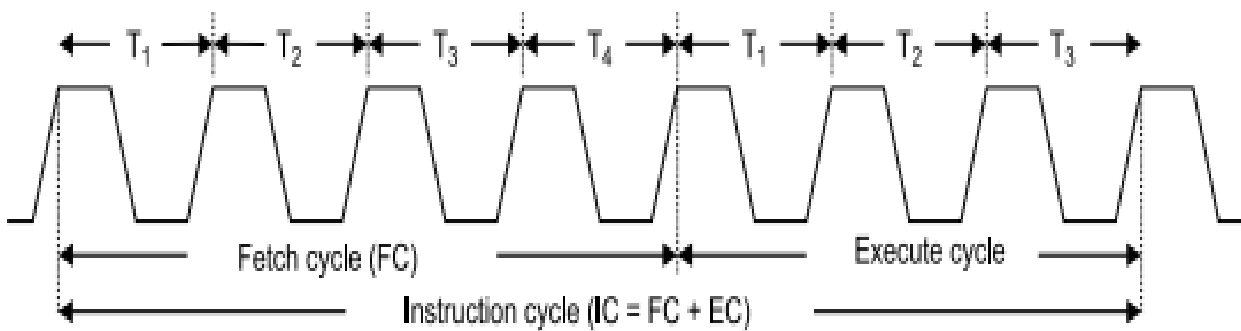
more advanced processors can execute multiple instructions per clock cycle, processing data more efficiently. Each clock cycle is called as T-states.

2. Instruction cycle

The sequence of operations that the CPU has to carry out while execution is called instruction cycle.

- 1:- Read an Instruction
- 2:- Decode the instruction
- 3:- Find the address of operand
- 4:- retrieve an operand
- 5:- perform desired operation
- 6:- find the address of destination
- 7:- store the result into the destination

TimingDiagram:



Where, Instruction cycle= Fetch Cycle(FC) (and decode the opcode) + Execute cycle(EC).

Performing each cycle needs performing one or more of the machine cycle.

3. Machine cycle:

8085 Microprocessor Architecture

The steps performed by the computer processor for each machine language instruction. The four common machine cycles are:-

- A. Opcode Fetch - Retrieve an instruction code from the memory.
- B. Memory Read (MR) - Retrieve the operand from the memory.
- C. Memory Write (MW) - Send the result to the memory Store.
- D. Input_Output Read (I/O R) - Read the operand from the Input port.
- E. Input_Output Write (I/O W) - Send the operand from the Input port.

Timing Diagram

Representation of Various Control signals generated during Execution of an Instruction. Following Buses and Control Signals must be shown in a Timing Diagram:

Higher Order Address Bus.

Lower Address/Data bus

ALE

RD

WR

IO/M

Opcode fetch:

The microprocessor requires instructions to perform any particular action. In order to perform these actions microprocessor utilizes Opcode which is a part of an instruction which provides detail (i.e. which operation μ p needs to perform) to microprocessor.

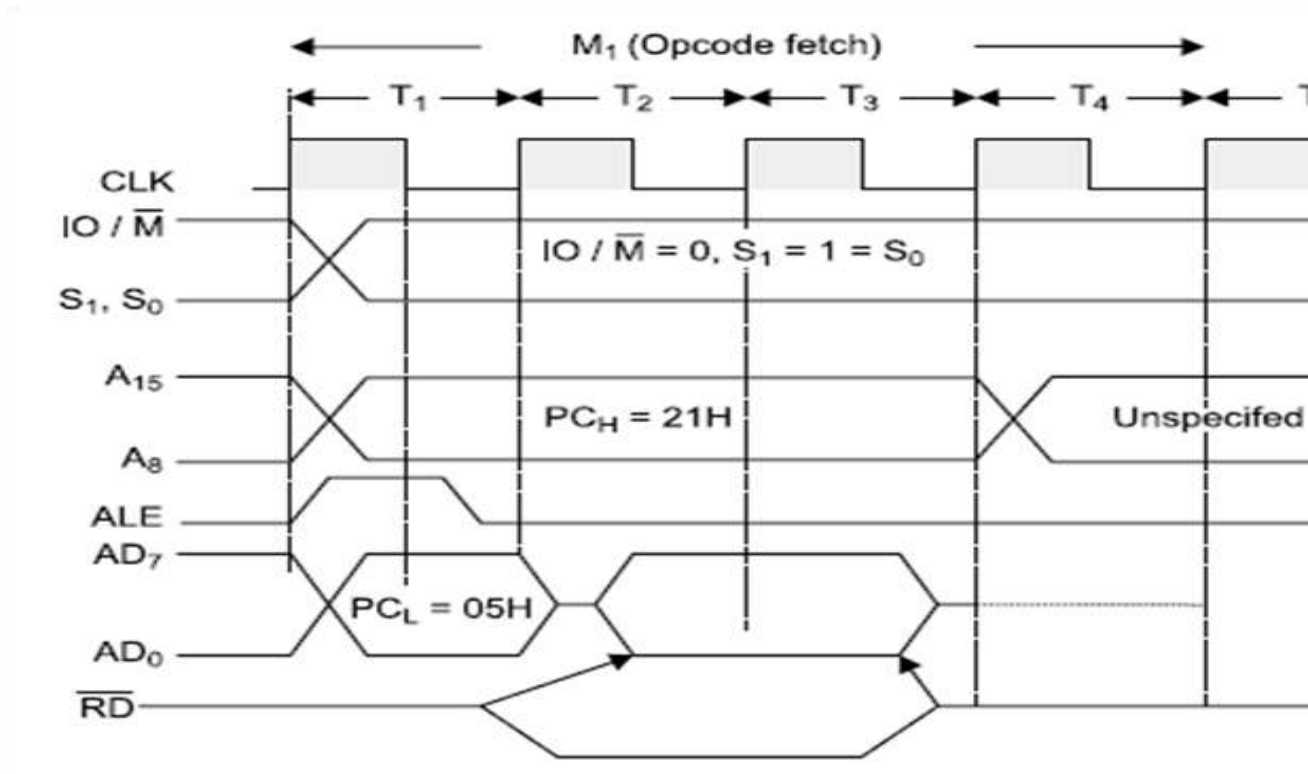


Figure (2): Opcode fetch timing diagram

Operation steps:

- 1- During T1 state, $IO/\overline{M}$, S₀, S₁ signals are used to instruct microprocessor to fetch opcode. Thus when $IO/\overline{M}=0$, S₀=S₁= 1, it indicates opcode fetch operation. During this operation 8085 transmits 16-bit address and also uses ALE signal for address latching. A₈-A₁₅ contains higher byte of address and lower byte of address A₀-A₇ is selected from AD₀-AD₇.
- 2- At T2 & T3 state, microprocessor uses read signal and make data ready from that memory location to read opcode from memory. Address is removed from AD₀-AD₇ and data D₀-D₇ appears on AD₀-AD₇.

Microprocessor reads opcode and stores it into instruction register to decode it further.

- 3- During T4 microprocessor performs internal operation like decoding opcode and providing necessary actions.

Read and write timing diagram for memory and I/O Operation

Memory Read:

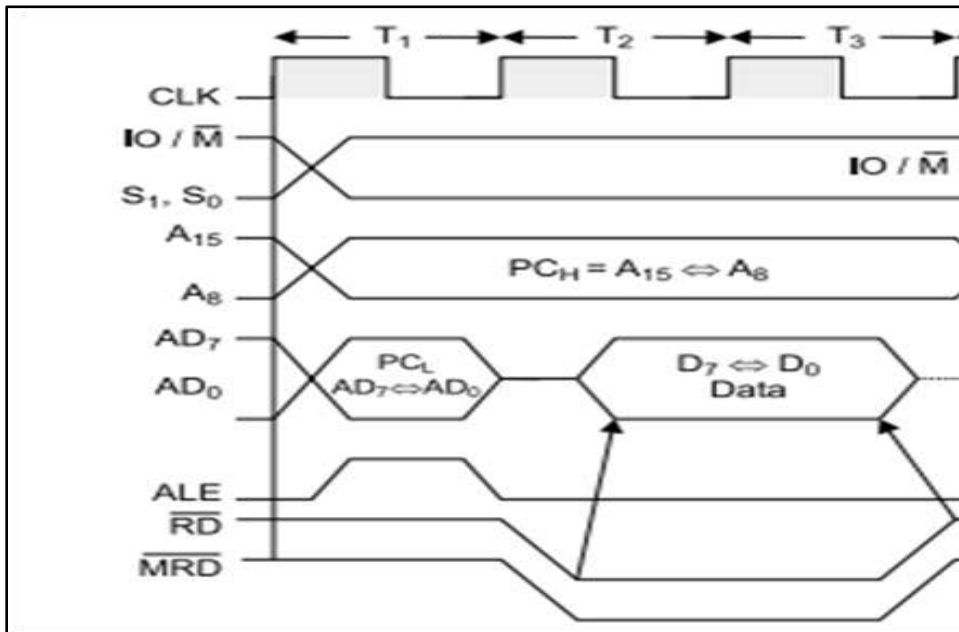


Figure (3): Memory read timing diagram

Operation steps:

- 1- It is used to fetch one byte from the memory.
- 2- It requires 3 T-States.
- 3- It can be used to fetch operand or data from the memory.
- 4- During T₁, A₈-A₁₅ contains higher byte of address. At the same time ALE is high. Therefore Lower byte of address A₀-A₇ is selected from AD₀-AD₇.
- 5- Since it is memory ready operation, IO/ $\overline{M}$ goes low.
- 6- During T₂ & T₃, ALE goes low, $\overline{RD}$ goes low. Address is removed from AD₀-AD₇ and data D₀-D₇ appears on AD₀-AD₇.

Memory Write:

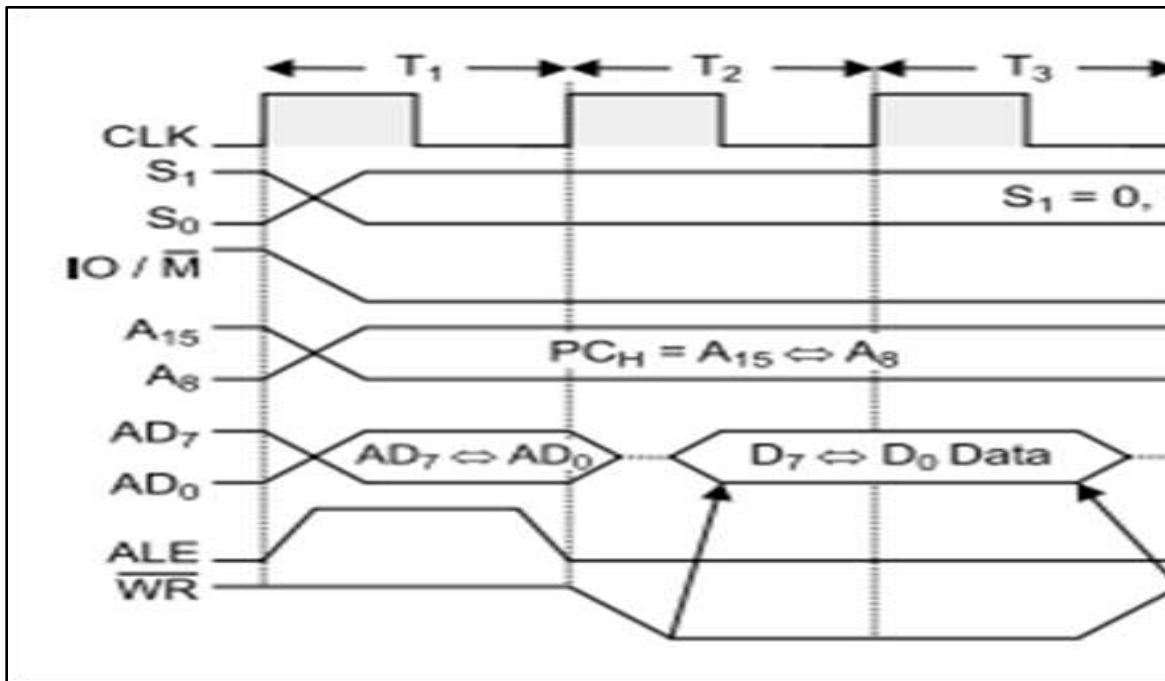


Figure (4): Memory write timing diagram

Operation:

- 1- It is used to send one byte to memory.
- 2- It requires 3 T-States.
- 3- During T₁, ALE is high and contains lower address A₀-A₇ from AD₀-AD₇.
- 4- A₈-A₁₅ contains higher byte of address.
- 5- As it is memory operation, $IO/\overline{M}$ goes low.
- 6- During T₂ & T₃, ALE goes low, $\overline{WR}$ goes low and Address is removed from AD₀-AD₇ and then data appears on AD₀-AD₇.

I/O Read:

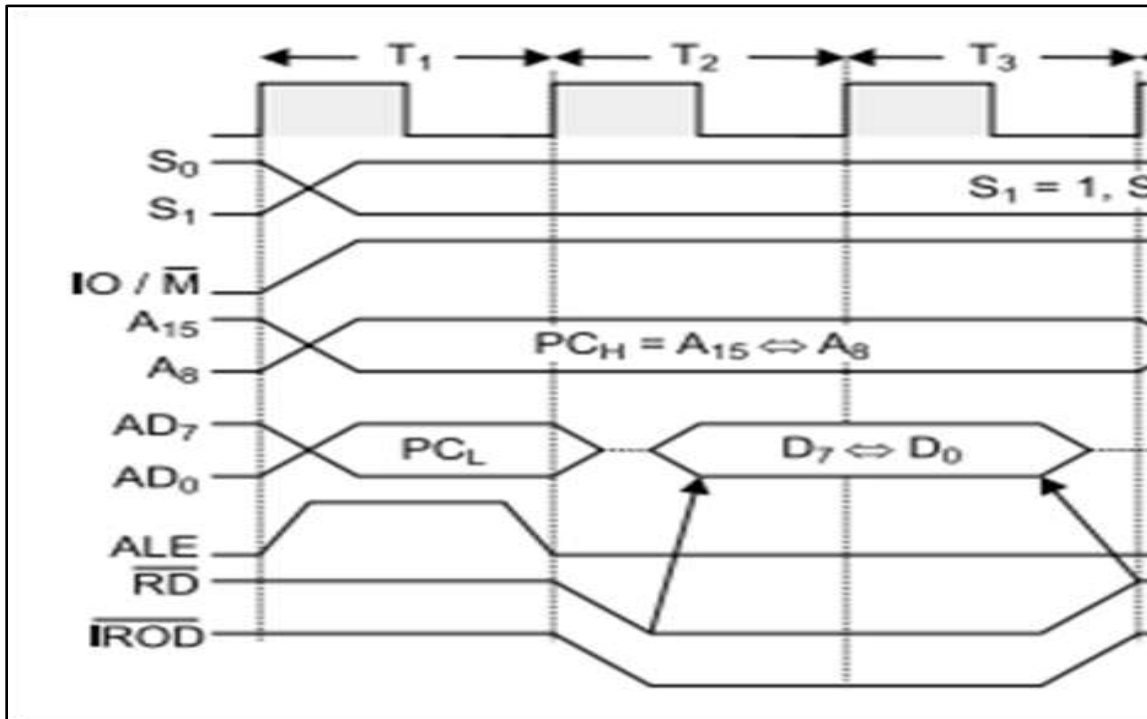


Figure (5): I/O Read timing diagram

Operation:

- 1- It is used to fetch one byte from an IO port.
- 2- It requires 3 T-States.
- 3- During T₁, The Lower Byte of IO address is duplicated into higher order address bus A₈-A₁₅.
- 4- ALE is high and AD₀-AD₇ contains address of IO device.
- 5- $IO/\overline{M}$ goes high as it is an IO operation.
- 6- During T₂ & T₃, ALE goes low, $\overline{RD}$ goes low and data appears on AD₀-AD₇ as input from IO device.

IO Write:

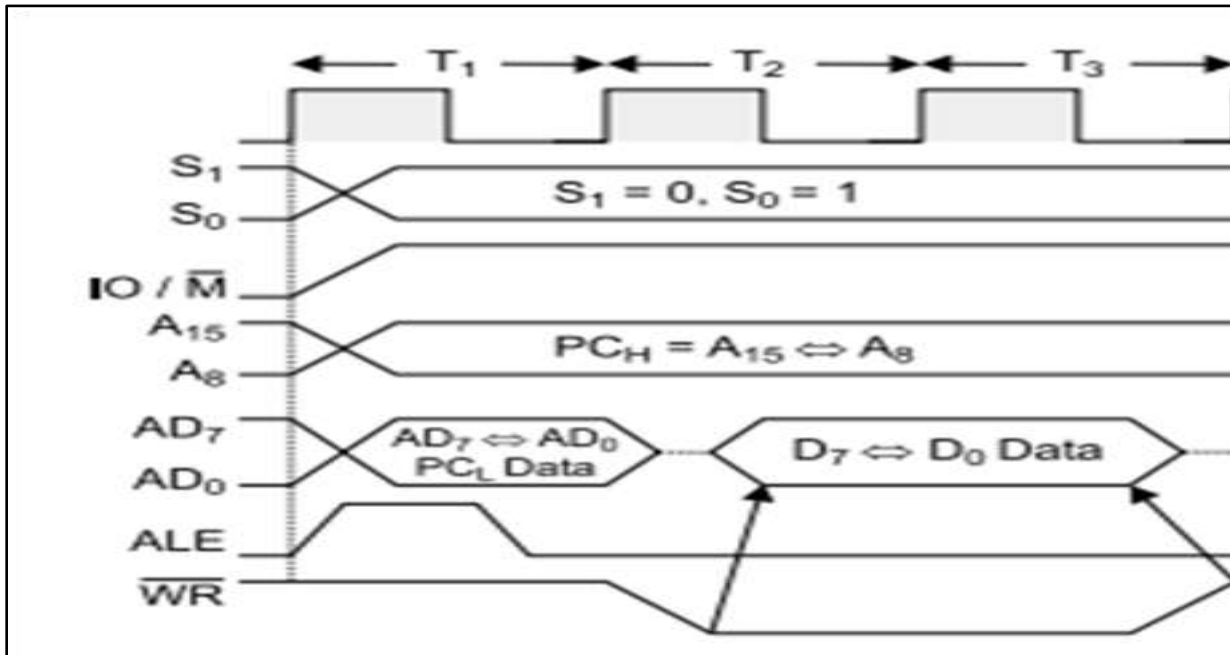


Figure (6): I/O Write timing diagram

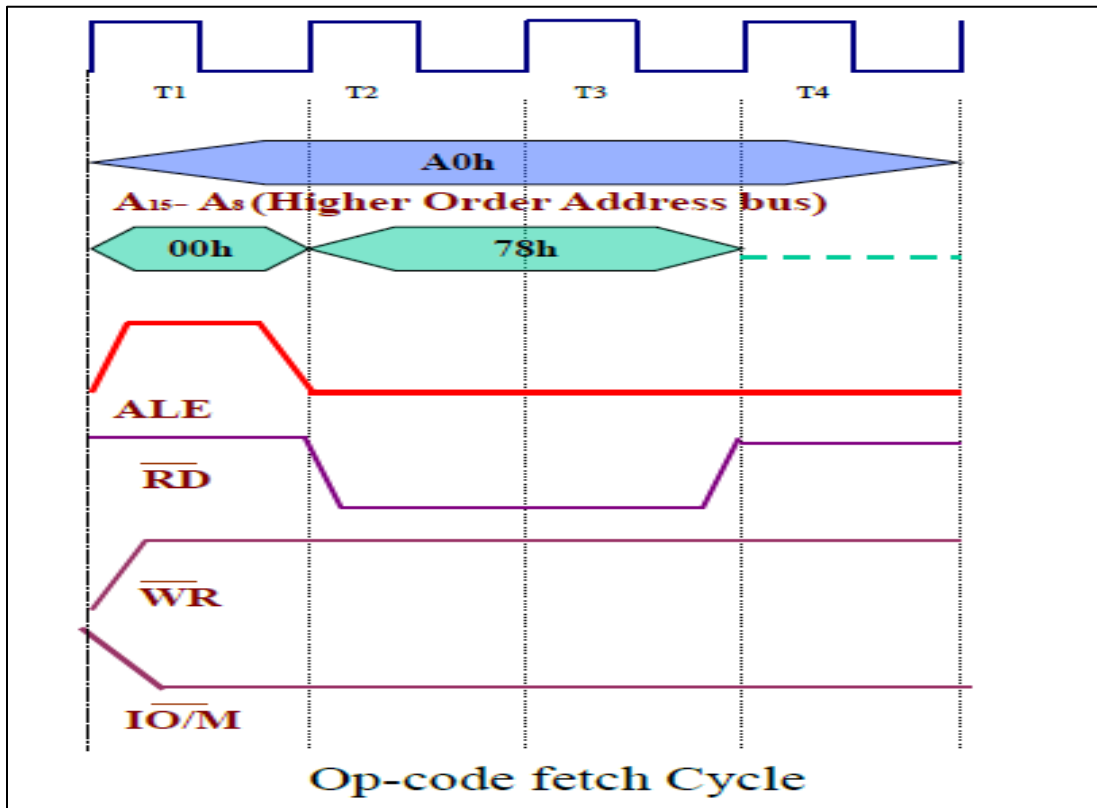
Operation:

- 1- It is used to write one byte to the IO device.
- 2- It requires 3 T-States.
- 3- During T₁, the lower byte of address is duplicated into higher order address bus A₈-A₁₅.
- 4- ALE is high and A₀-A₇ address is selected from AD₀-AD₇.
- 5- As it is an IO operation $IO/\overline{M}$ goes high.
- 6- During T₂ & T₃, ALE goes low, $\overline{WR}$ goes low and data appears on AD₀-AD₇ to write data to the IO device.

Examples

1. A000 MOV A,B 78

(1 Byte instruction, 1 machine cycle == opcode fetch == 4T state)



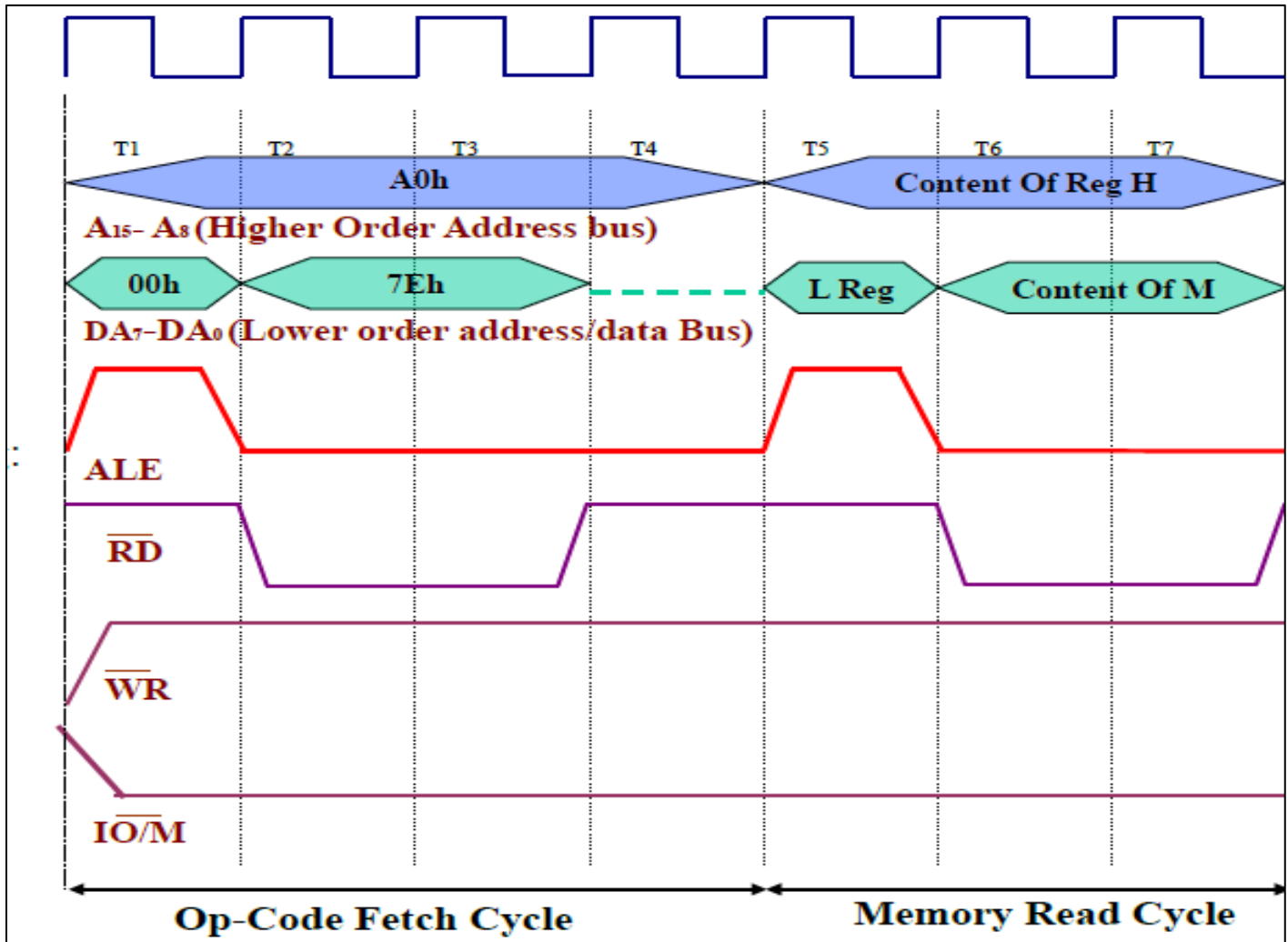
This 1 byte instruction needs 1 machine cycle (opcode fetch). During T1, the content of the program counter (PC) is placed on the A8-A16 and AD1-AD7 as ALE goes high. During T2 & T3 of op code fetch cycle (M1), the op code of the instruction is loaded in the instruction register (IR). The T4-state of this cycle M1 is for decoding and the contents of B are copied into A.

2. A000 MOV A,M 7E

This 1 byte instruction needs 2 machine cycles (opcode fetch and MR). During T1, the content of the program counter (PC) is placed on the A8-A16 and AD1-AD7 as ALE goes high. During T2 & T3 of op code fetch cycle (M1), the op code of the instruction is loaded in the instruction register (IR). The T4-state of this cycle M1 is for decoding. During the T1-state of the second machine cycle M2, the contents of the H-L pair are

8085 Microprocessor Architecture

placed on the address and address data bus. As usual during this state ALE sends a high pulse. During T2 and T3 states of machine cycle M2, contents of memory location are transferred (copied) to the specified register.



3. MVI A, 77 (2byte instruction , 2 machine cycle== (opcode fetch +1 MR) == 7T state

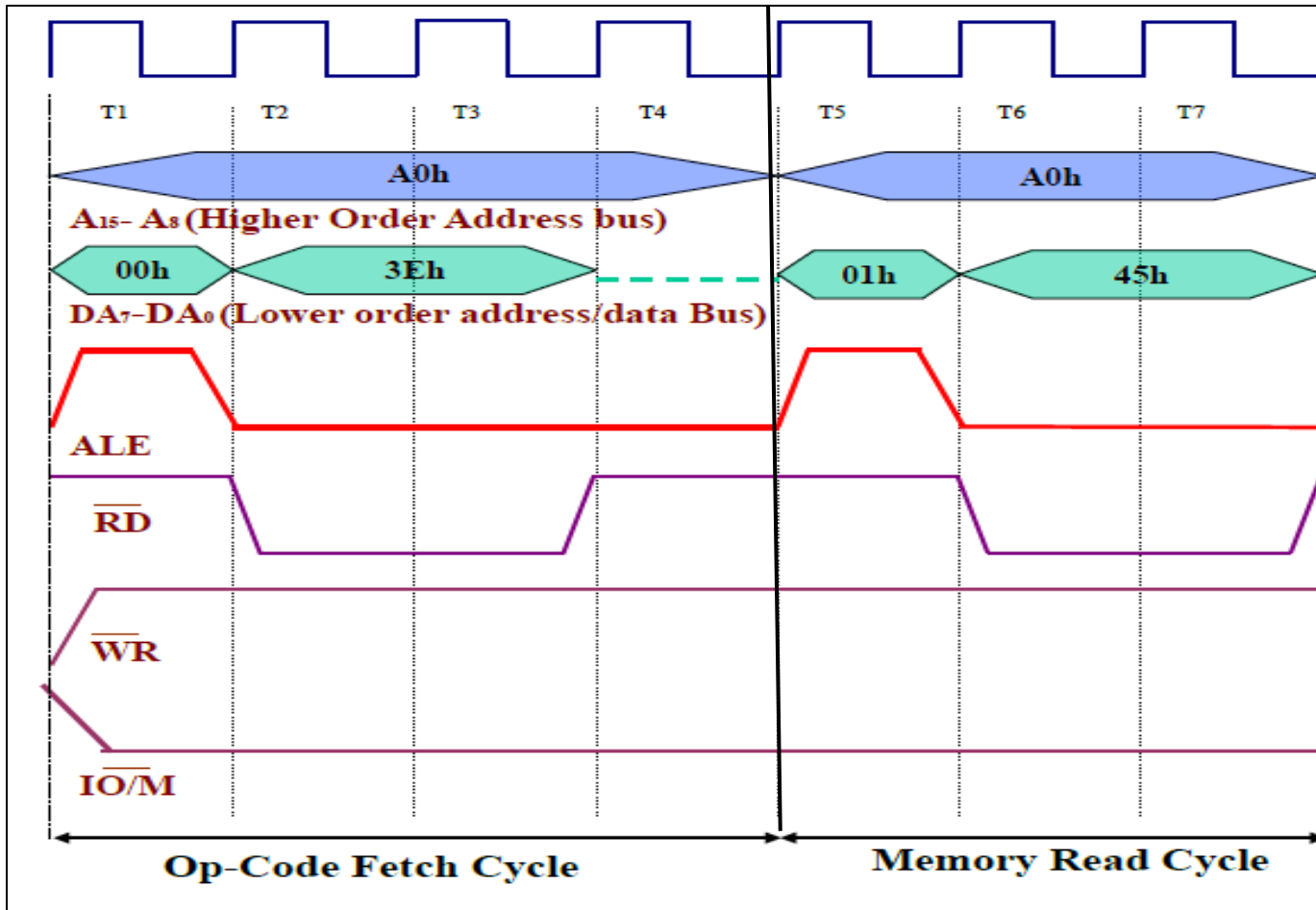
A000 3E

A001 77

This is an immediate move instruction. It is two byte instruction; one byte is used for the op code of the instruction and the other byte is used for the data. During T1, the content of the program counter (PC) is placed on the A8-A16 and AD1-AD7 as ALE goes high. During T2 & T3 of op code fetch cycle (M1), the op code of

8085 Microprocessor Architecture

the instruction is loaded in the instruction register (IR). The T4-state of this cycle M1 is for decoding. In the second machine cycle M2, the contents of the program counter (PC) is placed on the A8-A16 and AD1-AD7 during its T1 state. This is the address of the second byte. During T2 of M2 cycle, the PC is incremented. At the same time memory is accessed and immediate read data is loaded to the specified register during T3-state of M2 machine cycle. This instruction takes two machine cycles with 7 T-states.

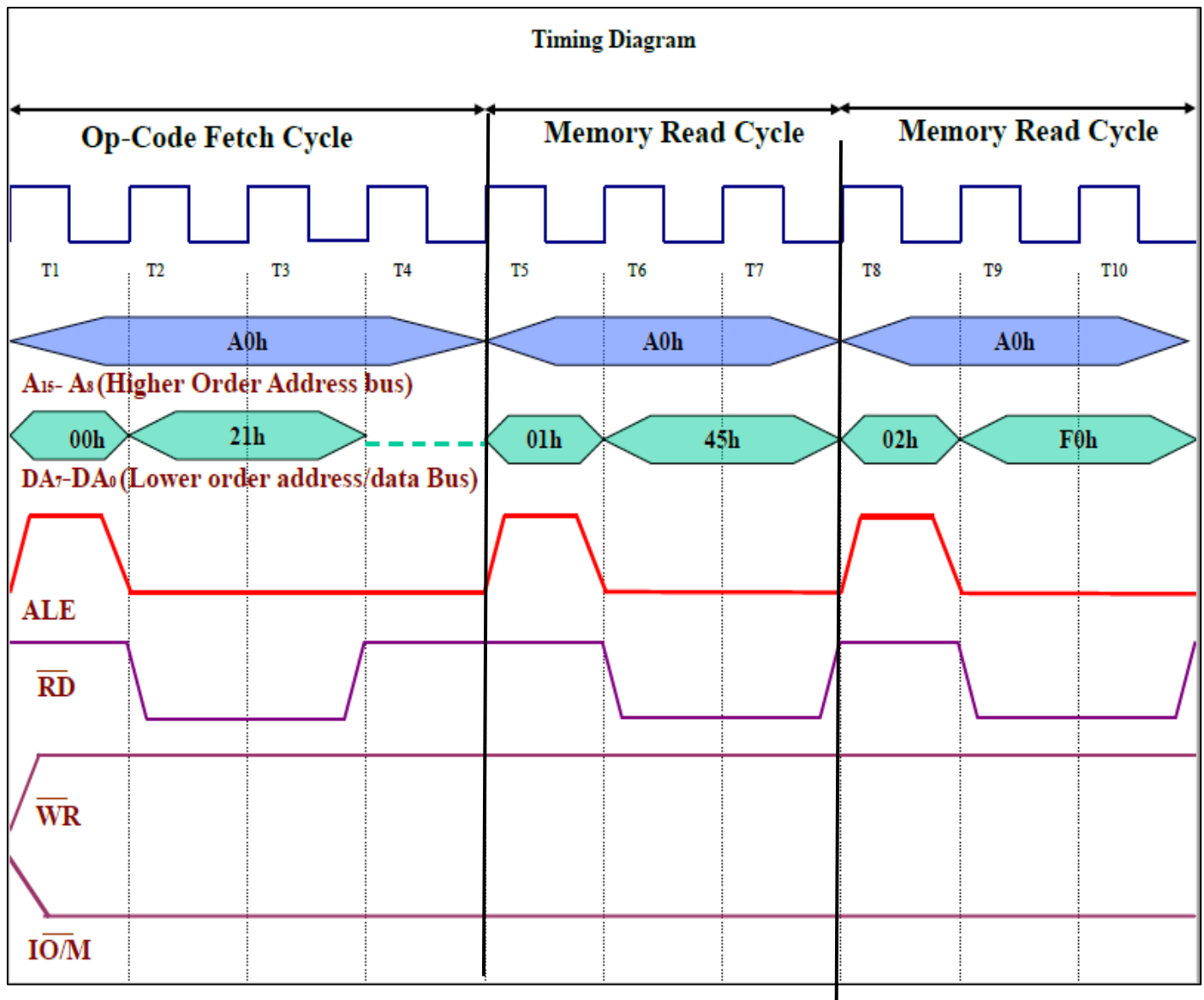


4. A000 LXI H, F045 21

A0001 45

A002 F0

This a three bytes instruction, which needs 3 machine cycle (opcode fetch + MR+MR). Each machine cycle is performed as previously discussed.



H.W.: For the following program , complete the table to determine the total instruction cycle time and the program processing time, if F= 2MHz. Draw the trimming diagram for each instruction.

M.L.	Instruction	Machine cycle	T state	RD WR	S0 S1 IO/M	Time (µsec)
2000	MOV B,A					
2001	MVI A, FF					
2003	LXI H, 2060					
2006	MOV M,B					

Exercises:

EX.1

6. The function of ALE is-----
7. RD signal will be -----during MEMR cycle, while IO/M will be----- .
8. OPcode fetch needs ----- T states.
9. The demultiplexing between address and data bus will occurs at ----- T states.
10. The time required to execute CMA is-----.

EX.2

Draw the bus timing diagram during the execution of SHLD, ADI 88, OUT 20.

Reference:

8085 μ p architecture and programming_Gonkar

Lesson Three

8085 Memory organization

Lesson Objectives:

1. To introduce the architecture of memory from the software point of view.
2. To determine the memory organization and capacity

Pre Test:

- 1- Determine the capacity of 1024 x 8bit memory chip.
- 2- Illustrate the possible memory organization for 4K bytes of memory.
- 3- Construct 8K bytes from 256 x 4 bits.

Accessing Information in Memory

For the microprocessor to access (Read or Write) information in memory (RAM or ROM), it needs to do the following:

Select the right memory chip (using part of the address bus).

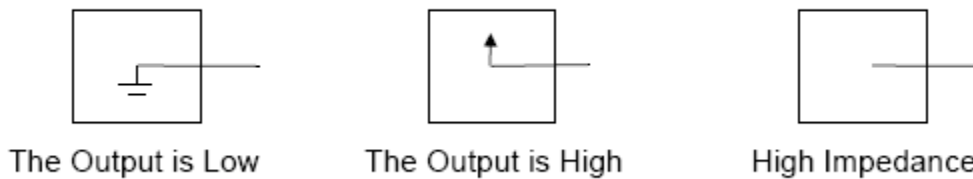
Identify the memory location (using the rest of the address bus).

Access the data (using the data bus).

Tri-State Buffers: An important circuit element that is used extensively in memory.

This buffer is a logic circuit that has three states: Logic 0, logic1, and high impedance.

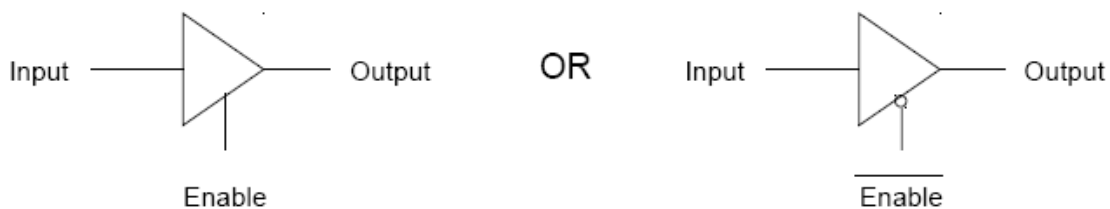
When this circuit is in high impedance mode it looks as if it is disconnected from the output completely.



This circuit has two inputs and one output. The first input behaves like the normal input for the circuit.

The second input is an “**enable**”. If it is set high, the output follows the proper circuit behavior.

If it is set low, the output looks like a wire connected to nothing.

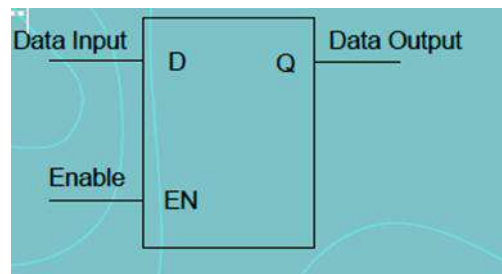


D latch.

The basic memory element is similar to a D latch.

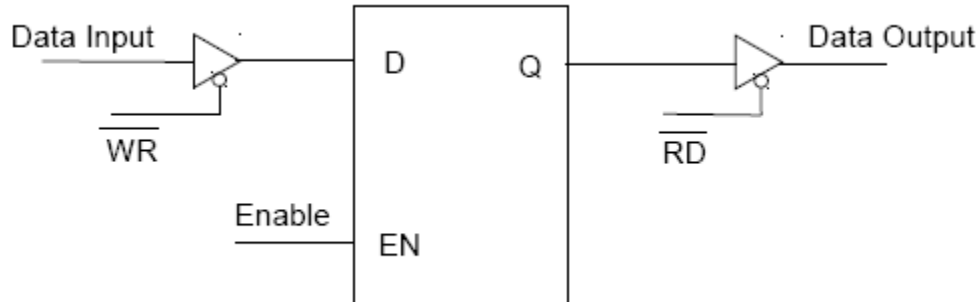
8085 Microprocessor Architecture

This latch has an input where the data comes in. It has an enable input and an output on which data comes out.

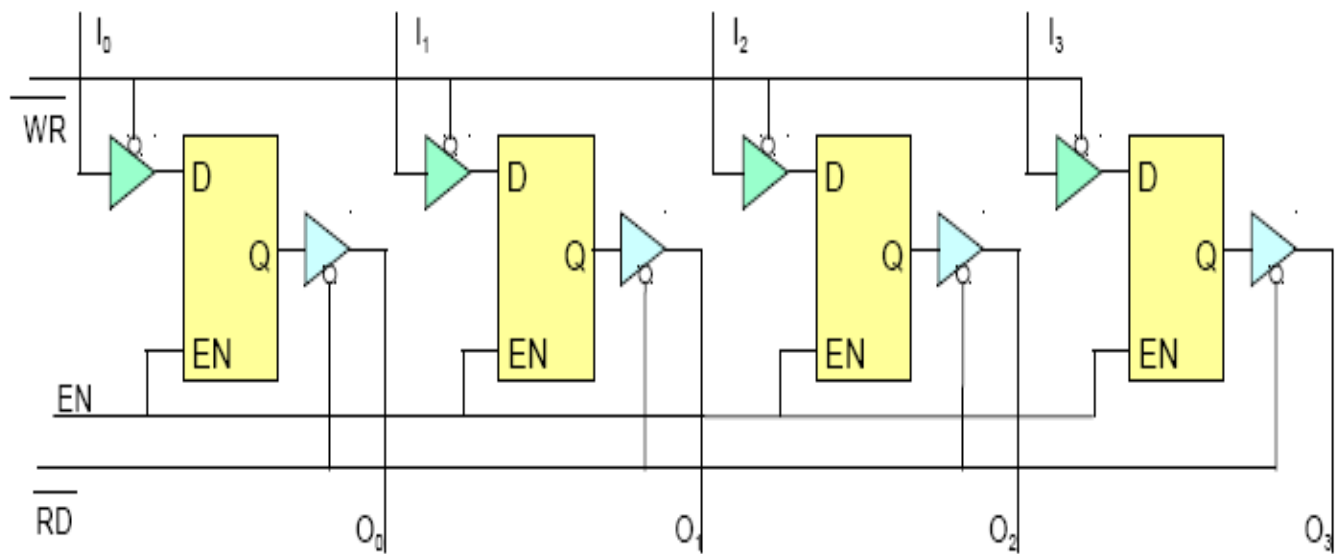


However, this is not safe. Data is always present on the input and the output is always set to the contents of the latch.

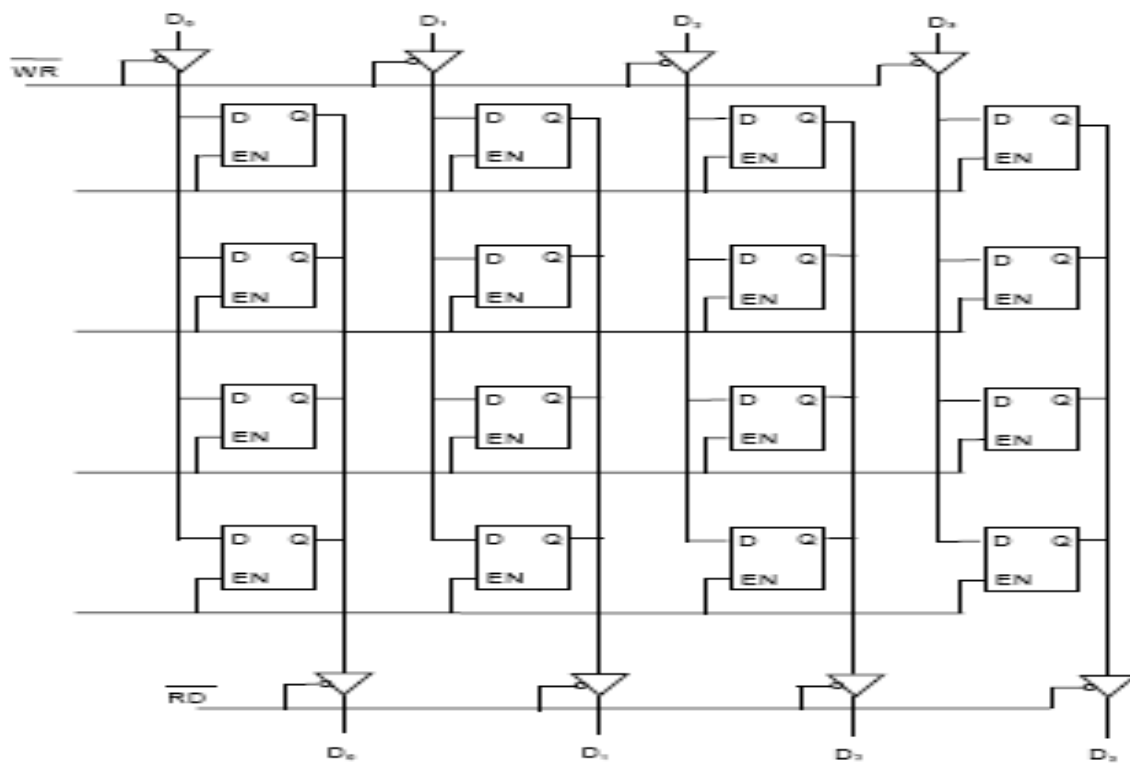
To avoid this, tri-state buffers are added at the input and output of the latch.



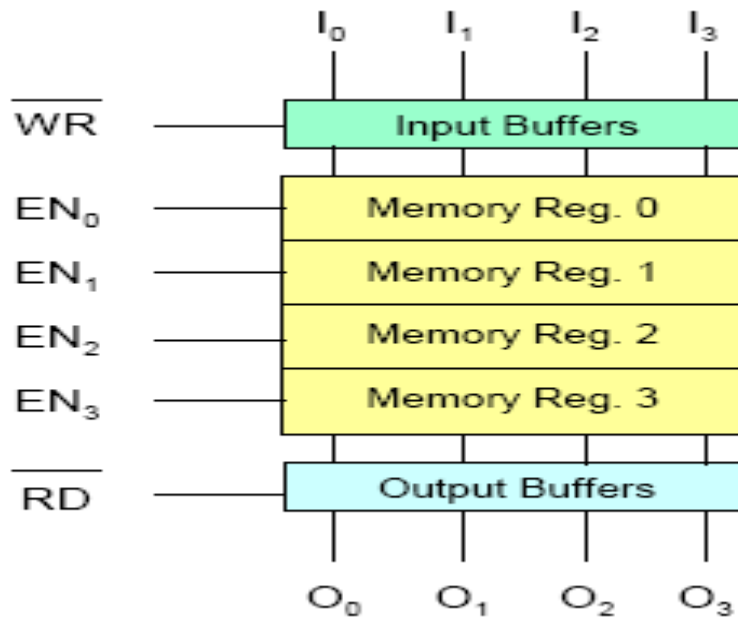
If we take four of these latches and connect them together, we would have a 4-bit memory register



Expanding on this scheme to add more memory registers we get



If we represent each memory location (Register) as a block we get the following

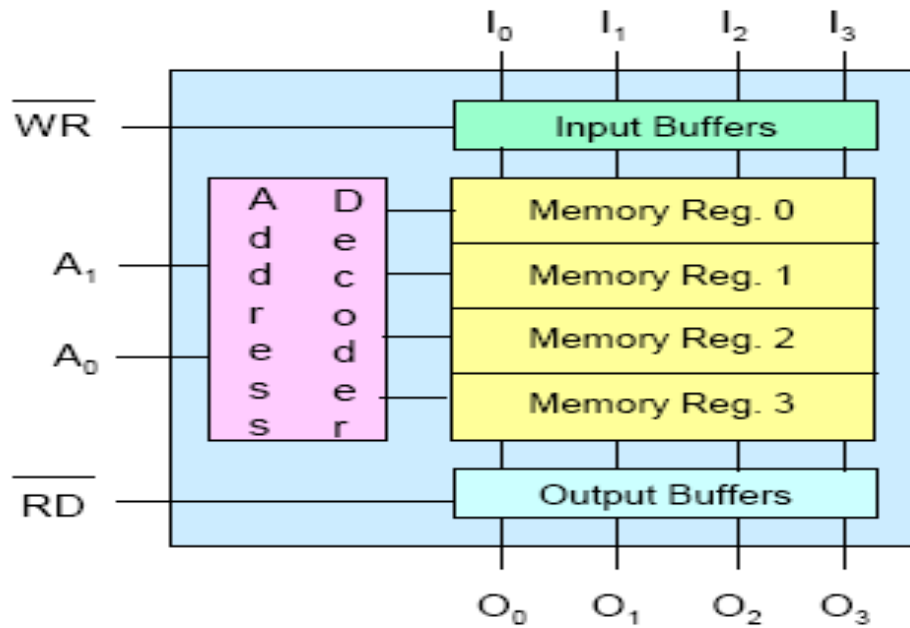


Using the RD and WR controls we can determine the direction of flow either into or out of memory. Then using the appropriate Enable input we enable an individual memory register.

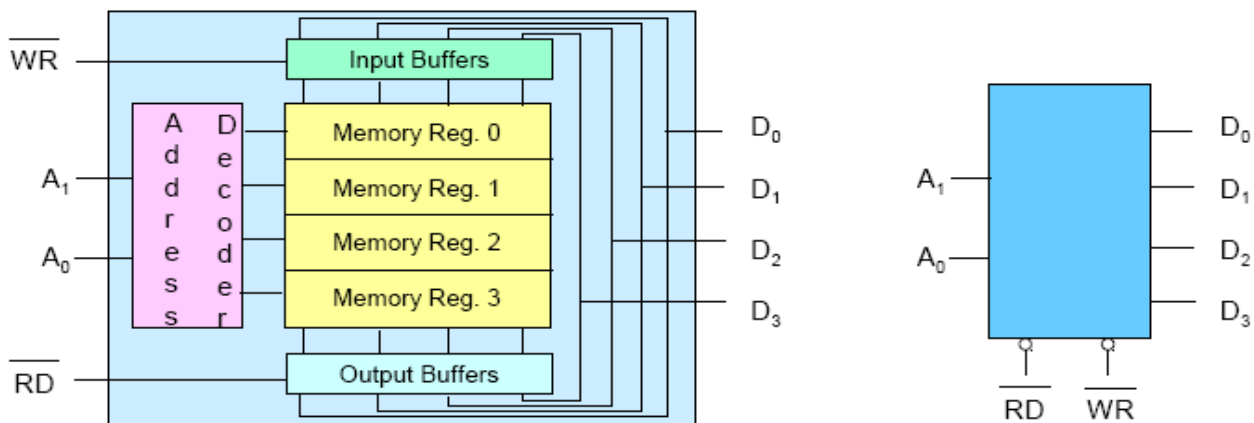
Since we can never have more than one of these enables active at the same time, we can have them encoded to reduce the number of lines coming into the chip.

These encoded lines are the address lines for memory.

So, the previous diagram would now look like the following:



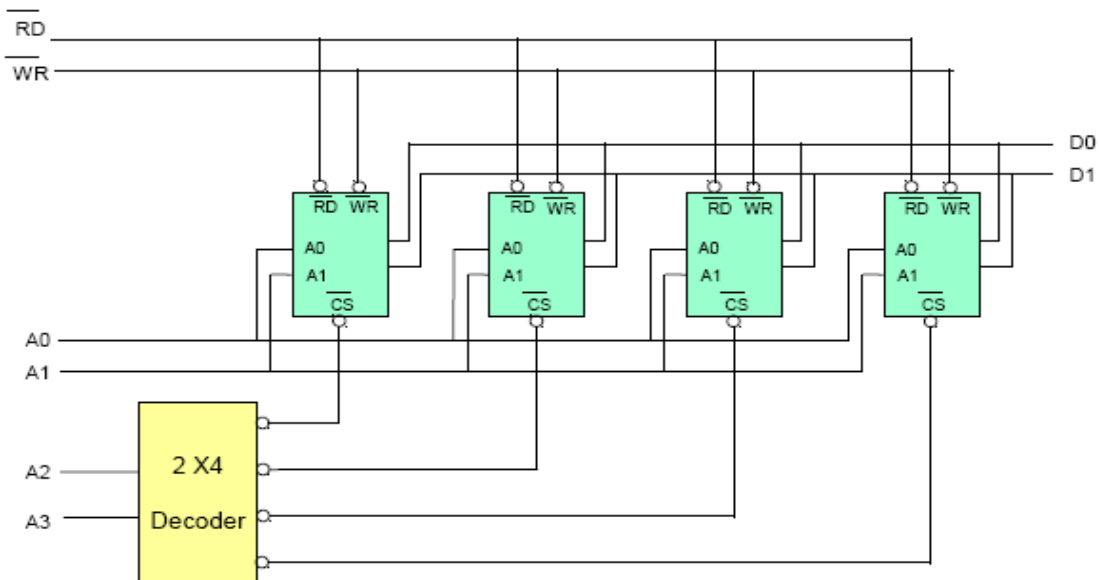
Since we have tri-state buffers on both the inputs and outputs of the flip flops, we can actually use one set of pins only



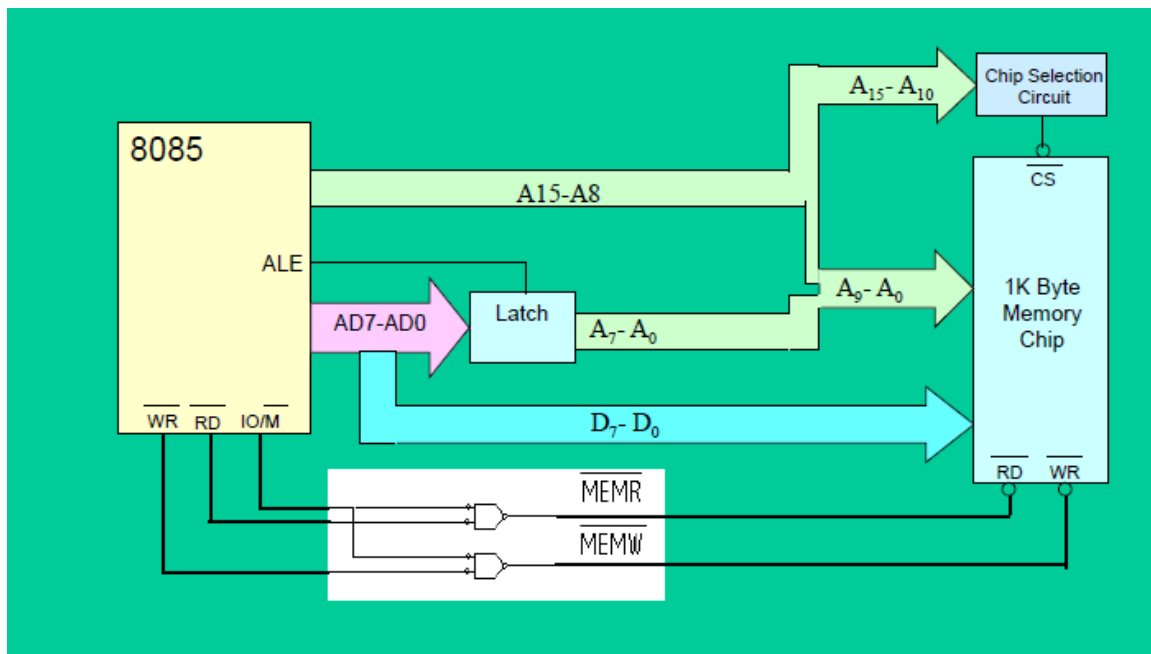
Chip Select

Usually, each memory chip has a CS (Chip Select) input. The chip will only work if an active signal is applied on that input. To allow the use of multiple chips in the make-up of memory, we need to use a number of the address lines for the purpose of "chip selection". These address lines are decoded to generate the 2^n necessary CS inputs for the memory chips to be used.

EX: 4x4 MEMORY chips



The diagram of 1k interfacing is shown below



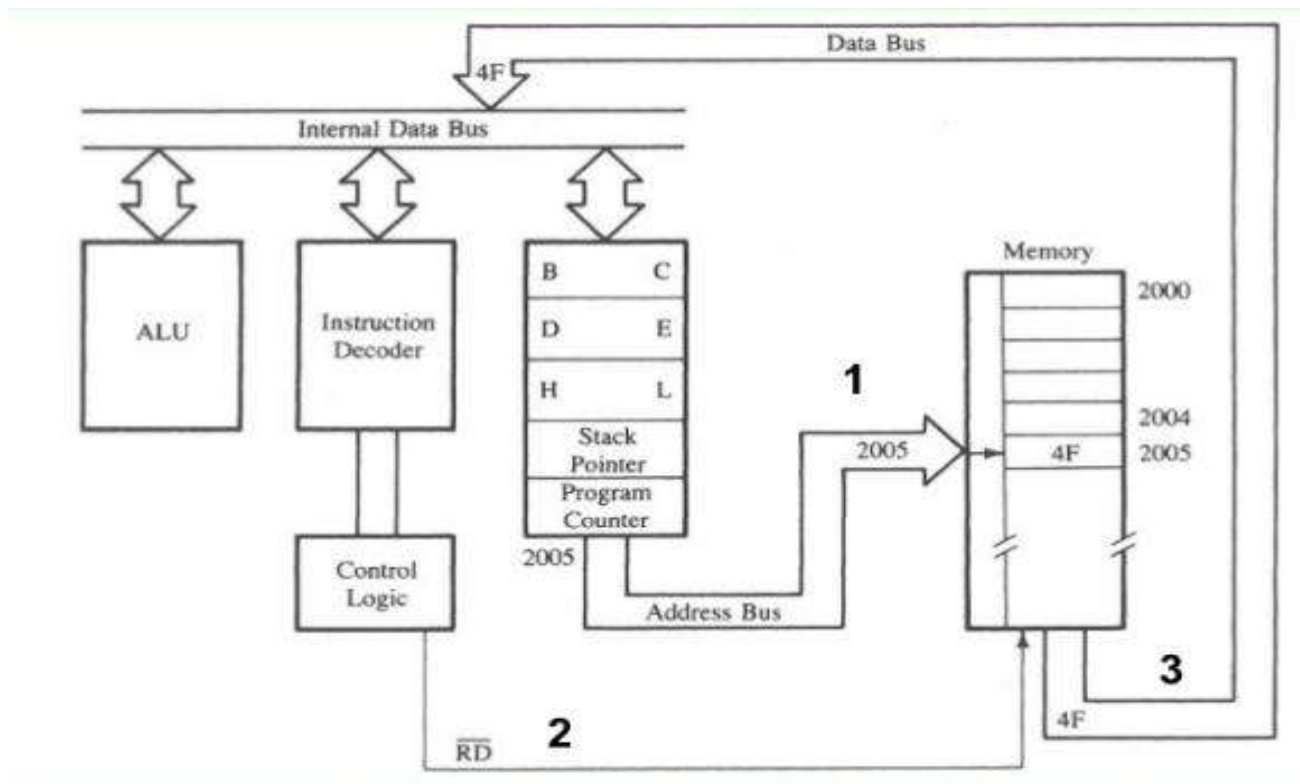
The execution of instruction is performed through number of machine cycles in order to :

- 1-Fetch the operation code (op code)
- 2-Decode the op code.
- 3-Fetch the operand.
- 4-Performed the required operation.
- 5-Store the results.

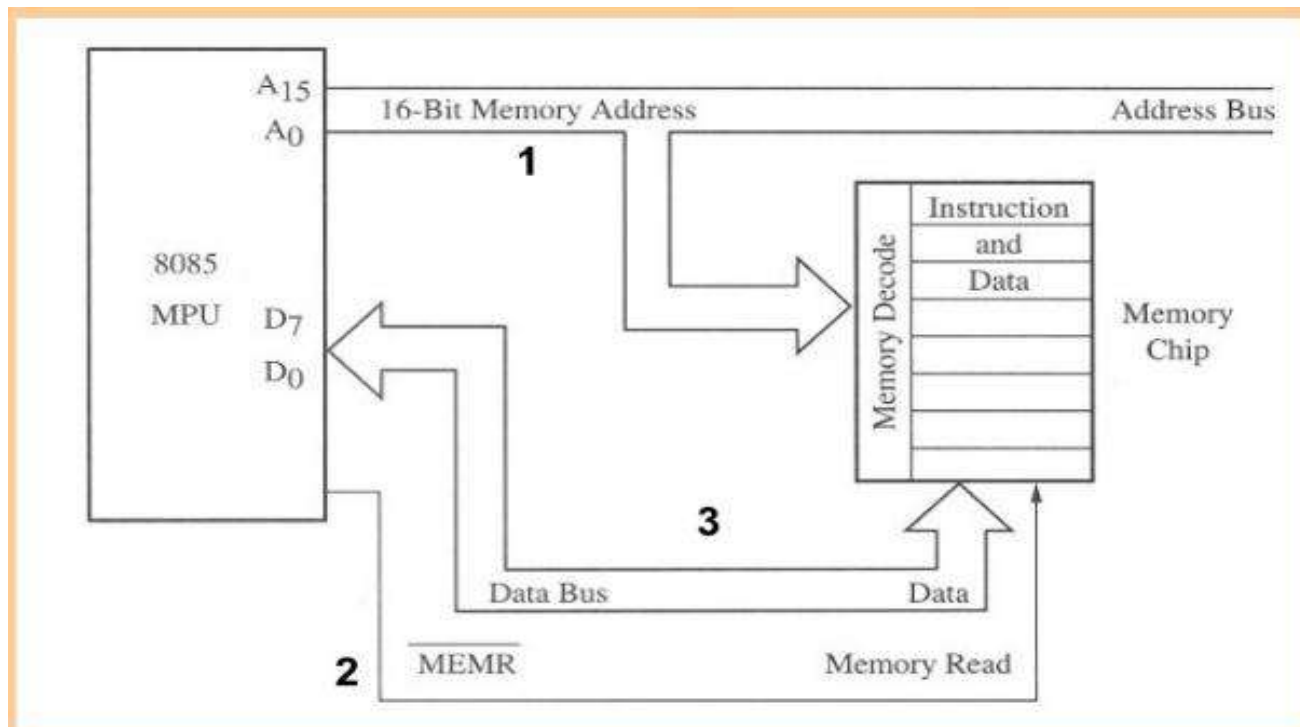
1. **Instruction Cycle:** Defined as the time required to complete the execution of an instruction.
2. **T-States:** It is defined as one subdivision of the operation performed in one clock period.
3. **Machine Cycle:** Defined as the time required to complete one operation of accessing memory, I/O, or ACKing an External request.
Machine cycle may consist of three to six T-States.

	Status			
Machine Cycle	$IO/\overline{M}$	$S1$	$S0$	Control Signals
Opcode Fetch	0	1	1	$\overline{RD} = 0$
Memory Read	0	1	0	$\overline{RD} = 0$
Memory Write	0	0	1	$\overline{RD} = 0$ $\overline{WR} = 0$
I/O Read	1	1	0	$\overline{RD} = 0$
I/O Write	1	0	1	$\overline{WR} = 0$
Interrupt ACK	1	1	1	$\overline{RD} = 0$ $\overline{WR} = 0$
Halt	Z	0	0	} $\overline{RD}, \overline{WR} = Z$ and $\overline{INTA} = 1$
Hold	Z	X	X	
Reset	Z	X	X	

EX: MOV C,A 4F(opcode fetch)



Ex: memory read



Exercises:

EX.1

11. Construct 1Mbytes from 2K x 8BITS.
12. Define; latch, decoder.
13. Define; machine cycle, Tstates

Reference:

8085 μ p architecture and programming_Gonkar

Lesson Four

8085 Memory map

Lesson Objectives:

- To determine the addresses range for each memory chip
- To draw the memory map
- To design a system the fulfills the requirements.

Pre Test:

- **Define the memory map**
- **Illustrate the Basic concept in memory interfacing.**
- **How many address lines needed to address 2048 bytes of memory.**

Memory map

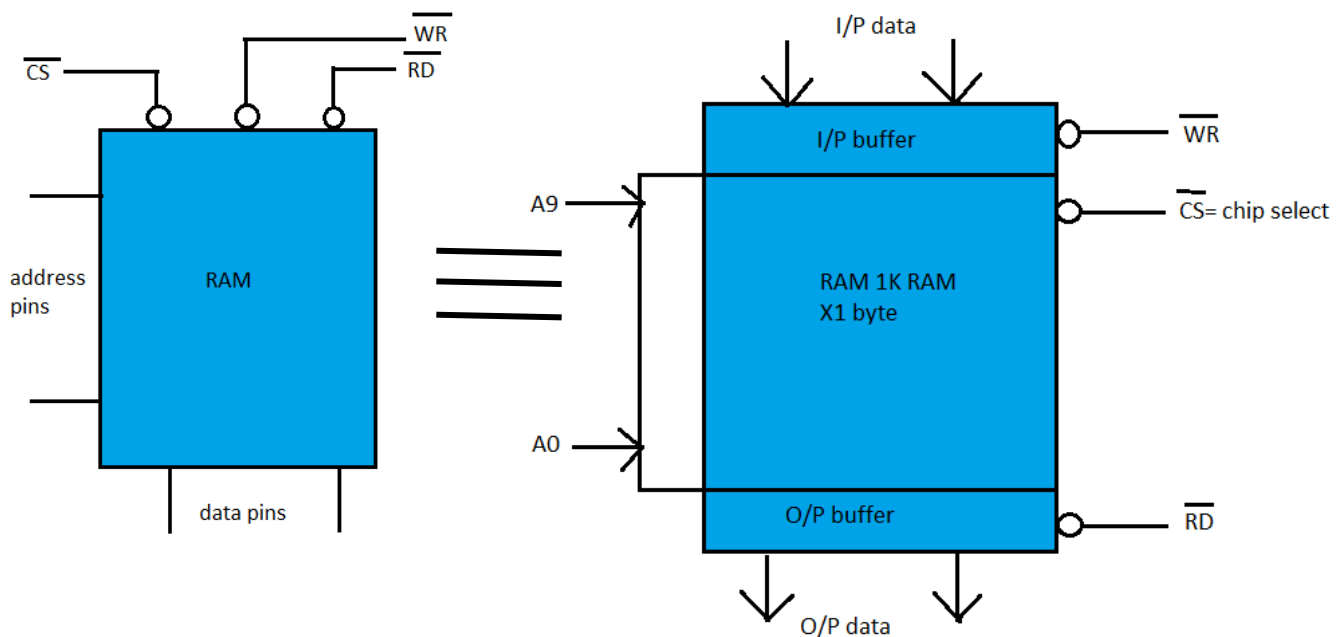
It is a pictorial representation in which memory devices are located in the entire range of addresses

Memory addresses: provide the location of various memory devices in the system.

❖ The interfacing logic defines the range of memory addresses of each memory device.

Memory interfacing:

- Typical memory chip



Basic concept in memory interfacing:

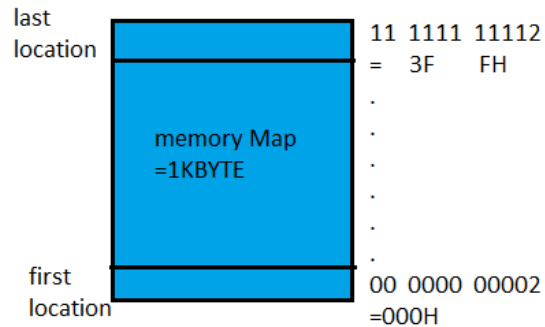
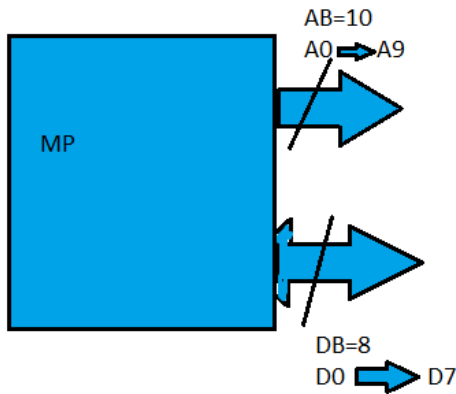
1. Select the chip (higher order address to activate **CS**)
2. Identify the memory location (low order address to enable the F/F)
3. Enable the I/P buffer (using WR) OR O/P buffer (using RD)

To interface the memory with MP:

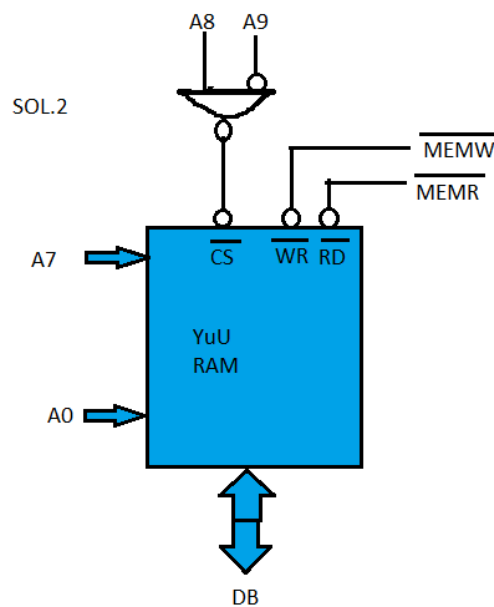
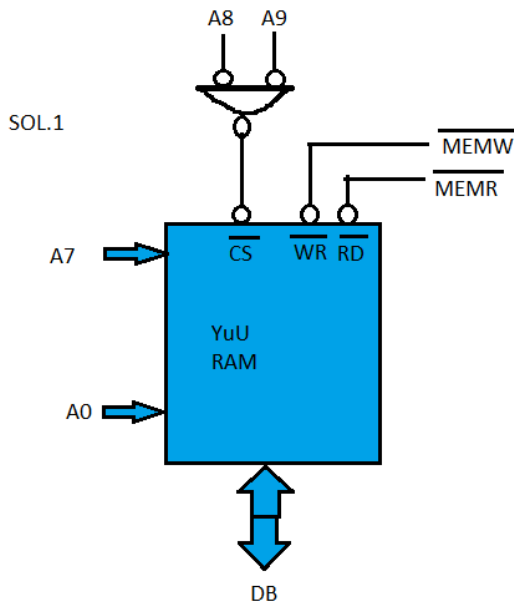
1. Connect the low order of AB to the address pins of memory chip.

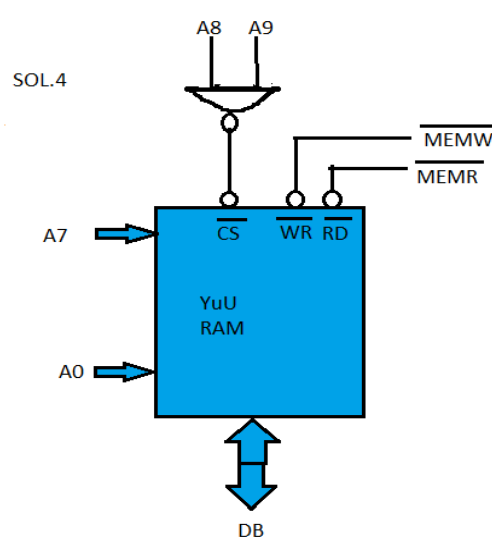
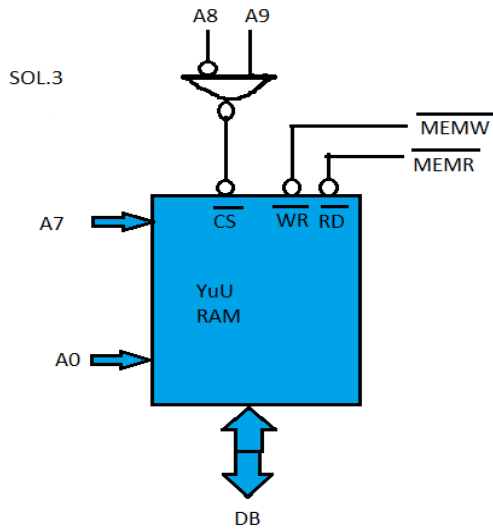
2. Decode the higher AB to generate unique address to the memory chip which activate the CS
3. Generate the control signals MEMR & MEMW by combining **RD** , **WR** with IO/M

Ex(1) :Design a cct. That connect $\frac{1}{4}$ k of RAM chip to a μ P with AB= 10 bits , DB=8 bits.
Draw the memory map.



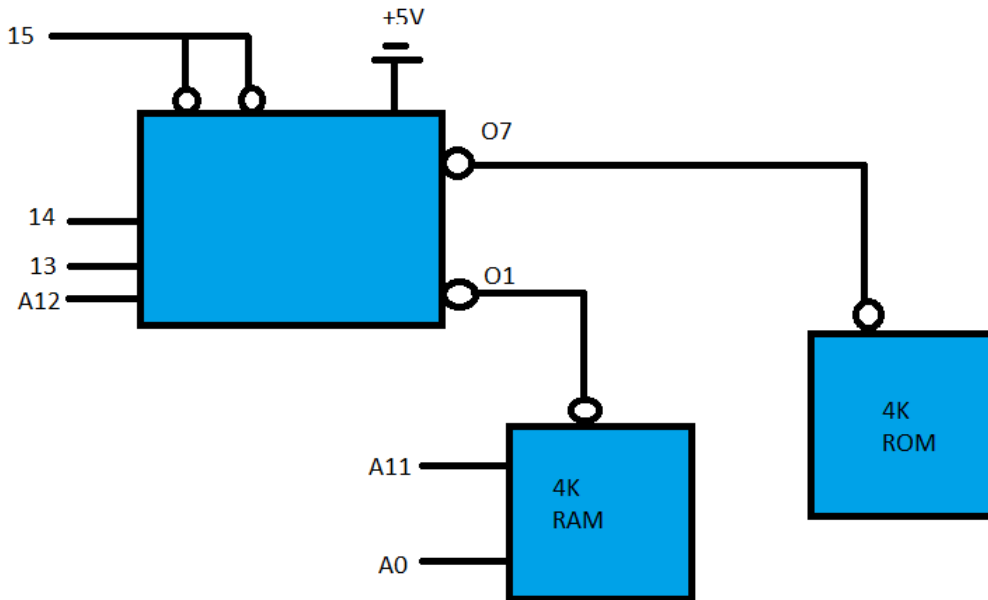
SOL.





					High order		Low order									
					A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀		
<u>Sol1</u>	O	O	O	H	=	0	0	0	0	0	0	0	0	0	0	1 st
	O	F	F	H	=	0	0	1	1	1	1	1	1	1	1	Last location
<u>Sol2</u>	1	O	O	H	=	0	1	0	0	0	0	0	0	0	0	
	1	F	F	H	=	0	1	1	1	1	1	1	1	1	1	
<u>Sol3</u>	2	O	O	H	=	1	0	0	0	0	0	0	0	0	0	
	2	F	F	H	=	1	0	1	1	1	1	1	1	1	1	
<u>Sol4</u>	3	O	O		=	1	1	0	0	0	0	0	0	0	0	
	3	F	F		=	1	1	1	1	1	1	1	1	1	1	

Ex: What will be the range of address & foldback space if exist for each memory chip.



1. Replace 4 K RAM by 1 K SRAM

$$A_{11}A_{10} = XX$$

2. Replace 4 K ROM by 512 ROM

$$A_{11}A_{10}A_9 = XXX$$

3. Replace A_{12} by A_{15}

4. Connect E1 & E2 to ground

5. Exchange CS of RAM to O_5

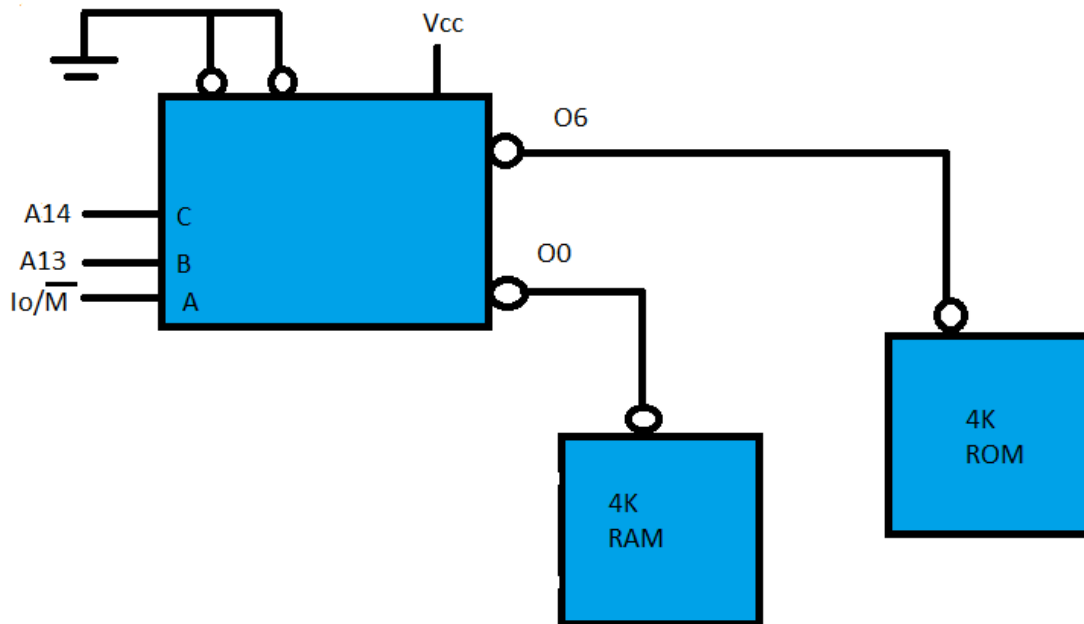
6. Connect E_3 to A_{15}

7. Connect 4K $\longrightarrow$ 2K with linear decode

8085 Microprocessor Architecture

Ex (2): For the same cct in Ex1, find the range of addresses, fold back space & type of decoding for each memory chip if:

- A) Connect E_3 to V_{cc}
- B) Can we connect $I/O/M$ to E_3 and RD , WR pins of each chip to RD & WR signal?
- C) Replace RAM by 2K RAM
- D) Replace ROM by 1 K EPROM
- E) Connect A to V_{cc} and eliminate A_{12} .
- F) For the same ROM of addresses in Ex1, modify the cct while connecting E_3 to A_{15} . So that RAM the range of ram addresses equals to $C000 \rightarrow CFFF$



Reference:

8085 μ p architecture and programming_Gonkar

Lesson Five

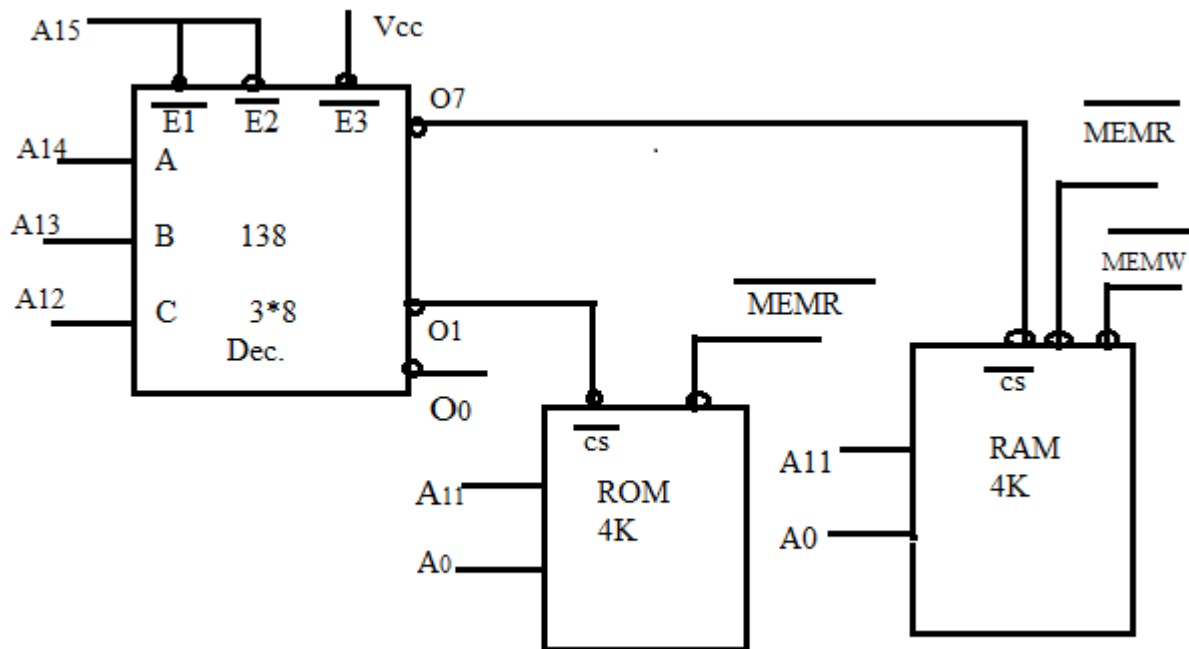
8085 Memory Map II

Lesson Objectives:

- To determine the addresses range for each memory chip
- To draw the memory map
- To design a system that fulfills the requirements.
- To compare between absolute and linear decoding.

- Pre Test:
- Define the memory map
- Illustrate the Basic concept in memory interfacing.
- How many address lines needed to address 2048 bytes of memory.

For the circuit shown below:



1) Find the range of addresses for each memory chip & draw the memory map.

Ans:

$$2^x = 4K \implies x=12 \quad A_{11} \longleftarrow A_0$$

ROM	A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀
1 st location	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
Last location	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1

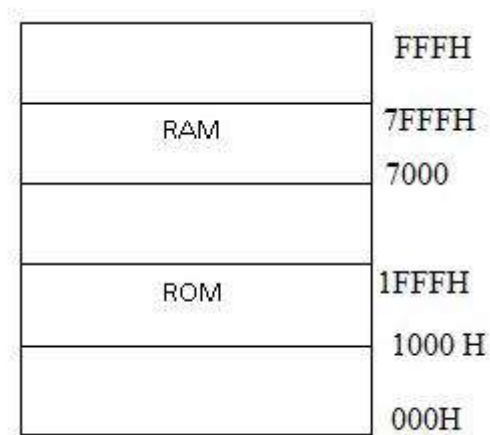
Range of addresses 1000H $\longrightarrow$ 1FFFH

8085 Microprocessor Architecture

RAM	A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀
1 st location	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0
last location	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Range of addresses 7000 H $\longrightarrow$ 7FFFH

Memory map



2) H.W. Find the range of addresses and draw the memory map if we connect (1) the CS of the ROM to O/P 4 (2) if we connect CS of the RAM to O/P 5.

3) Repeat step 1 if we:

- Exchange A14 with A15.
- Exchange A12 with A15.

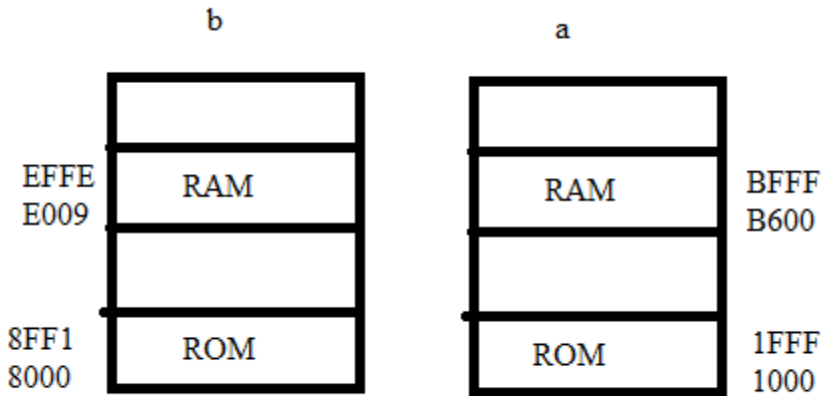
(a) ROM $\longrightarrow$ the range will not change 1000H $\longrightarrow$ 1FFFH because both E₁, E₂ and I/P of the decoder must be 0

8085 Microprocessor Architecture

RAM	A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀
1 st location	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0
Last location	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1

(b)

ROM	A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀
8000→8FFF	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	1	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1
RAM	A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀
E000→EFFF	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0
	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1



Absolute decoding:

5) H.W. repeat step 1 if we exchange A12 with A14.

6) Repeat step 1 if we exchange RAM by 1 K byte of EPROM.

RAM doesn't change 1000 → 7FFFH

	Higher order								Lower order							
ROM	A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀
E000 → EFFF	0	0	0	1	x	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	1	x	1	1	1	1	1	1	1	1	1	1	1

A₁₁ → (X) don't care

If A₁₁=0 → range of addresses 1000 → 17FF

If A₁₁=1 → range of addresses 1800 → 1FFF

Fold back space:

Linear decoding:

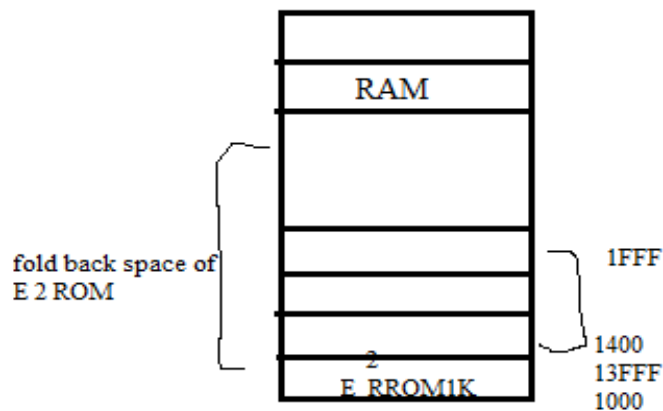
8085 Microprocessor Architecture

H.W. (7) Repeat step 1 if we exchange 4K byte of ROM by 1 K byte of EPROM

RAM doesn't change 7000 → 7FFFH

ROM	A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀
	0	0	0	1	X	X	0	0	0	0	0	0	0	0	0	0
	0	0	0	1	X	X	1	1	1	1	1	1	1	1	1	1

A ₁₁	A ₁₀	Range of addresses				
0	0	1000	→	13FF		
0	1	1400	→	17FF		
1	0	1800	→	1BFF		
1	1	1C00	→	1FFF		



Note that: (1000)=(1400)=(1800)=(1(100))

9) Repeat step 1 if we connect E1 & E2 to ground

8085 Microprocessor Architecture

ROM	A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀
	X	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
	X	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1

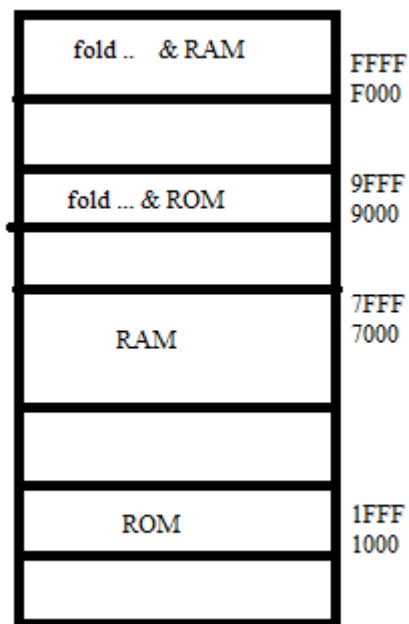
If A₁₅=0 $\Rightarrow$ 1000 $\rightarrow$ 1FFF range of addresses

If A₁₅=1 $\Rightarrow$ 9000 $\rightarrow$ 9FFF fold back space

RAM	A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀
	X	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0
	X	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

If A₁₅=0 $\Rightarrow$ 7000 $\rightarrow$ 7FFF range of addresses

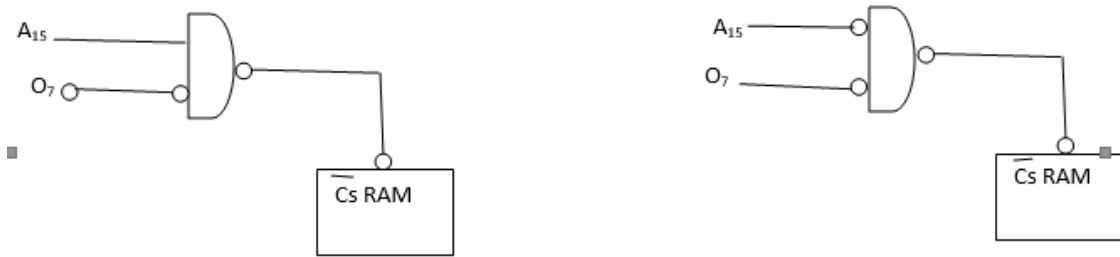
If A₁₅=1 $\Rightarrow$ F000 $\rightarrow$ FFFF fold back space



10) Repeat step 1 if we connect E1 & E2 to ground & I/P c to Vcc.

8085 Microprocessor Architecture

11) In step 9 and eliminate fold back fold space for the RAM by modifying the H/W.



12) Connect another RAM2 to the cct .so that the range of address will be 6800 6FFF .

What is the size of the RAM 2

Range of address	A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀
	0	1	1	0	X	0	0	0	0	0	0	0	0	0	0	0
	0	1	1	0	X	1	1	1	1	1	1	1	1	1	1	1
Fold back	A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀
	0	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0
	0	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1

All don't care the size of RAM 2 is 2K byte and the CS of RAM 2 is connected to O6.

Ex: H/W 13) Repeat step 12 if range off address = 3000 → 37FF

& fold back space = 3800 → 3FFF

Reference:

8085 μ p architecture and programming_Gonkar

Lesson Six

8085 Interfacing I/O Devices

Lesson Objectives:

- To determine the addresses range for each I/O port
- To draw the memory map
- To design a system that fulfills the requirements.
- To compare between peripheral and memory mapped.

Pre Test:

- Compare between peripheral and memory mapped
- Compare between common anode and common cathode 7 segment display.
- Write a program to read the data from input port and display it through output port.

Interfacing I/O Devices

❖ I/O devices are the communications channels between the MP and the outside world.

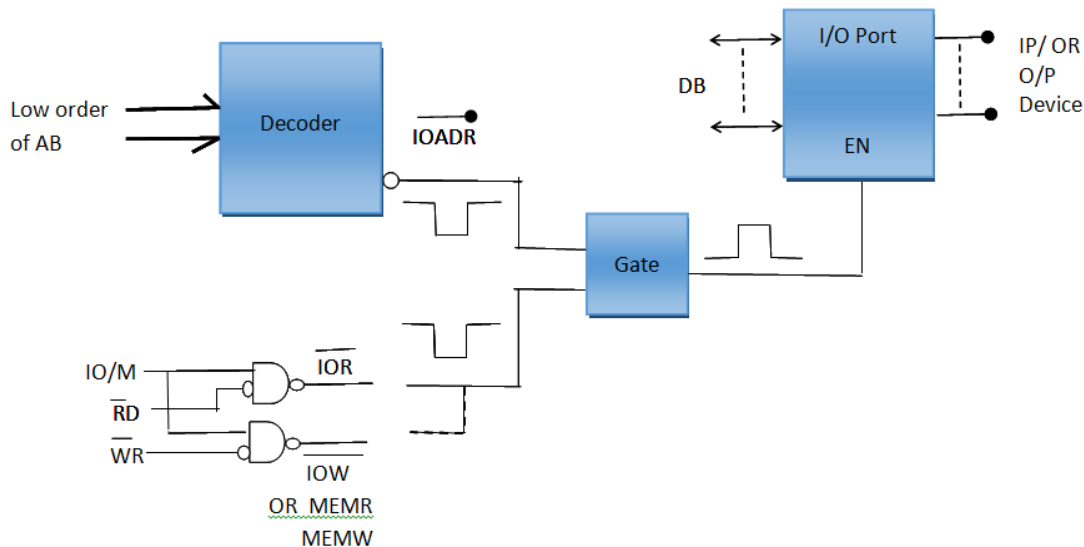
❖ Data transfer

- Parallel I/O mode (group of bits transferred simultaneously via group of lines)
- Serial I/O mode (one bit is transferred via single line at a time)

❖ Interfacing techniques

- Peripheral- mapped I/O
- Memory – mapped I/O

❖ Typical block diagram representing the interfacing between MP and I/O device.



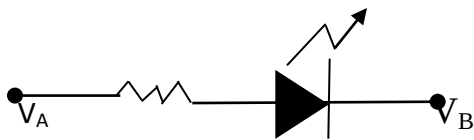
Peripheral- mapped	Memory- mapped
I/O device is identified by 8 bit ($A_7 \rightarrow A_0$) only i.e. 8085 MP can address 256 I/P 8256 O/P device.	I/O device is identified by 16 bit ($A_{15} \leftarrow A_0$) 8085 MP considers each device as a memory location.
IOR & IOW are used to control the data transfer.	MEMR & MEMW are used to control the data transfer.
IN & OUT instruction are used for programming.	ALL memory related instructions for data transfer can be used e.g. MOV A, M, STA, LDA etc.

BUS timing during the execution of IN & OUT instruction:

Ex. OUT FF Stored in M.L 2050 H, the content of Acc. Equals to (BBH)	2 byte instruction with opcode (D3)
	During the execution of this instruction the content of register (A) is transferred to the O/P device through O/P port which is addressed by (FF)
	Need O/P code fetch & memory read machine cycles for fetching part and I/O write machine cycle for execution part.

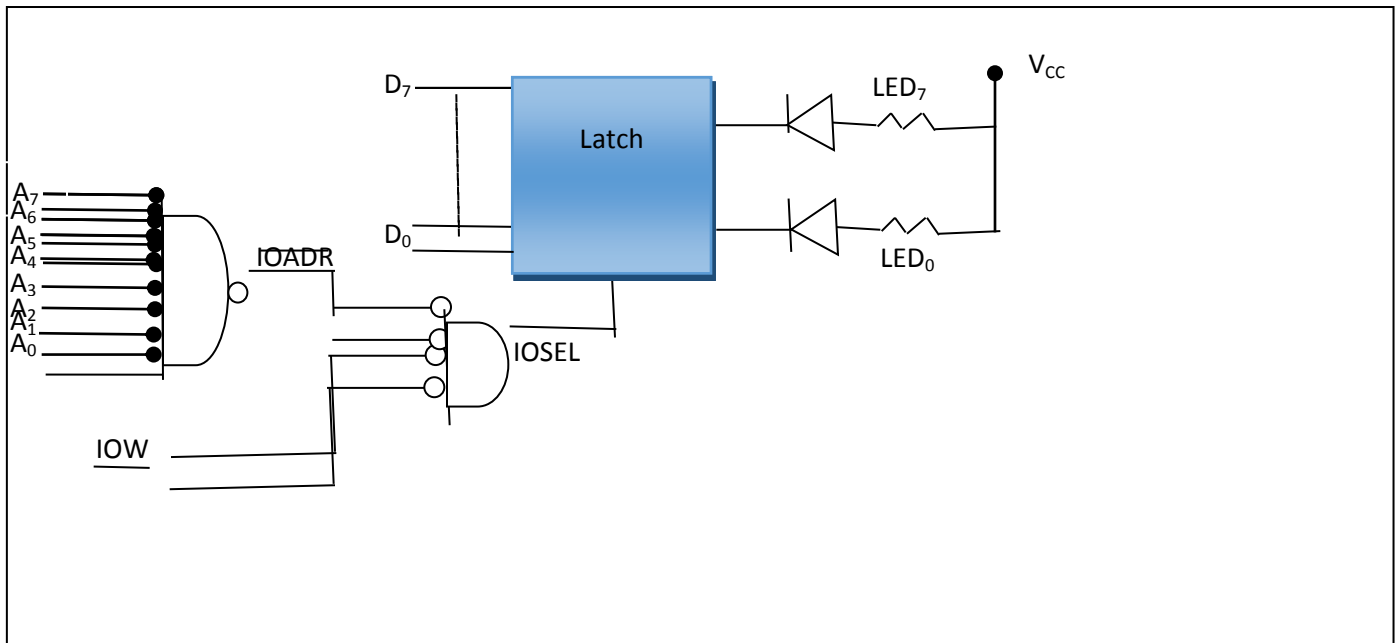
A. Interfacing O/P devices:

1. Lighting Emitting Diode (LED)



If $V_A > V_B \implies$ LED is in forward biasing $\implies$ LED emits light $\equiv$ logic (1)

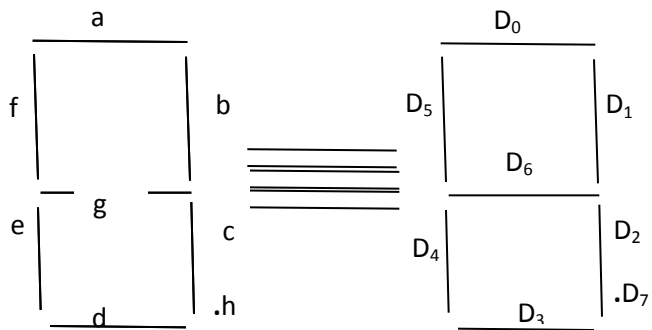
Ex: Analyze the cct. Shown below:



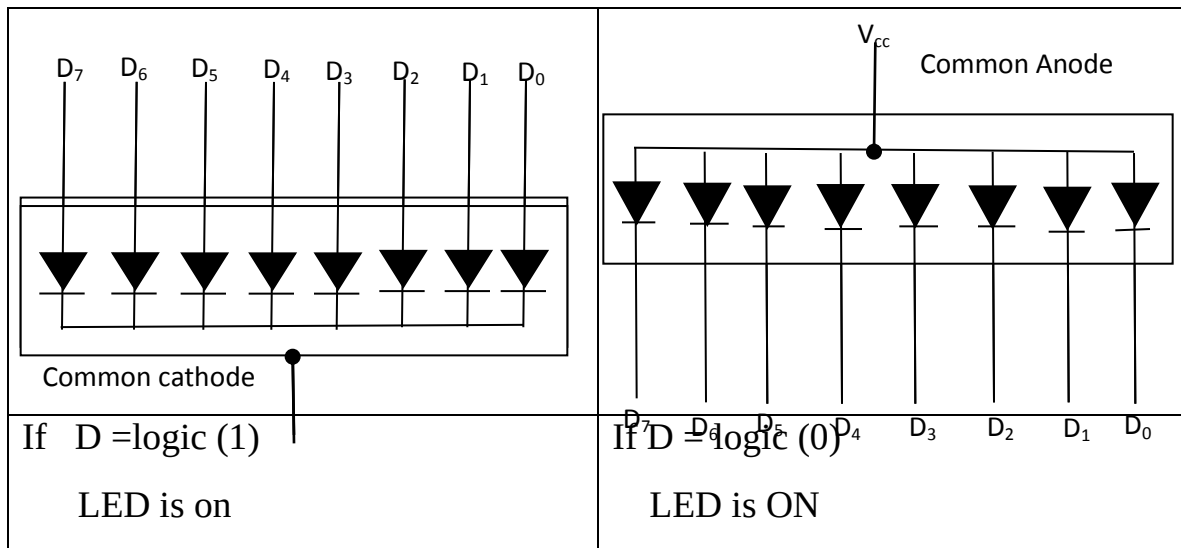
1. $A_0 \longrightarrow A_7$ are decoded to generate IOADR active low only when $A_7 \longleftarrow A_0 = 00000001_2 = 01H$.
2. IOADR is combined with IOW to generate IOSEL which is used to enable the O/P port (latch)
3. Since IOW is used to control the data transfer, the interfacing technique is peripheral – mapped and the O/P port is identified only by 8 lines $A_7 \longleftarrow A_0$.
4. The data is transferred from the Acc. through the latch in such a way that:
If the logic level on the data line is (1), then the status of the corresponding LED is off and vice versa.
5. The type of decoding is absolute since all lines are used ($A_0 \longleftarrow A_7$)

2. Segment Display:

Each segment is a LED



There are two types of 7segment display as below:-



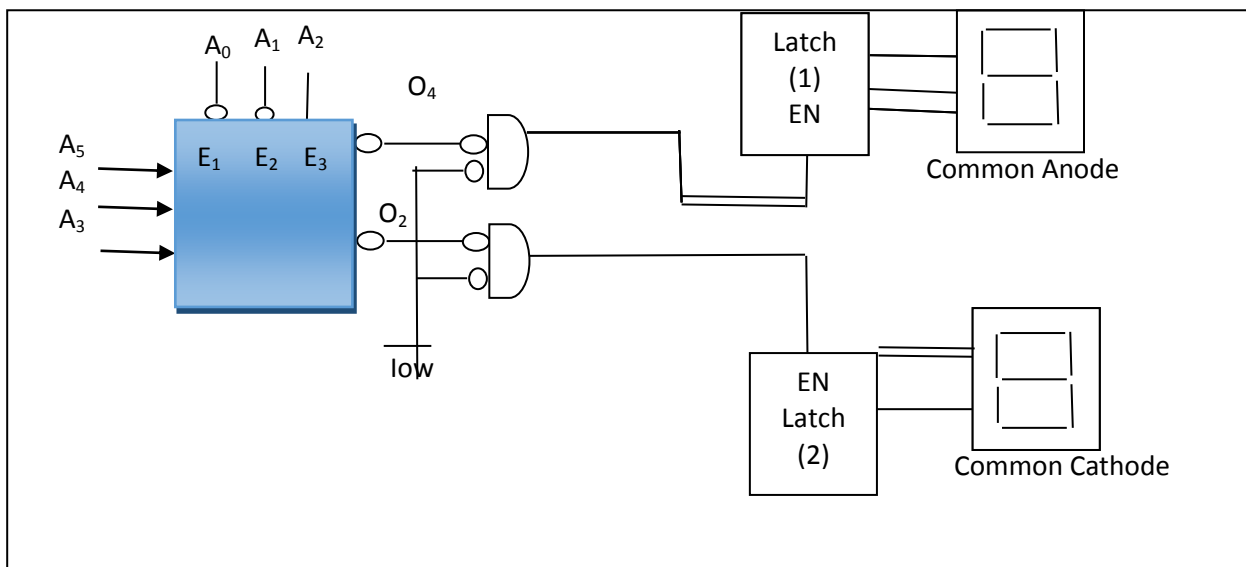
Ex. No. 3

DB= BOH

Ex. No. 3

DB= 4FH

Ex. What is the range of addresses, interfacing technique & type of decoding for each port in the following cct. What will be display if (Acc) = 06H



Sol: Both O/P port are interfaced through peripheral techniques both ports are identified by lines $A_7 \leftarrow A_0$.

Latch. (2)

Latch (2) can be address one of the following:

14H, 54H, 94H, D4H (linear decoding)

A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0
X	X	0	1	0	1	0	0

8085 Microprocessor Architecture

Latch (1)

Latch (1) can be addressed by:

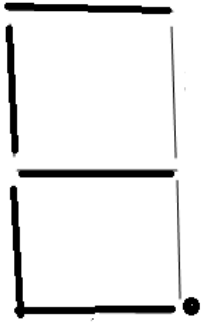
24H, 64H, A4H, E4H (linear decoding)

To transfer (Acc) through latch (1)

A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀
X	X	1	0	0	1	0	0

MVI A,06

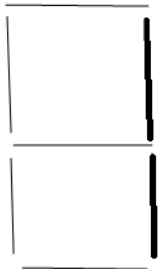
OUT 24 H, OUT A4H, OUT E4H, OUT 64H



E will be displayed

- To transfer (Acc) through latch (2), MVI A 06

OUT 14, OUT 54, OUT 94, OUT D4

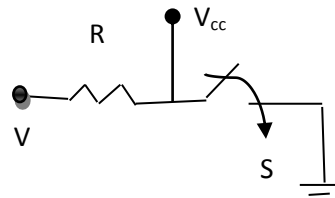


No. (1) will be displayed

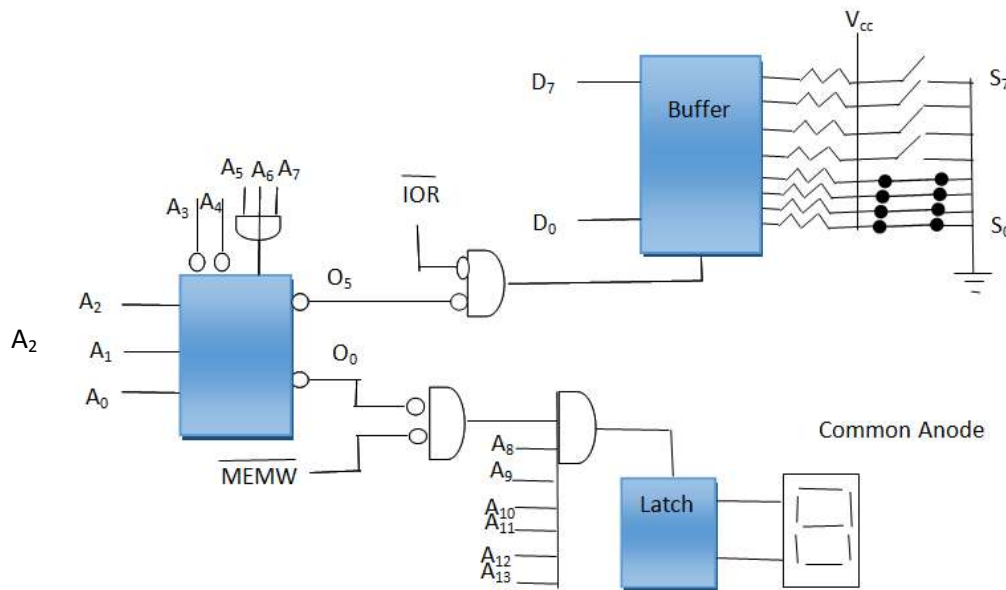
B. Input device

Switches

When S is closed $\rightarrow V=0V = \text{logic } (0)$



Ex. What is the address, type of decoding and interfacing for each port.



Sol.

- For buffer, the interfacing technique is peripheral- mapped and can be addressed by E5H (absolute decoding)

A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀
1	1	1	0	0	1	0	1

- For latch, the interfacing results in multiple addresses 3FE0, 7FE0, BFE0 and FFE0.

A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀
X	X	1	1	1	1	1	1	1	1	1	0	0	0	0	0

Ex. Write a program to transfer the switches status from I/P port to 7-Segment display. What will be displayed ?

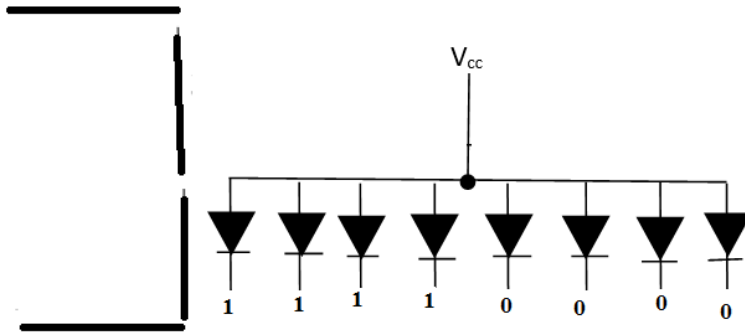
Sol .

IN E5

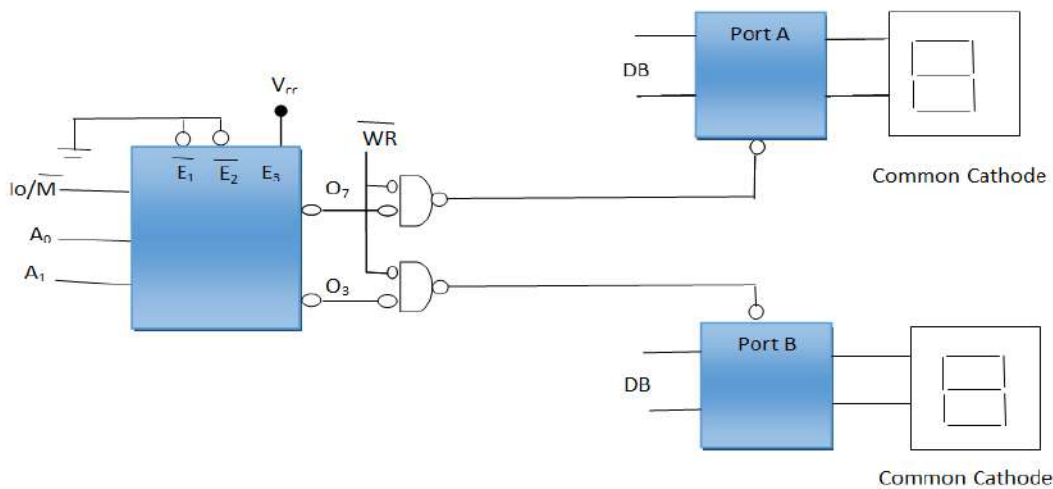
LXI H, 3FE0

MOV M, A

The input data on DB=F0_H



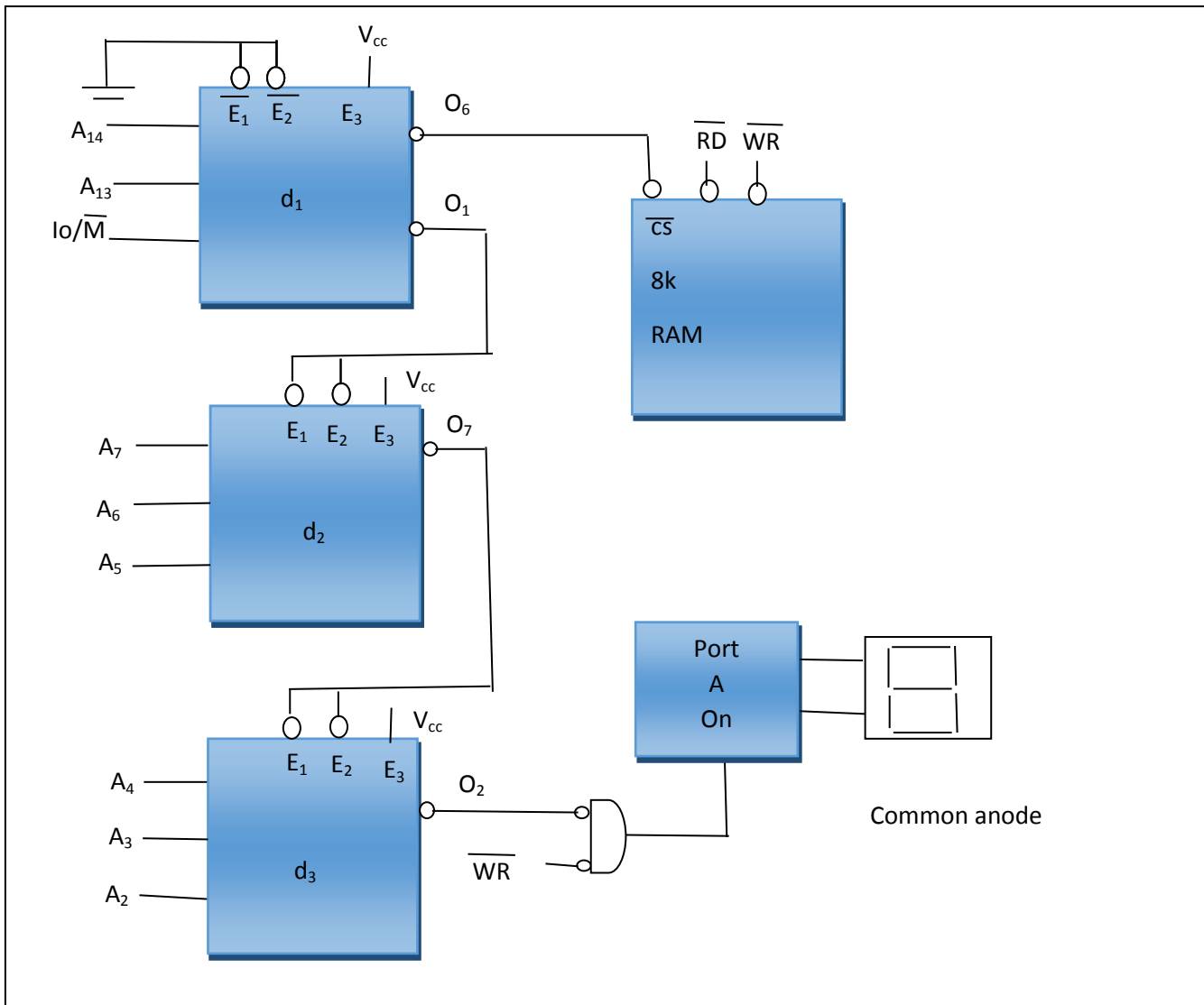
H.W. For the circuit shown below: Find the port address , type of decoding & interfacing technique for each or. Draw the memory map. Write a program to display No.8 trough port A&B (assume all don't care to be logic (1))



8085 Microprocessor Architecture

EX.: For the cct. Shown below:

Draw the memory map and write a program to display a letter “A”.



Reference:

8085 μ p architecture and programming_Gonkar

Lesson Seven

8085 Time Delay and Counters

Lesson Objectives:

- To introduce the types of time delay.
- To write a program that generates specified time delay.
- To learn how to determine the time delay based on one register or pair of registers.

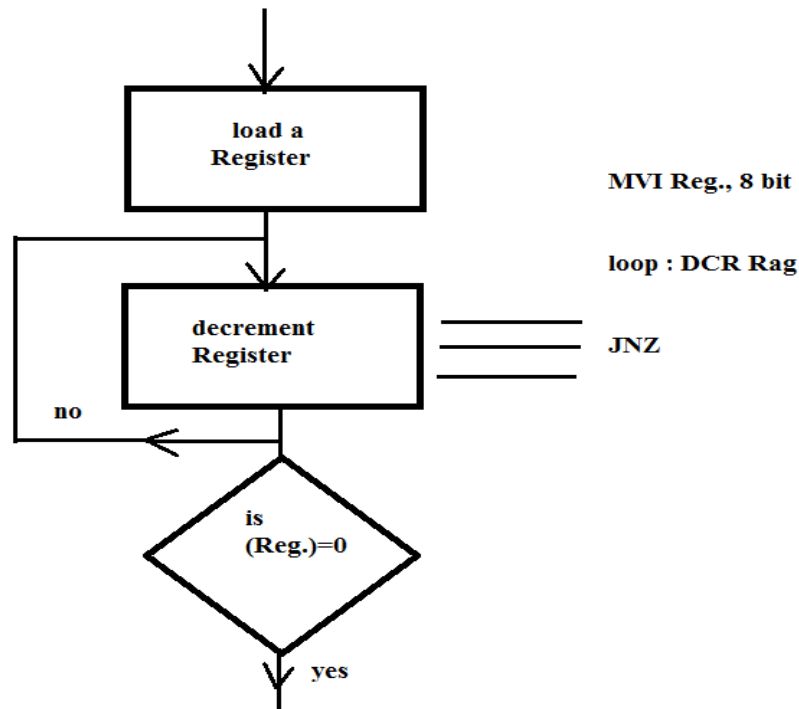
Pre Test:

- What are the types of time delay can be produced by 8085 μ p.
- What is the maximum time delay generated by pair of register.
- Write a program to generate 5msec time delay

Time Delay and Counters

Time delay: a register (or pair of registers) is loaded with a number depending on the time delay required, then the register is decremented until it equals to zero. Also, (NoP) instruction can be used to generate time delay equals to 4 T- states.

1- Time Delay With One Register



Ex.:

What is time delay generated by the following program if the clock frequency = 2MHz?

MVI C, FF → 7T
 Loop: DCR C. → 4T
 JNZ Loop → 10/7T

Total delay = time outside the loop + loop execution time

$$T_D = T_O + T_L$$

$$T_O = 7 * 1/2\text{MHz} = 3.5 \mu\text{sec}$$

$$T_L = (\sum (N_{Tstates} * 1/f)) * N, \text{ where } N \text{ is the number loaded to the register}$$

$$= ((4+10) * (1/2\text{MHz})) * 255 \quad \text{Note FFH} = 255_{10}$$

$$= 1785 \mu\text{sec} \approx \underline{\underline{1.8 \text{ msec}}}$$

$$T_D \text{ approximate} = 3.5 \mu\text{sec} + 1785 \mu\text{sec} = 1788.5 \mu\text{sec}$$

$$T_D \text{ accurate} = 3.5 \mu\text{sec} + (1785 - 1.5) \mu\text{sec} = 1787 \mu\text{sec}$$

Note: The max. time delay is generated by one register loaded with (FFH) and equals to 1787 μsec (~ 1.8 msec).

H.W. What will be the time delay generated with one register if the number loaded to that register equals to :

1- 78 H, 2- A3 H, 3- 5EH

EX.: Write a program to generate time delay equals to (1 msec).

MVI C, ? $\longrightarrow$ 7 T
 *: DCR C $\longrightarrow$ 4 T
 JNZ * $\longrightarrow$ 10 / 7 T

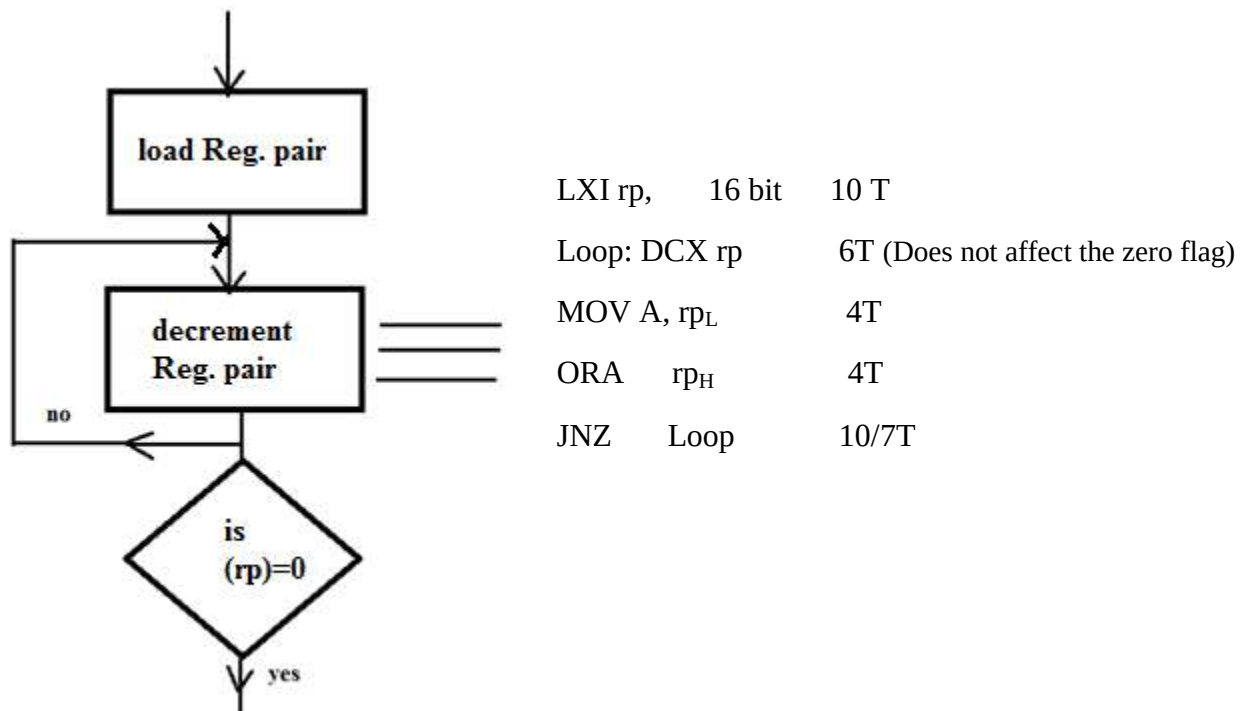
$T_D = T_O + T_L = 1 \text{ msec}$

$T_O = 3.5 \mu\text{sec}$

$T_L = 1000 \mu\text{sec} - 3.5 \mu\text{sec} = 996.5 \mu\text{sec}$

$996.5 = N_{10} * 14 * (1/2\text{MHz}), \quad N_{10} = 142.35 \sim 8 \text{ EH}$

2- Time Delay With Pair Register



EX.:

What is the max. time delay can be generated with pair of register?

Solution:

The max. time delay can be generated when the pair of register is loaded by FFFFH (=65535₁₀)

LXI B, FFFF	10T
*: DCX B	6T
MOV A, C	4T
ORA B	4T
JNZ *	10/7 T

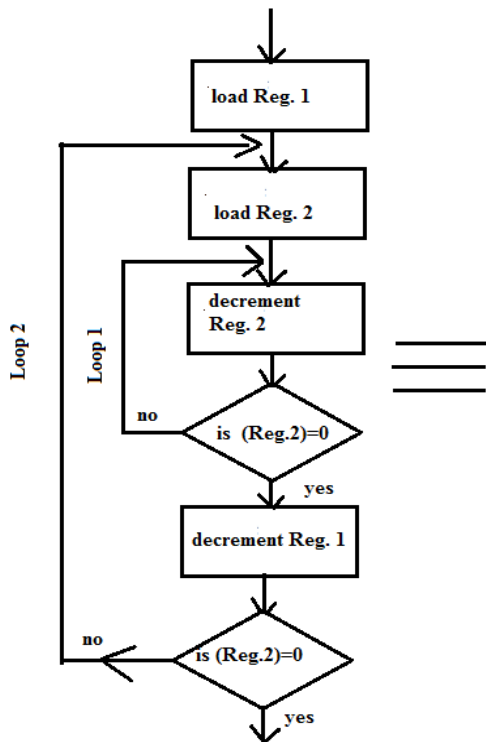
$$T_D = T_O + T_L$$

$$T_D = 10 * (1/2) = 5 \mu\text{sec}$$

$$T_L = (6+4+4+10) * (1/2) * 65535$$

$$= 786420 \mu\text{sec} \approx 786 \text{ msec}$$

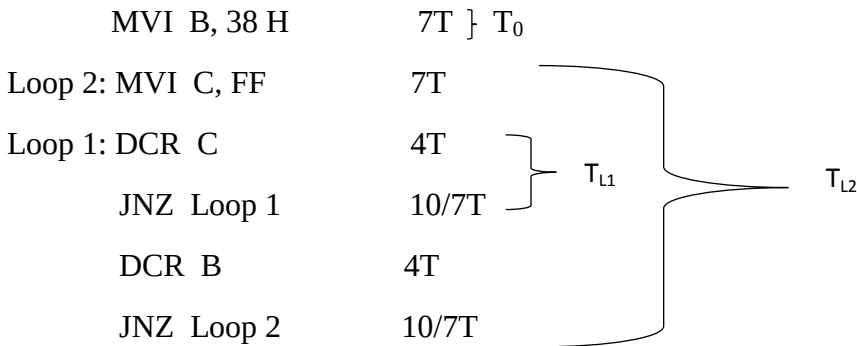
3- Time Delay Generated by Loop within Loop



MVI Reg. ₁ , 8 bit	7T	
Loop 2: MVI Reg. ₂ , 8 bit	7T	
Loop 1: DCR Reg. ₂ ,	4T	} T _{L1}
JNZ Loop 1	10/7T	
DCR Reg. ₁	4T	} T _{L2}
JNZ Loop 2	10/7T	

EX.:

What is the time delay generated by following program?



Time delay (T_D) = T_O + T_L

$$T_O = 7 * (1/2) = 3.5 \mu\text{sec}$$

$$\begin{aligned}
 T_L = T_{L2} &= [T_{L1} + (\sum (T)_{L2} * 1/f)] * 56_{10} \quad (38H = 56_{10}) \\
 &= [T_{L1} + (7+4+10) * 1/2] * 56_{10} \\
 &= [T_{L1} + 10.5] * 56_{10}
 \end{aligned}$$

$$\begin{aligned}
 T_{L1} &= (\sum (T_{L1}) * 1/f) * 255_{10} \quad (FFH = 255_{10}) \\
 &= (14 * 1/2) * 255_{10} \\
 &\approx 1.8 \text{ msec} = 1783.5 \mu\text{sec}
 \end{aligned}$$

$$T_L = (1783.5 + 10.5) * 56_{10} = 100.46 \text{ msec}$$

$$T_D = 100460 \mu\text{sec} + 3.5 \mu\text{sec} \approx 101 \text{ msec}$$

Note: performing time delay has the following disadvantages:

- 1- Calculating time delay is tedious.
- 2- μp is occupied waiting the delay loops.
- 3- Needs accuracy.

However time delay can be generated by using extra chip (e.g. 8254), but at the cost of complexity.

µp Applications With Delay

Ex. (1): Design an up counter to continuously counting from 0 → 9₁₀ with a delay = 0.5 sec between each consecutive numbers, Display the number via o/p 33 H (f=2 MHz) .

Sol.:

```

Again:  MVI  B, 00
        MOV  A, B
Next:   OUT  33H
        LXI  H, N?
Delay:  DCX  H      6
        MOV  A, L    4
        ORA  H      4
        JNZ  Delay  10/7
        INR  B
        MOV  A, B
        CPI  0A
        JNZ  Next
        JMP  Again
    
```

T_D=0.5 sec

$$T_D = (6+4+4+10) * \frac{1}{2} * N$$

$$N = 0.5 \text{ sec} / 12$$

$$= 41666.6_{10} \approx A2C2H \text{ (note that } T_0 \text{ is ignored)}$$

Ex. (2):

Write a program to generate a sequence waveform with a period of (500) µsec if f= 2 MHz. (used D₀ to output the wave through O/P port 00H).

```

Next:   MVI  D, AA    7T
        MOV  A, D    4T →
        RLC          4T → To alternate Dn from 0 → 1, AA → 55 → AA →
        MOV  D, A    4T
        ANI  01      7T
        OUT  00      10T
        MVI  B, N?   7T
Delay:  DCR  B        4T
        JNZ  Delay  10/7 T
        JMP  Next    10
    
```

Mask D₁ → D₇ and display only D₀

$$T_D = T_O + T_L$$

$$T_O = (4+4+4+7+10+7+10) * \frac{1}{2M} = 46 * \frac{1}{2} = 23 \text{ µsec}$$

$$T_L = (4+ 10) * \frac{1}{2M} * N = 12 * N \text{ µsec}$$

$$500 \text{ µsec} = 23 \text{ µsec} + (12 * N) \text{ µsec}$$

$$N = 39.75_{10} \approx 40_{10} = 28 H$$

Examples

H.W.

What is the time delay generated by pair of register if it is loaded by: - 00FF, 3E05, A32F?

H.W.

Write a program to generate a time delay equals to (5 msec)?

H.W.: if Reg. (B) and (C) in the previous example are loaded with 23 H and 84 H respectively, what will be the time delay?

Reference:

8085 μ p architecture and programming_Gon

Lesson Eight

8085 Instruction Set

Lesson Objectives:

- To illustrate the instruction set of the 8085 μ p.
- To learn how to write several programs

Pre Test:

- Write a program count the odd numbers in a set of 10 numbers stored in ML starting at 2050.
- Write a program to add a set of 10 numbers stored in ML starting at 2050.

8085 Microprocessor Architecture

EX.1:

Write a program to transfer 10 bytes stored in M.LS starting at 2080 to another M.Ls starting 2090.

MVI L, 0A $\longrightarrow$ 10 bytes counter
LXI B, 2080 $\longrightarrow$ Source index
LXI D, 2090 $\longrightarrow$ Destination index

next: LDAX B ((BC)) $\longrightarrow$ (A)
 STDX D ((DE)) $\longleftarrow$ (A)
 INX B (BC) + 1 } increment index
 INX D (DE) + 1 }
 DCR L (L) - 1
 JNZ next If (L) $\neq$ 0 then transfer the next bytes
 HTL $\longrightarrow$ end

EX. 2:

Write a program to add 20 bytes stored in M.L starting at 2070 (discard the carry).

MVI C, 14 H $\longrightarrow$ 20 bytes counter
XRA, A $\longrightarrow$ Rest A_{CC}
LXI H, 2070 $\longrightarrow$ Memory index
* :ADD M $\longrightarrow$ (A) + (M) $\longrightarrow$ (A)
 INX H $\longrightarrow$ Increment index
 DCR C $\longrightarrow$ Decrement the counter
 JNZ * $\longrightarrow$ Check for 10 bytes completion go to add if not
 HLT $\longrightarrow$ end

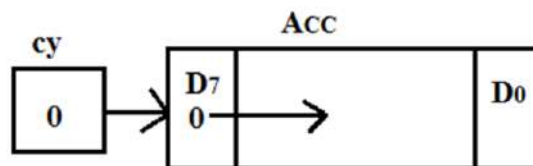
EX. 3:

Write a program to compare 2 number, which are stored in A_{cc} and Reg. B.

- 1- If (A_{cc}) > (B) $\longrightarrow$ reset the carry
- 2- If (A_{cc}) = (B) $\longrightarrow$ set the 1st bit in (B) using rotate instruction
- 3- If (A_{cc}) < (B) $\longrightarrow$ Reset the last bit in (B) using rotate instructions.

2000: CMP B $\longrightarrow$ Compare (A_{cc}) with (B)
 JC 2080 $\longrightarrow$ Compare (A_{cc}) < (B)
 JZ 2090 $\longrightarrow$ Compare (A_{cc}) = (B)
 STC $\longrightarrow$ Set carry
 CMC $\longrightarrow$ Complement carry } Reset the carry
 JMP 20A0

2080: STC }
 CMC } Load c_y by (0)
 MOV A, B
 RRC

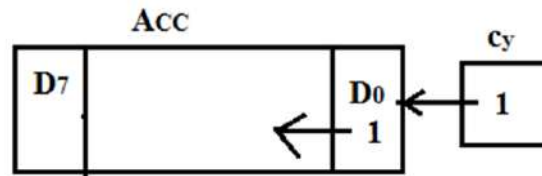


8085 Microprocessor Architecture

```
MOV B, A  
JMP 20A0
```

2090: STC

```
MOV A, B  
RLC  
MOV B, A  
JMP 20A0
```



20A0 RST5 (end) of the program

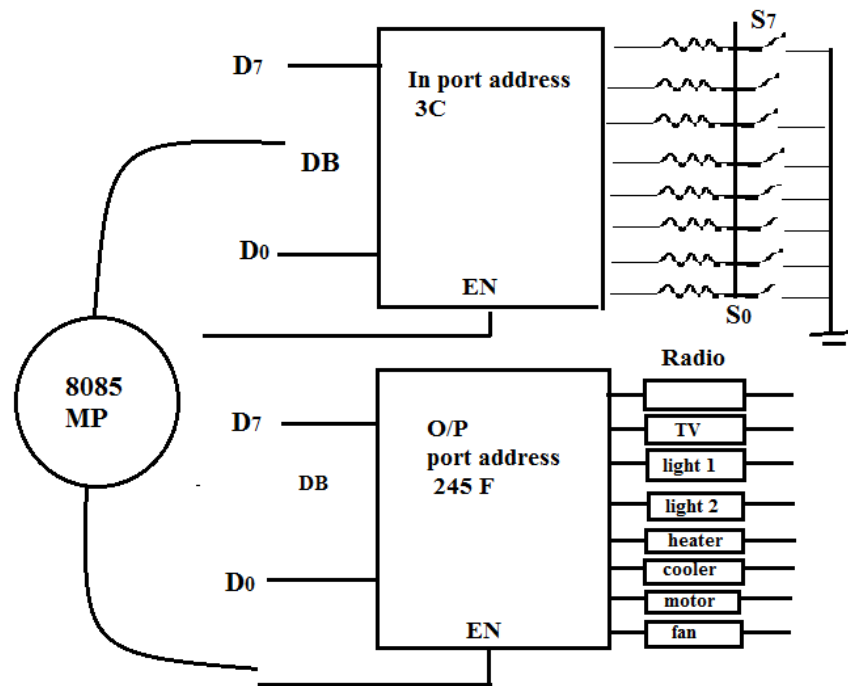
EX.4 write a program to switch ON the heater despite the corresponding switches status and return OFF the cooler despite corresponding switches status.

IN 3C → Transfer the switches status to DB then to A_{CC}

ORI 08 → Always the heater is switched ON

ANI FB → Always the cooler is switch OFF

STA 245F → Enable the O/P port to send the data.



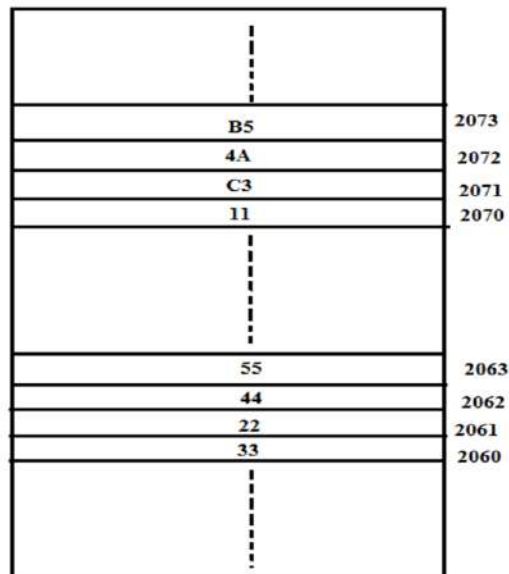
EX.5:

Write a program to add 2 numbers (24 bits each) stored in M.L starting 2060 and 2070 respectively, stored the results in M.L starting at 2080.

```

LXI D, 2080
LXI H, 2060
LXI B, 2070
LDAX B
ADD M
STAX D
INX H
INX B
INX D
LDAX B
ADC M
STAX D
INX H
INX B
INX D
LDAX B
ADC M
STAX D
INX H
INX B
INX D
LDAX B
ADC M
STAX D

```



```

55 44 66 33
B5 4A C3 11
-----
0A 8F 29 44

```

EX. 6:

8085 Microprocessor Architecture

Write a program to add 10 bytes stored in M.L starting at 2090 (discard the carry).

MVIC, OA	—————→	Counter for 10 bytes
XRA A	—————→	Reset A
LXI H, 2090	—————→	Memory index
*:ADD M	—————→	(A) + (M) —→ (A)
INX H	—————→	Increment index
DCR C	—————→	Decrement the counter
JNZ *	—————→	Check for 10 bytes completion and go to ADD if not
RSTS	—————→	Else end the program

EX.7 Write a program to:-

- 1- Complement the content of (A) and (C).
- 2- Set the 1st and the last bits of (E).
- 3- Reset the first 4 bits of (D).

Sol.

- 1- CMA —————→ Complement the contents of (A)

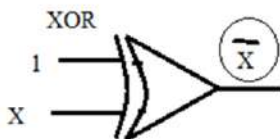
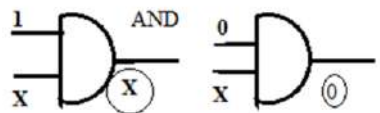
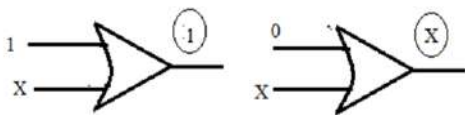
MOV A,C	} —————→	Complements the contents of (C)
XRI FF		
MOV C,A		

- 2- MOV A, E } —————→ Reg. E
- ORI 81

XXXX	XXXX
1 0 0 0	0 0 0 1
1X X X	X X X 1

- 3- MOV A, D } —————→ Reg. D
- ANI FO

XXXX	XXXX
1 1 1 1	0 0 0 0
XXXX	0 0 0 0



EX.8: Write a program to count the number of blank M.Ls that are contained within a set of 20 bytes stored in M.L starting at 2088. Store the results in 2090.

```
LXI H, 2088
MVI B, 00
```

8085 Microprocessor Architecture

```
    MVI C, 14
Next: MOV A, M
    CPI 00
    JNZ reject
    INR B
Reject: INX H
    DCR C
    JNZ Next
    MOV A, B
    STA 2090
```

EX.9: Compare 2 I/P values from port A and B which are addressed 3E and 072A respectively if (port A) > (port B), then reset flag else end program.

```
2000:  IN  3E
2002:  MOV  C, A
2003:  LDA  072A
2006:  CMP  C
2007:  JC  200B
2008:  JZ  200B
2009:  STC
200A:  CMC
200B:  RST 5
```

Examples

H.W.

Rewrite the program in Example (6) taken into account the carry.

Reference:

8085 μ p architecture and programming_Gonkar

Lesson Nine

8085 Interrupts

Lesson Objectives:

- To define Interrupt, introduce its types and procedure.
- To write a program to manipulate the maskable interrupts

Pre Test:

- Define Interrupt and its classification.
- What is the function of “EI” and “DI” instructions?
- Explain the 8085 Non-Vectored Interrupt Process.
- Explain the function of SIM and RIM to manipulated individual masks for RST 5.5, RST 6.5 and RST 7.5.

Interrupts

- ❖ Interrupt is a process where an external device can get the attention of the microprocessor. The process starts from the I/O device.
- ❖ Interrupts can be classified into two types:
 - Maskable (can be delayed)
 - Non-Maskable (cannot be delayed)
- ❖ Interrupts can also be classified into
 - Vectored (the address of the service routine is hard-wired)
 - Non-vectored (the address of the service routine needs to be supplied externally)
- ❖ An interrupt is considered to be an emergency signal. The microprocessor should respond to it as soon as possible.
- ❖ When the Microprocessor receives an interrupt signal, it suspends the currently executing program and jumps to an Interrupt Service Routine (ISR) to respond to the incoming interrupt. Each interrupt will most probably have its own ISR.

Responding to Interrupts

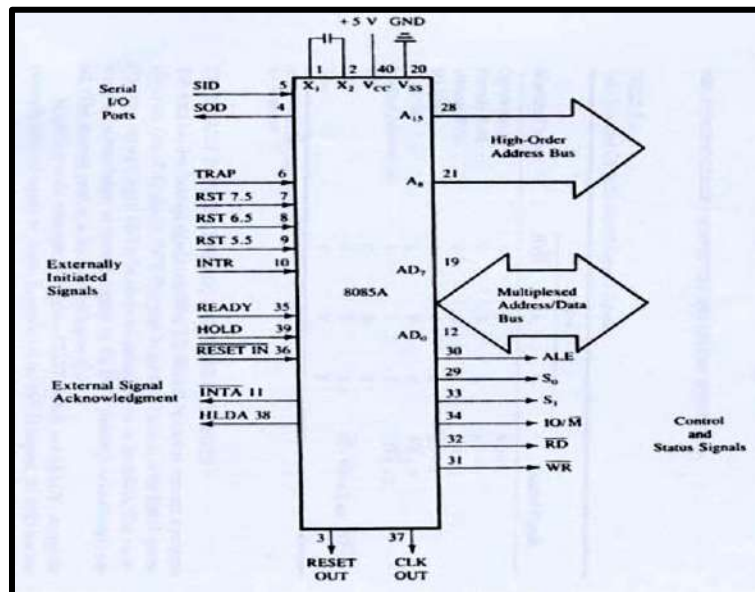
Responding to an interrupt may be immediate or delayed depending on whether the interrupt is maskable or non-maskable and whether interrupts are being masked or not.

There are two ways of redirecting the execution to the ISR depending on whether the interrupt is vectored or non-vectored. The vector is already known to the Microprocessor, while in the non-vectored type, the device will have to supply the vector to the Microprocessor

The 8085 Interrupts

The maskable interrupt process in the 8085 is controlled by a single flip flop inside the microprocessor. This Interrupt Enable flip flop is controlled using the two instructions “EI” and “DI”.

The 8085 has a single Non-Maskable interrupt. The non-maskable interrupt is not affected by the value of the Interrupt Enable flip flop.



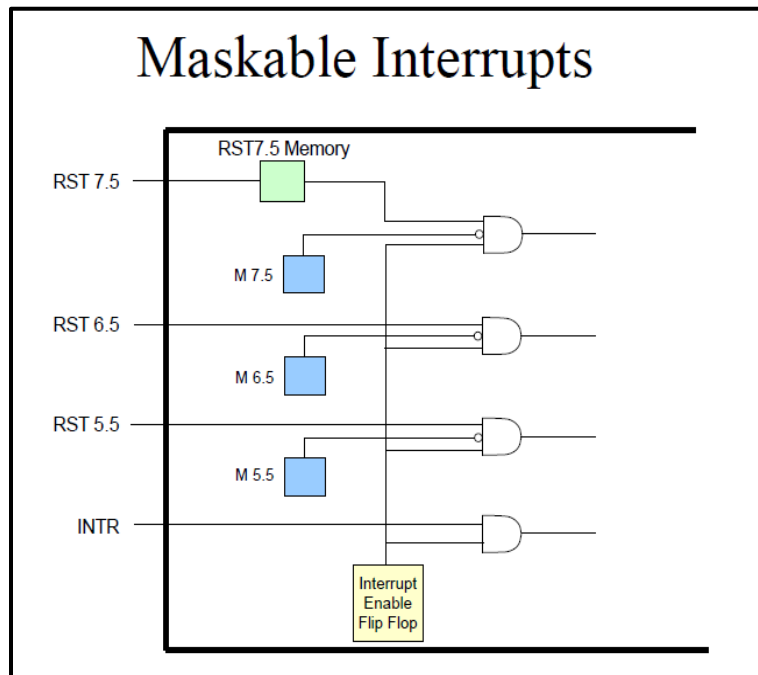
The 8085 has 5 interrupt inputs:

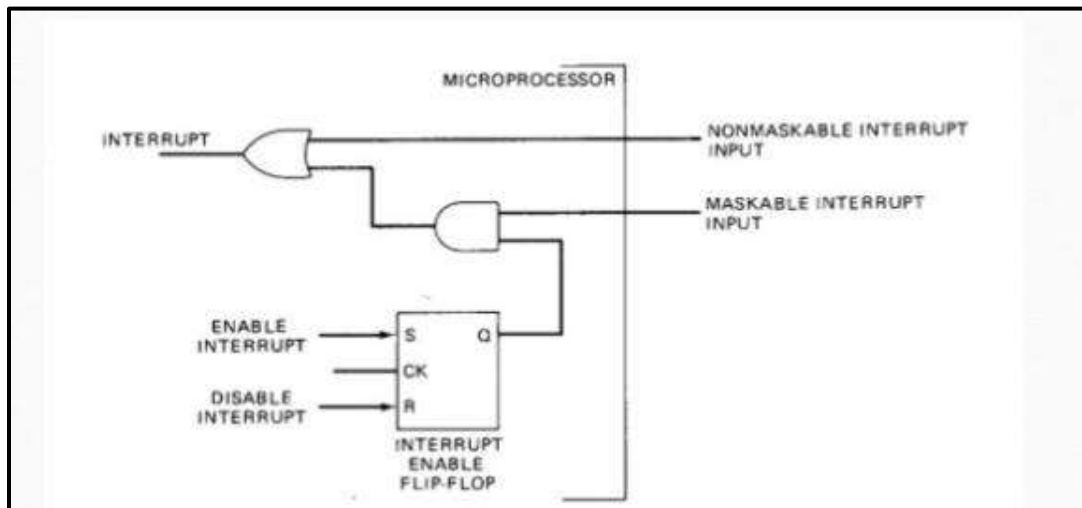
1) The INTR input. The INTR input is the only non-vectored interrupt. INTR is maskable using the EI/DI instruction pair.

2) RST 5.5, RST 6.5, RST 7.5 are all automatically vectored. RST 5.5, RST 6.5, and RST 7.5 are all maskable.

3) TRAP is the only non-maskable interrupt in the 8085. TRAP is also automatically vectored.

Interrupt name	Maskable	Vectored
INTR	Yes	No
RST 5.5	Yes	Yes
RST 6.5	Yes	Yes
RST 7.5	Yes	Yes
TRAP	No	Yes





An interrupt vector is a pointer to where the ISR is stored in memory.

All interrupts (vectored or otherwise) are mapped onto a memory area called the Interrupt Vector Table (IVT). The IVT is usually located in memory page 00(0000H -00FFH).

The purpose of the IVT is to hold the vectors that redirect the microprocessor to the right place when an interrupt arrives. The IVT is divided into several blocks. Each block is used by one of the interrupts to hold its “vector”

The 8085 Non-Vectored Interrupt Procedure

1. The interrupt process should be enabled using the EI instruction.
2. The 8085 checks for an interrupt during the execution of every instruction.
3. If there is an interrupt, the microprocessor will complete the executing instruction, and start a RESTART sequence.
4. The RESTART sequence resets the interrupt flip flop and activates the interrupt acknowledge signal (INTA).

5. Upon receiving the INTA signal, the interrupting device is expected to return the op-code of one of the 8 RST instructions.

6. When the microprocessor executes the RST instruction received from the device, it saves the address of the next instruction on the stack and jumps to the appropriate entry in the IVT.

7. The IVT entry must redirect the microprocessor to the actual service routine.

8. The service routine must include the instruction EI to re-enable the interrupt process.

9. At the end of the service routine, the RET instruction returns the execution to where the program was interrupted.

❖ The 8085 recognizes 8 RESTART instructions: RST0 -RST7. each of these would send the execution to a predetermined hard-wired memory location:

Restart instruction	OP code	Binary code								Memory location
		D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀	
RST0	C7	1	1	0	0	0	1	1	1	0000
RST1	CF	1	1	0	0	1	1	1	1	0008
RST2	D7	1	1	0	1	0	1	1	1	0010
RST3	DF	1	1	0	1	1	1	1	1	0018
RST4	E7	1	1	1	0	0	1	1	1	0020
RST5	EF	1	1	1	0	1	1	1	1	0028
RST6	F7	1	1	1	0	1	1	1	1	0030
RST7	FF	1	1	1	1	1	1	1	1	0038

Restart Sequence

The restart sequence is made up of three machine cycles.

In the 1st machine cycle: The microprocessor sends the INTA signal.

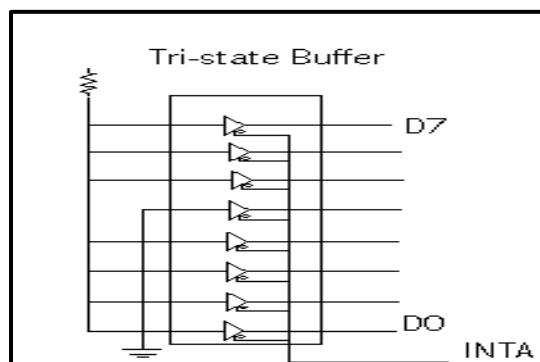
While INTA is active the microprocessor reads the data lines expecting to receive, from the interrupting device, the opcode for the specific RST instruction.

In the 2nd and 3rd machine cycles: the 16-bit address of the next instruction is saved on the stack. Then the microprocessor jumps to the address associated with the specified RST instruction.

EX: RST5

During the interrupt acknowledge machine cycle, (the 1st machine cycle of the RST operation): The Microprocessor activates the INTA signal. This signal will enable the Tri-state buffers, which will place the value EFH on the data bus. Therefore, sending the Microprocessor the RST 5 instruction.

The RST 5 instruction is exactly equivalent to CALL 0028H



Issues in Implementing INTR Interrupts

1) How long must INTR remain high?

The microprocessor checks the INTR line one clock cycle before the last T-state of each instruction. The interrupt process is Asynchronous.

The INTR must remain active long enough to allow for the longest instruction. The longest instruction for the 8085 is the conditional CALL instruction which requires 18 T-states.

Therefore, the INTR must remain active for 17.5 T-states.

2) How long can the INTR remain high?

The INTR line must be deactivated before the EI is executed. Otherwise, the microprocessor will be interrupted again.

The worst case situation is when EI is the first instruction in the ISR.

Once the microprocessor starts to respond to an INTR interrupt, INTA becomes active (=0).

Therefore, INTR should be turned off as soon as the INTA signal is received

3) Can the microprocessor be interrupted again before the completion of the ISR?

As soon as the 1st interrupt arrives, all maskable interrupts are disabled.

They will only be enabled after the execution of the EI instruction.

Therefore, the answer is: “only if you allow it to”.

If the EI instruction is placed early in the ISR, other interrupt may occur before the ISR is done.

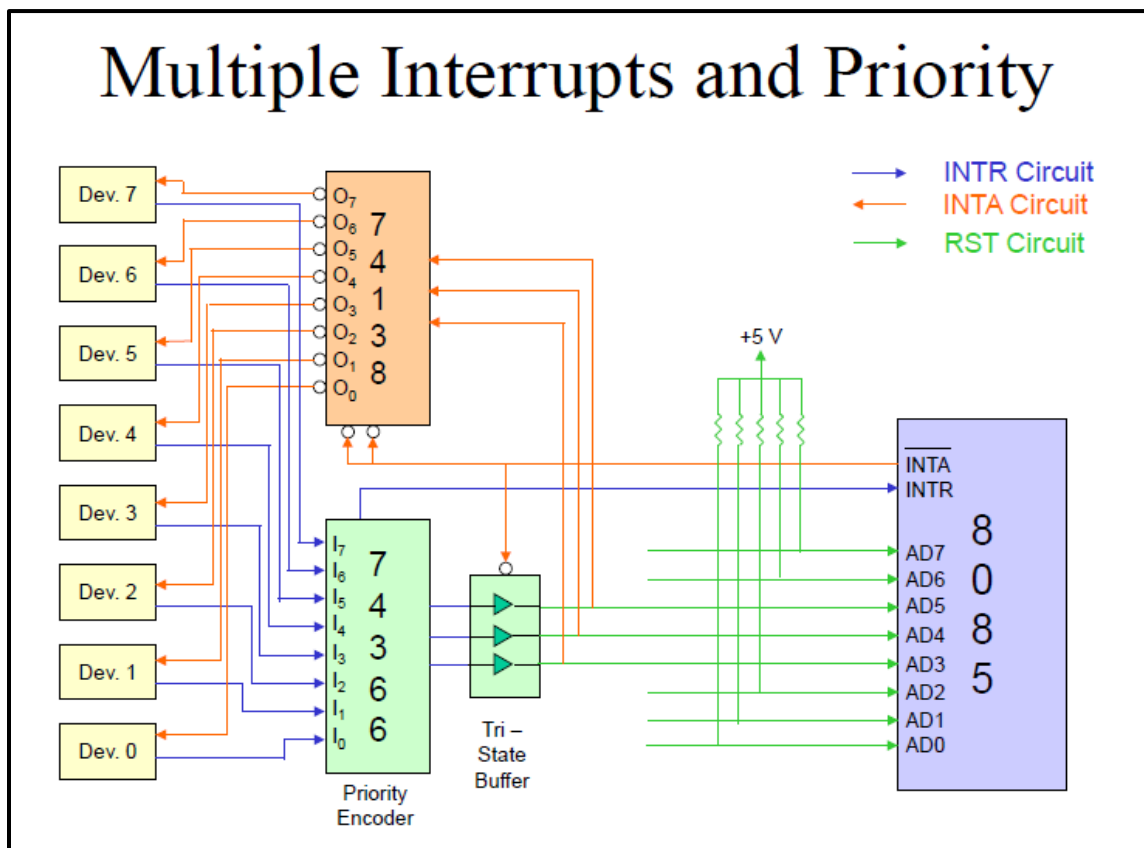
Multiple Interrupts & Priorities

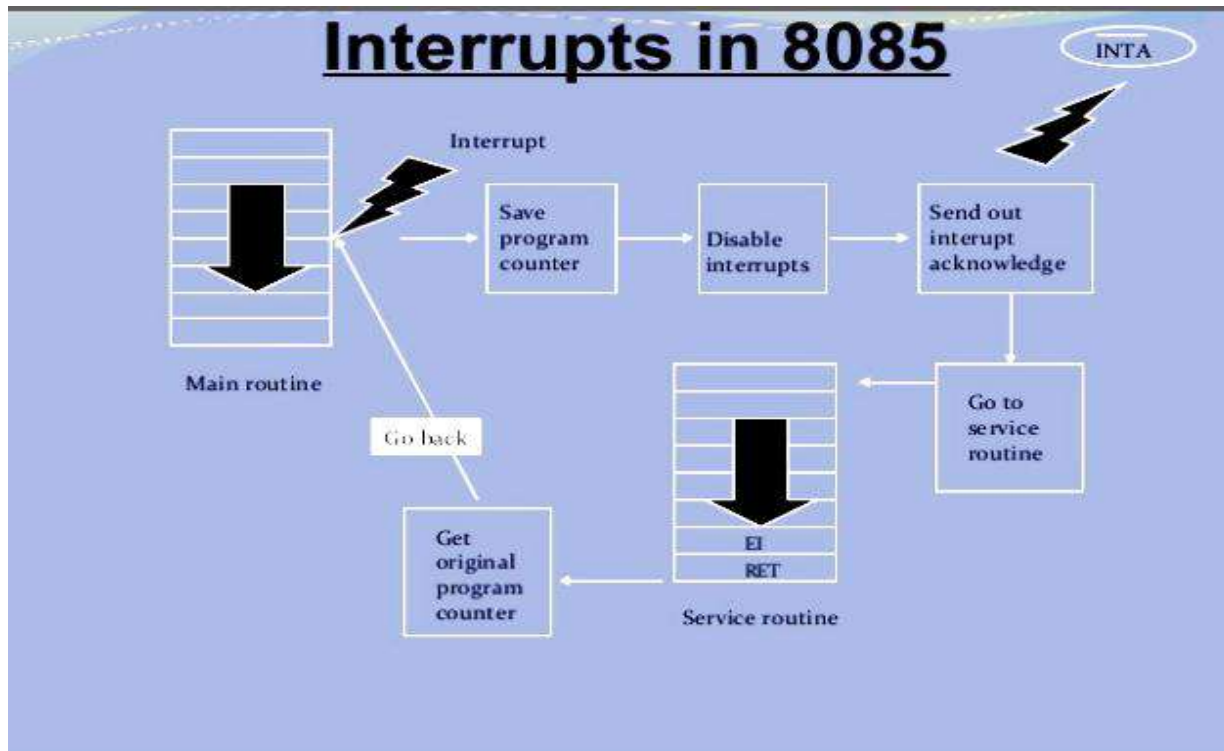
How do we allow multiple devices to interrupt using the INTR line?

The microprocessor can only respond to one signal on INTR at a time. Therefore, we must allow the signal from only one of the devices to reach the microprocessor. We must assign some priority to the different devices and allow their signals to reach the microprocessor according to the priority.

Note that the opcodes for the different RST instructions follow a set pattern. Bit D5, D4 and D3 of the opcodes change in a binary sequence from RST 7 down to RST 0. The other bits are always 1.

This allows the code generated by the 74366 to be used directly to choose the appropriate RST instruction. The one drawback to this scheme is that the only way to change the priority of the devices connected to the 74366 is to reconnect the hardware.





The 8085 Maskable/Vectored Interrupts

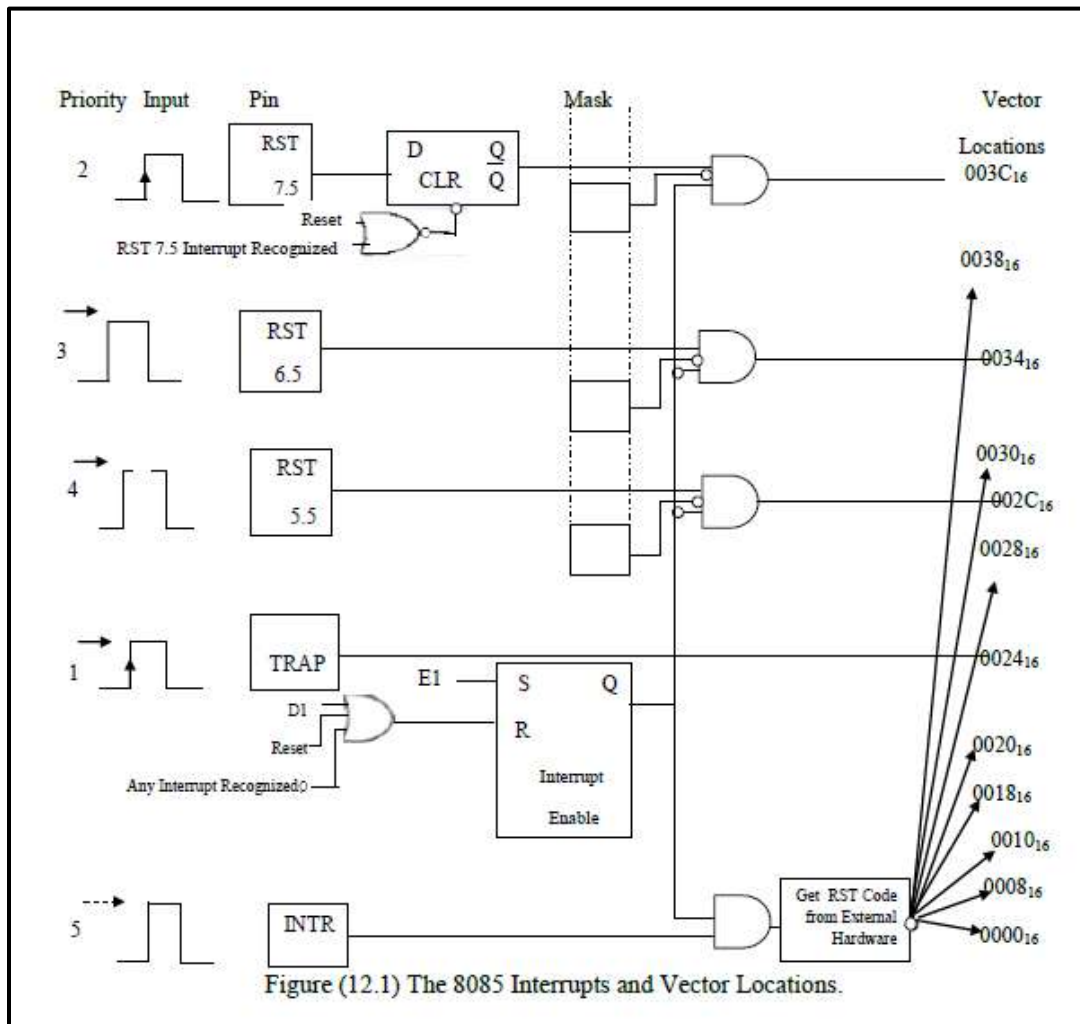
The 8085 has 4 Masked /Vectored interrupt inputs. RST 5.5, RST 6.5, RST 7.5. They are all maskable and are vectored automatically to the following table: The vectors for these interrupt fall in between the vectors for the RST instructions. That's why they have names like RST 5.5 (RST 5 and a half)

Interrupt	Vector
RST 5.5	002CH
RST 6.5	0034H
RST 7.5	003CH

Masking RST 5.5, RST 6.5 and RST 7.5

These three interrupts are masked at two levels:

- 1) Through the Interrupt Enable flip flop and the EI/DI instructions. The Interrupt Enable flip flop controls the whole maskable interrupt process.
- 2) Through individual mask flip flop that control the availability of the individual interrupts. These flip flops control the interrupts individually.



The 8085 Maskable/Vectored Interrupt Process

The interrupt process should be enabled using the EI instruction.

The 8085 checks for an interrupt during the execution of every instruction. If there is an interrupt, and if the interrupt is enabled using the interrupt mask, the microprocessor will complete the executing instruction, and reset the interrupt flip flop. The microprocessor then executes a call instruction that sends the execution to the appropriate location in the interrupt vector table.

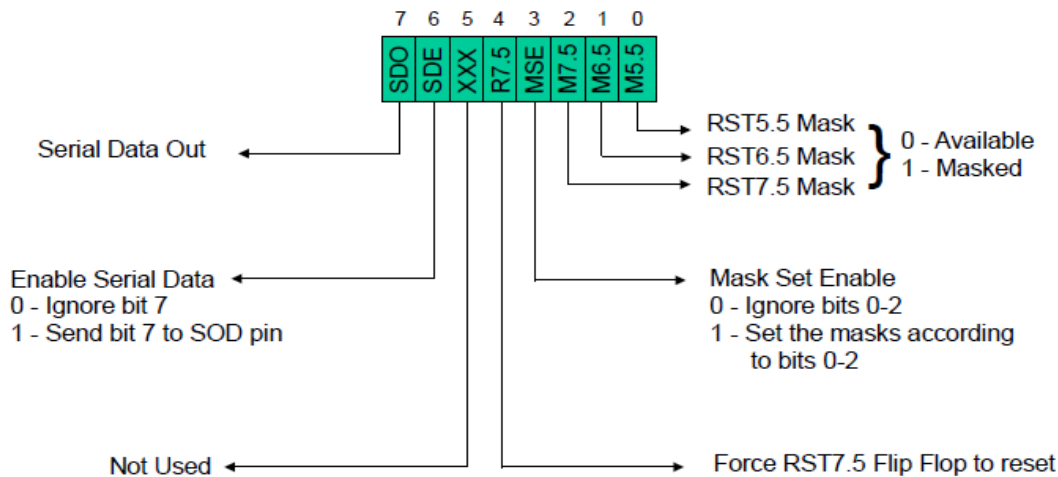
When the microprocessor executes the call instruction, it saves the address of the next instruction on the stack. The microprocessor jumps to the specific service routine. The service

routine must include the instruction EI to re-enable the interrupt process. At the end of the service routine, the RET instruction returns the execution to where the program was interrupted.

Manipulating the Masks

- The Interrupt Enable flip flop is manipulated using the EI/DI instructions.
- The individual masks for RST 5.5, RST 6.5 and RST 7.5 are manipulated using the SIM instruction. This instruction takes the bit pattern in the Accumulator and applies it to the interrupt mask enabling and disabling the specific interrupts.

SIM Instruction



The RST 7.5 interrupt is the only 8085 interrupt that has memory. If a signal on RST7.5 arrives while it is masked, a flip flop will remember the signal. When RST7.5 is unmasked, the microprocessor will be interrupted even if the device has removed the interrupt signal.

This flip flop will be automatically reset when the microprocessor responds to an RST 7.5 interrupt. Bit 4 of the accumulator in the SIM instruction allows explicitly resetting the RST 7.5 memory even if the microprocessor did not respond to it.

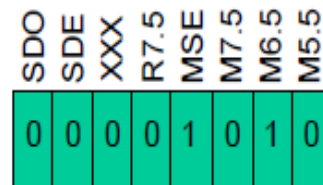
8085 Microprocessor Architecture

The SIM instruction can also be used to perform serial data transmission out of the 8085's SOD pin. One bit at a time can be sent out serially over the SOD pin. Bit 6 is used to tell the microprocessor whether or not to perform serial data transmission. If 0, then do not perform serial data transmission. If 1, then do. The value to be sent out on SOD has to be placed in bit 7 of the accumulator. Bit 5 is not used by the SIM instruction.

Example: Set the interrupt masks so that RST5.5 is enabled, RST6.5 is masked, and RST7.5 is enabled.

Sol.: First, determine the contents of the accumulator

- | | |
|-----------------------------|-----------|
| - Enable 5.5 | bit 0 = 0 |
| - Disable 6.5 | bit 1 = 1 |
| - Enable 7.5 | bit 2 = 0 |
| - Allow setting the masks | bit 3 = 1 |
| - Don't reset the flip flop | bit 4 = 0 |
| - Bit 5 is not used | bit 5 = 0 |
| - Don't use serial data | bit 6 = 0 |
| - Serial data is ignored | bit 7 = 0 |



Contents of accumulator are: 0AH

EI	; Enable interrupts including INTR
MVI A, 0A	; Prepare the mask to enable RST 7.5, and 5.5, disable 6.5
SIM	; Apply the settings RST masks

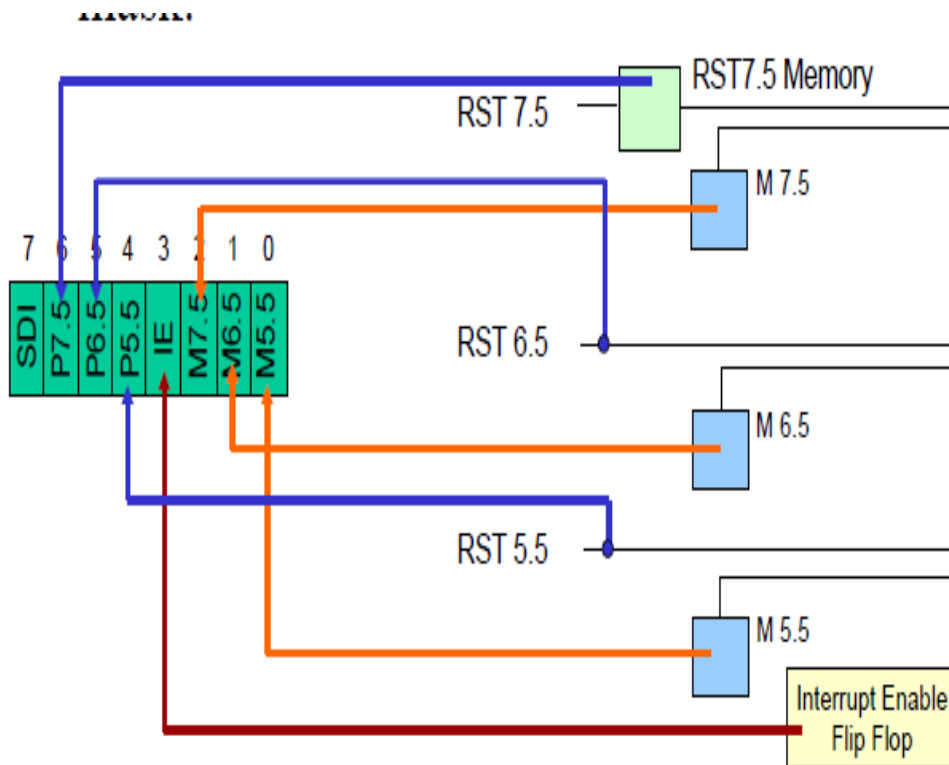
Triggering Levels

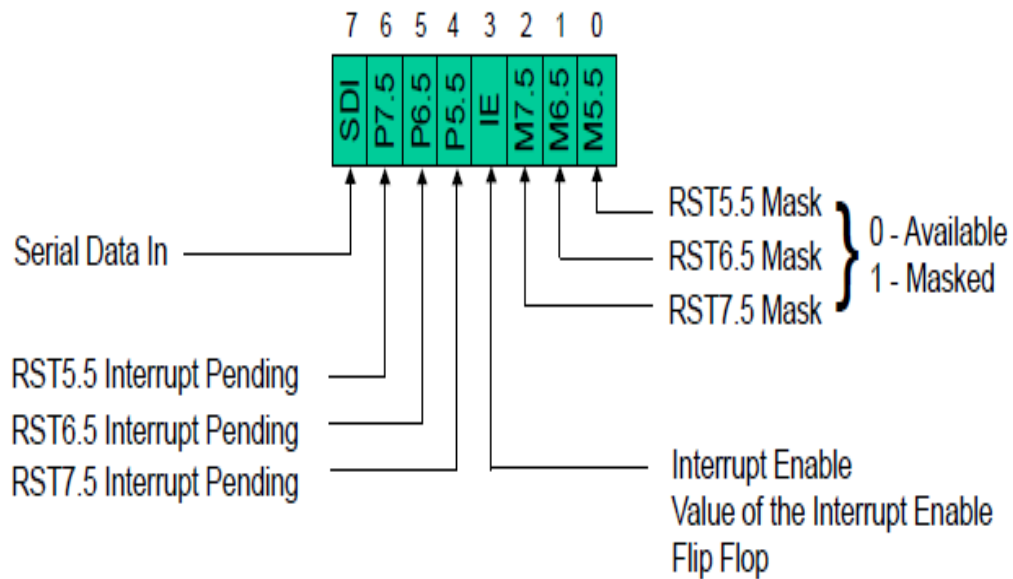
RST 7.5 is positive edge sensitive. When a positive edge appears on the RST7.5 line, logic 1 is stored in the flip-flop as a “pending” interrupt. Since the value has been stored in the flip flop, the line does not have to be high when the microprocessor checks for the interrupt to be recognized. The line must go to zero and back to one before a new interrupt is recognized.

RST 6.5 and RST 5.5 are level sensitive. The interrupting signal must remain present until the microprocessor checks for interrupts.

Determining the Current Mask Settings

RIM instruction: Read Interrupt Mask Load the accumulator with an 8-bit pattern showing the status of each interrupt pin and mask.





The RIM Instruction and the Masks

Bits 0-2 show the current setting of the mask for each of RST 7.5, RST 6.5 and RST 5.5. They return the contents of the three mask flip flops.

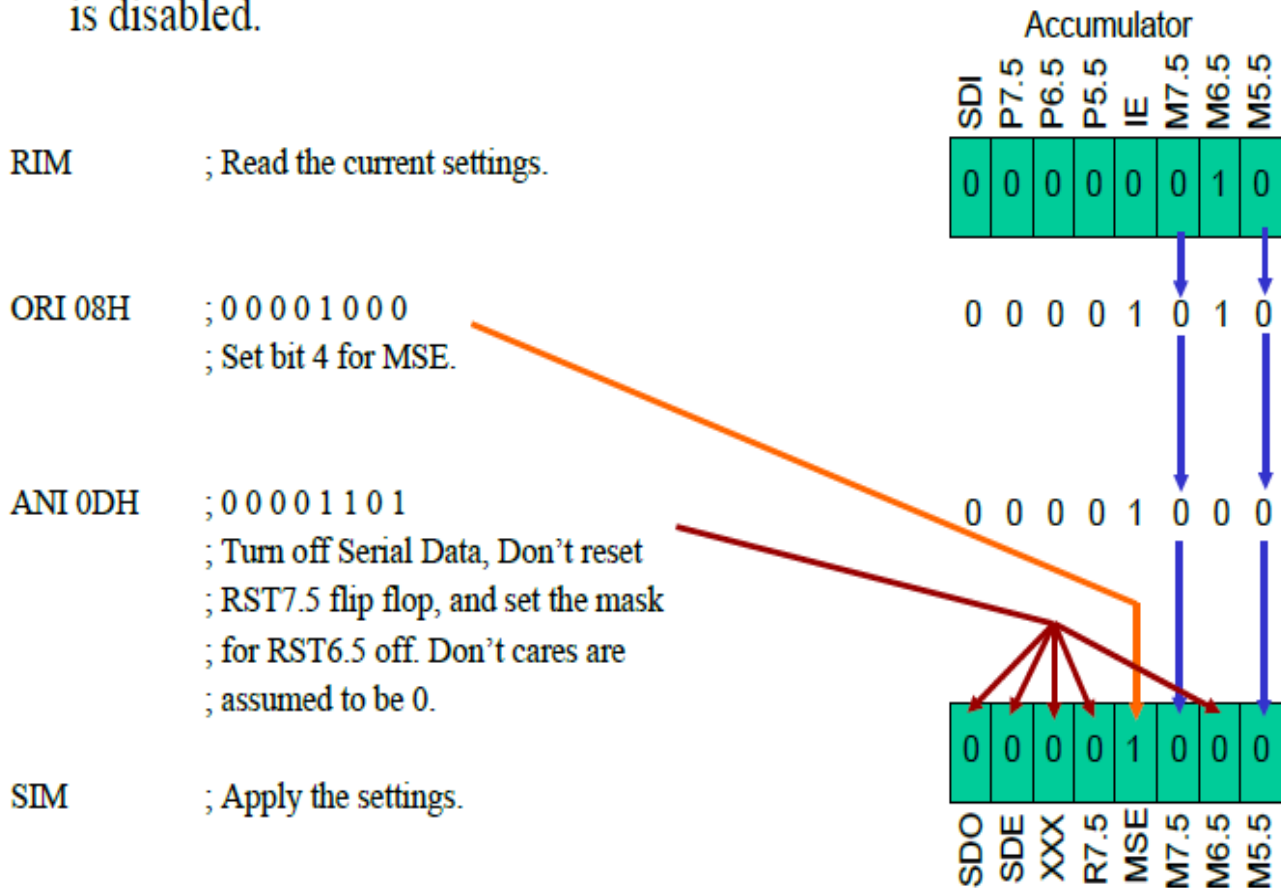
They can be used by a program to read the mask settings in order to modify only the right mask.

Bit 3 shows whether the maskable interrupt process is enabled or not. It returns the contents of the Interrupt Enable Flip Flop. It can be used by a program to determine whether or not interrupts are enabled.

Bits 4-6 show whether or not there are pending interrupts on RST 7.5, RST 6.5, and RST 5.5. Bits 4 and 5 return the current value of the RST5.5 and RST6.5 pins. Bit 6 returns the current value of the RST7.5 memory flip flop.

Bit 7 is used for Serial Data Input. The RIM instruction reads the value of the SID pin on the microprocessor and returns it in this bit.

- Assume the RST5.5 and RST7.5 are enabled and the interrupt process is disabled.



TRAP

TRAP is the only non-maskable interrupt. It does not need to be enabled because it cannot be disabled. It has the highest priority amongst interrupts. It is edge and level sensitive. It needs to be high and stay high to be recognized. Once it is recognized, it won't be recognized again until it goes low, then high again. TRAP is usually used for power failure and emergency shutoff.

The 8085 Interrupts

Interrupt Name	Maskable	Masking Method	Vectored	Memory	Triggering Method
INTR	Yes	DI / EI	No	No	Level Sensitive
RST 5.5 / RST 6.5	Yes	DI / EI SIM	Yes	No	Level Sensitive
RST 7.5	Yes	DI / EI SIM	Yes	Yes	Edge Sensitive
TRAP	No	None	Yes	No	Level & Edge Sensitive

Internally, the 8085 implements an interrupt priority scheme. The interrupts are ordered as follows:

TRAP
RST 7.5
RST 6.5
RST 5.5
INTR

However, TRAP has lower priority than the HLD signal used for DMA.

Programmable Interrupt Controller 8259 A

A programmable interrupt managing device. It manages 8 interrupt requests. It can vector an interrupt anywhere in memory without additional H/W. It can support 8 levels of interrupt priorities.

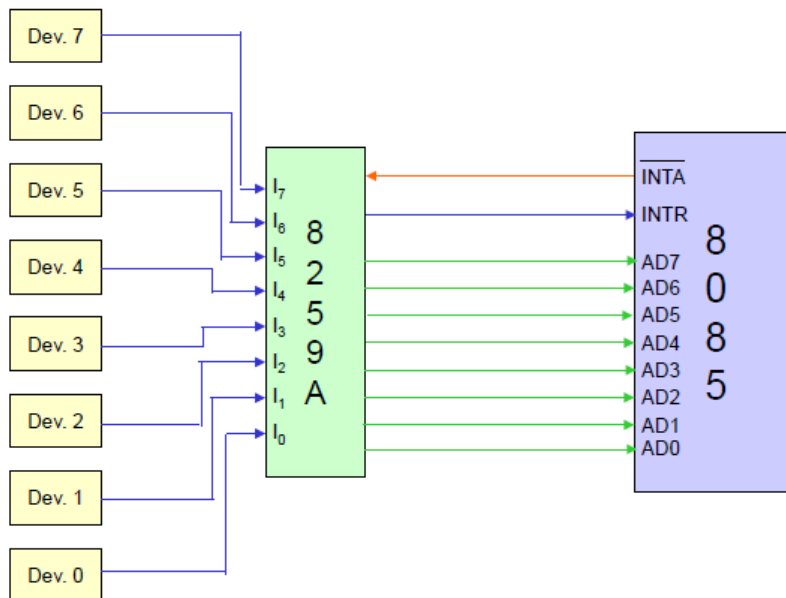
The priority scheme can be extended to 64 levels using a hierarchy of 8259 device.

The 8085 INTR interrupt scheme presented earlier has a few limitations:

- 1) The RST instructions are all vectored to memory page 00H, which is usually used for ROM.
- 2) It requires additional hardware to produce the RST instruction opcodes.
- 3) Priorities are set by hardware.

Therefore, we need a device like the 8259A to expand the priority scheme and allow mapping to pages other than 00H.

Interfacing the 8259A to the 8085



Operating of the 8259A

- 1) When one or more interrupts come in. The 8259A resolves the interrupt priorities based on its internal settings.
- 2) The 8259A sends an INTR signal to the microprocessor. The microprocessor responds with an INTA signal and turns off the interrupt enable flip flop.
- 3) The 8259A responds by placing the op-code for the CALL instruction (CDH) on the data bus.

- 4) When the microprocessor receives the op-code for CALL instead of RST, it recognizes that the device will be sending 16 more bits for the address.
- 5) The microprocessor sends a second INTA signal. The 8259A sends the high order byte of the ISR's address. The microprocessor sends a third INTA signal. The 8259A sends the low order byte of the ISR's address.
- 6) The microprocessor executes the CALL instruction and jumps to the ISR.

Direct Memory Access

This is a process where data is transferred between two peripherals directly without the involvement of the microprocessor. This process employs the HOLD pin on the microprocessor. The external DMA controller sends a signal on the HOLD pin to the microprocessor.

The microprocessor completes the current operation and sends a signal on HLDA and stops using the buses.

Once the DMA controller is done, it turns off the HOLD signal and the microprocessor takes back control of the buses.

H.W. Write a program to Set the interrupt masks so that RST6.5 is enabled, RST5.5 is masked, and RST7.5 is masked.

H.W. How can multiple devices to interrupt using the INTR line?

H.W. Define DMA, Vectored interrupt, TRAP.

H.W. Illustrate the procedure of the 8085 Non-Vectored Interrupt.

Reference:

8085 μ p architecture and programming_Gonkar